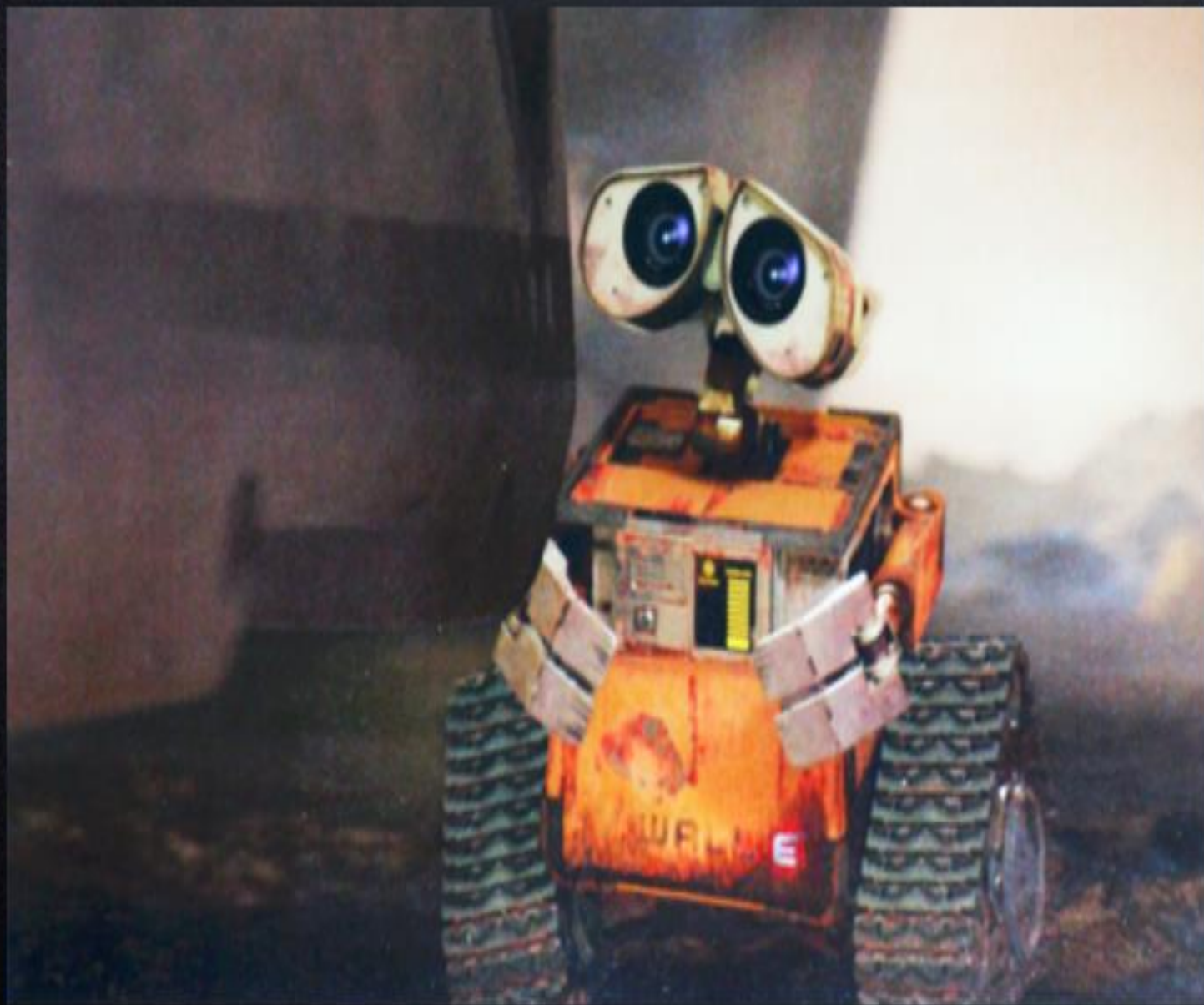
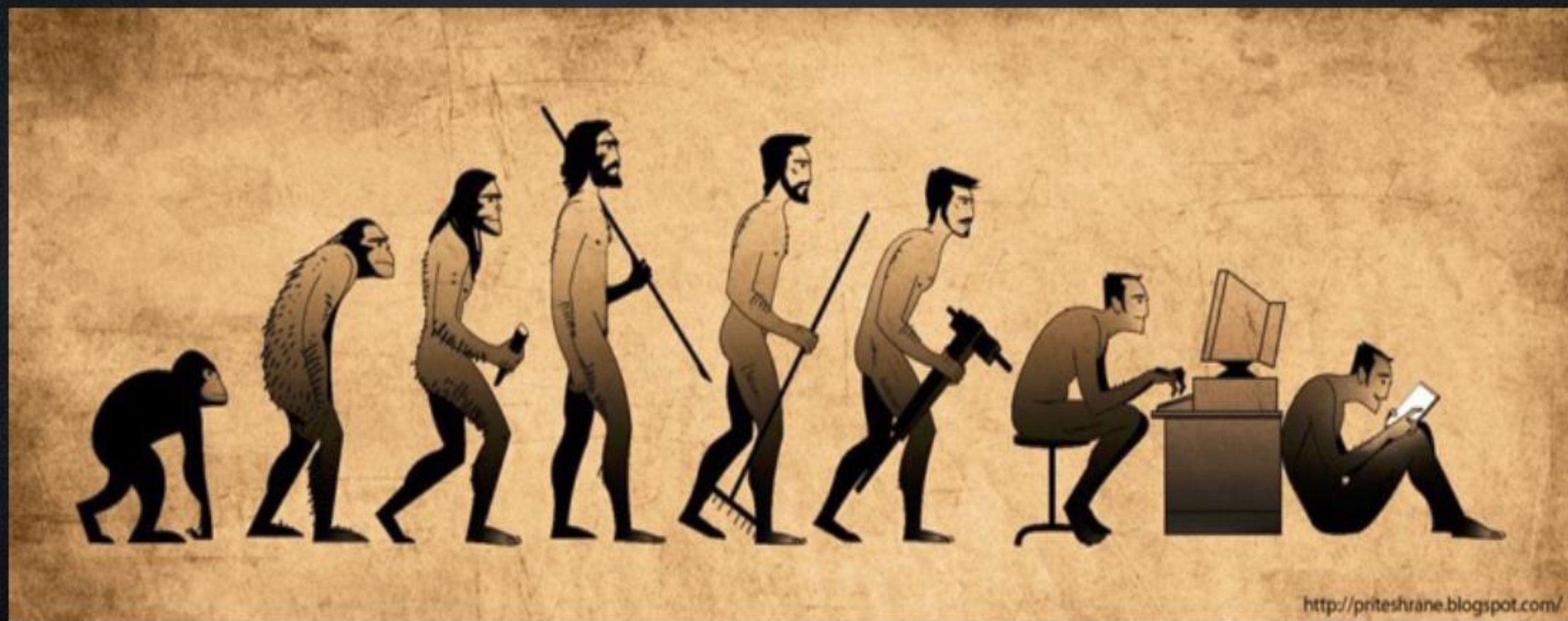




什么是
人工智能？



学习的能力，是智能的本质！

万物互联

万物智能

大数据时代





www.image-net.org

22K categories and **14M** images

- Animals
 - Bird
 - Fish
 - Mammal
 - Invertebrate
- Plants
 - Tree
 - Flower
- Food
- Materials
- Structures
 - Artifact
 - Tools
 - Appliances
 - Structures
- Person
- Scenes
 - Indoor
 - Geological Formation
- Sport Activities



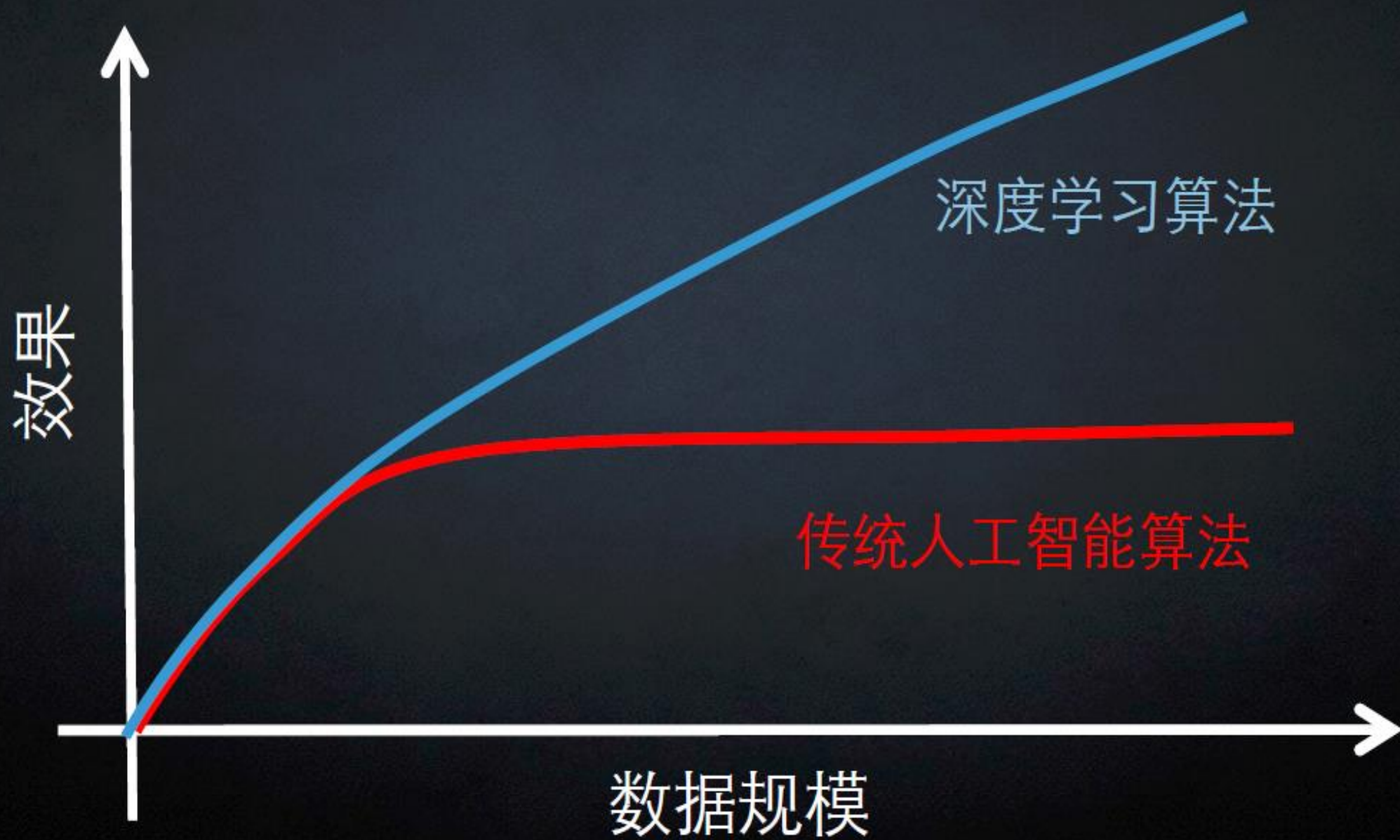
为什么围棋选手不信阿法狗呢？

结果呢？

4:1 李世石惨败

人工智能的时代已经来临

为什么人工智能技术这么厉害？



述说图片的故事



A yellow bus driving down a road with green trees and green grass in the background.



Living room with white couch and blue carpeting. The room in the apartment gets some afternoon sun.

这些字幕是深度学习程序写的



无人驾驶汽车：

1.物体检测

2.行人检测

3.标志识别

4.速度识别

。 。 。



黑科技：Image Transfer

Content + Style = Interesting thing

01. Take a picture



02. Pick a style



03. Enjoy the result



图像分类：计算机视觉核心任务



（假设我们有一系列的标签：狗，
猫，汽车，飞机。。。）

————→ 猫

图像在计算机中长什么样呢

一张图片被表示成三维数组的形式，每个像素的值从0到255

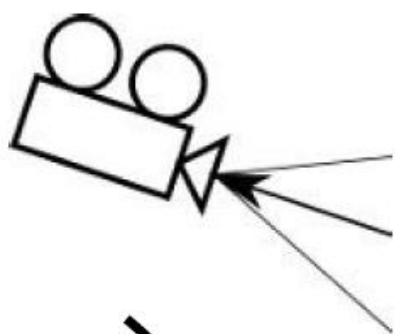
例如：300*100*3



08	02	22	97	38	15	00	40	00	75	04	05	07	78	52	12	50	77	91	56
49	49	99	40	17	81	18	57	60	87	17	40	98	43	69	48	04	56	62	00
81	49	31	73	55	79	14	29	93	71	40	67	55	08	30	03	49	13	36	65
52	70	95	23	04	60	11	42	69	24	68	56	01	32	56	71	37	02	36	91
22	31	16	71	51	67	83	89	41	92	36	54	22	40	40	28	66	33	13	80
24	47	38	60	99	03	45	02	44	75	33	53	78	36	84	20	35	17	12	50
32	98	81	28	64	23	67	10	26	38	40	67	59	54	70	66	18	38	64	70
67	26	20	68	02	62	12	20	95	63	94	39	63	08	40	91	66	49	94	21
24	55	58	05	66	73	99	26	97	17	78	78	96	83	14	88	34	89	63	72
21	36	23	09	75	00	76	44	20	45	35	14	00	61	33	97	34	31	33	95
78	17	53	28	22	75	31	67	15	94	03	80	04	62	16	14	09	53	56	92
16	39	05	42	96	35	31	47	55	58	88	24	00	17	54	24	36	29	85	57
86	56	00	48	35	71	89	07	05	44	44	37	44	60	21	58	51	54	17	58
19	80	81	68	05	94	47	69	28	73	92	13	86	52	17	77	04	89	55	40
04	52	08	83	97	35	99	16	07	97	57	32	16	24	26	79	33	27	98	66
50	86	68	87	57	62	20	72	03	46	33	67	46	55	12	32	63	93	53	49
04	42	16	73	35	85	39	11	24	94	72	18	08	46	29	32	40	62	76	36
20	69	36	41	72	30	23	88	34	08	89	69	82	67	59	85	74	04	36	16
20	73	35	29	78	31	90	01	74	31	49	71	48	88	81	16	23	57	05	54
01	70	54	71	83	51	54	69	16	92	33	48	61	43	52	01	89	15	67	48

What the computer sees

挑战：照射角度



08	02	22	97	38	15	00	40	00	75	04	03	07	78	52	12	50	77	97	68
49	49	99	40	17	81	18	57	60	87	17	40	98	43	69	45	04	56	62	00
81	49	31	73	55	79	14	29	93	71	40	67	52	88	30	03	49	13	36	65
52	70	95	23	04	60	11	42	65	41	88	56	01	32	56	71	37	02	36	91
22	31	16	71	51	63	83	89	41	92	36	54	22	40	40	28	66	33	13	80
24	47	39	00	99	03	45	02	44	75	33	53	78	36	84	20	35	17	12	50
32	98	81	28	64	23	67	10	26	38	40	67	59	54	70	66	18	38	64	70
67	26	20	68	02	62	12	20	95	63	94	39	63	08	40	91	66	49	94	21
24	55	58	05	66	73	99	26	97	17	78	78	96	83	14	88	34	89	63	72
21	36	23	09	75	00	76	44	20	45	35	14	00	61	33	97	34	31	33	95
78	17	53	28	22	75	31	67	15	94	03	80	04	62	16	14	09	53	56	92
16	39	05	42	96	35	31	47	55	58	88	24	00	17	54	24	36	29	85	57
86	56	00	48	35	71	89	07	05	44	44	37	44	60	21	58	51	54	17	58
19	80	81	68	05	94	47	69	28	73	92	13	86	52	17	77	04	89	55	40
04	52	08	83	97	35	99	16	07	97	57	32	16	26	26	79	33	27	98	66
88	86	68	87	57	62	20	72	03	46	33	67	46	55	12	32	63	93	53	69
04	42	16	73	38	84	39	11	24	94	72	18	08	46	29	32	40	62	76	36
20	69	36	41	72	30	23	88	34	62	93	69	82	67	59	85	74	04	36	16
20	73	35	29	78	31	90	01	74	31	49	71	48	26	61	16	23	57	05	54
01	70	54	71	83	51	54	69	16	92	33	48	61	43	52	01	89	28	62	48

挑战：光照强度



挑战：形状改变



挑战：部分遮蔽



挑战：背景混入



Viewpoint variation



Scale variation



Deformation



Occlusion



Illumination conditions



Background clutter



Intra-class variation



常规套路:

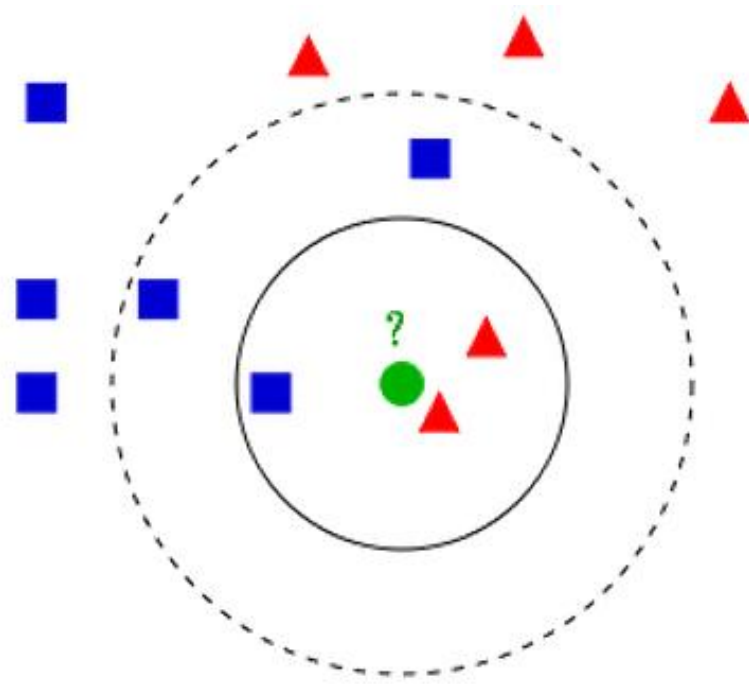
1. 收集数据并给定标签
2. 训练一个分类器
3. 测试, 评估

```
def train(train_images, train_labels):  
    # build a model for images -> labels...  
    return model  
  
def predict(model, test_images):  
    # predict test_labels using the model...  
    return test_labels
```

Example training set



K-近邻



- 如果 $K=3$ ，绿色圆点的最近的3个邻居是2个红色小三角形和1个蓝色小正方形，少数从属于多数，基于统计的方法，判定绿色的这个待分类点属于红色的三角形一类。
- 如果 $K=5$ ，绿色圆点的最近的5个邻居是2个红色三角形和3个蓝色的正方形，还是少数从属于多数，基于统计的方法，判定绿色的这个待分类点属于蓝色的正方形一类。

K-近邻

对于未知类别属性数据集中的点：

1. 计算已知类别数据集中的点与当前点的距离
2. 按照距离依次排序
3. 选取与当前点距离最小的K个点
4. 确定前K个点所在类别的出现概率
5. 返回前K个点出现频率最高的类别作为当前点预测分类。

K-近邻

概述：

KNN 算法本身简单有效，它是一种 lazy-learning 算法。

分类器不需要使用训练集进行训练，训练时间复杂度为0。

KNN 分类的计算复杂度和训练集中的文档数目成正比，也就是说，如果训练集中文档总数为 n ，那么 KNN 的分类时间复杂度为 $O(n)$ 。

K-近邻

K 值的选择，距离度量和分类决策规则是该算法的三个基本要素

问题：该算法在分类时有个主要的不足是，当样本不平衡时，如一个类的样本容量很大，而其他类样本容量很小时，有可能导致当输入一个新样本时，该样本的 K 个邻居中大容量类的样本占多数

解决：不同的样本给予不同权重项

数据库样例: CIFAR-10

10类标签

50000个训练数据

10000个测试数据

大小均为 $32*32$

airplane



automobile



bird



cat



deer



dog



frog



horse



ship



truck



如何计算的呢:

L1 distance:

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

test image

56	32	10	18
90	23	128	133
24	26	178	200
2	0	255	220

-

training image

10	20	24	17
8	10	89	100
12	16	178	170
4	32	233	112

=

pixel-wise absolute value differences

46	12	14	1
82	13	39	33
12	10	0	30
2	32	22	108

add
→ 456

测试结果:



最近邻代码:

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

记录所有训练数据

对于每一个测试数据找出与其L1距离最小的样本的标签，作为它的标签

超参数:

L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

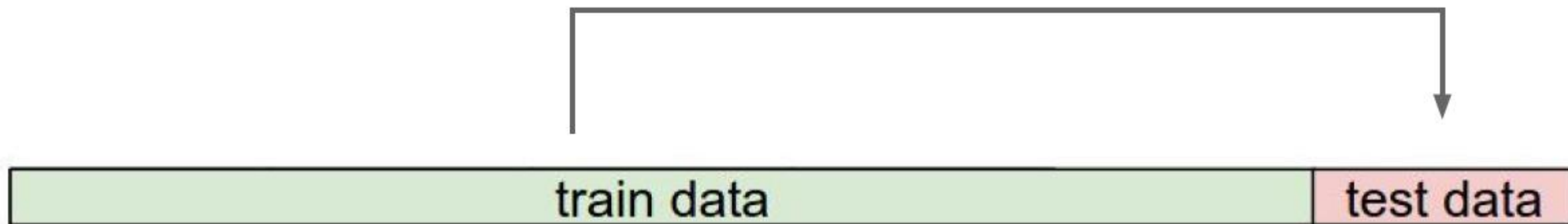
L2 (Euclidean) distance

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$

问题：

- 1.对于距离如何设定？
- 2.对于K近邻的K该如何选择？
- 3.如果有的话，其它的超参数该怎么设定呢？

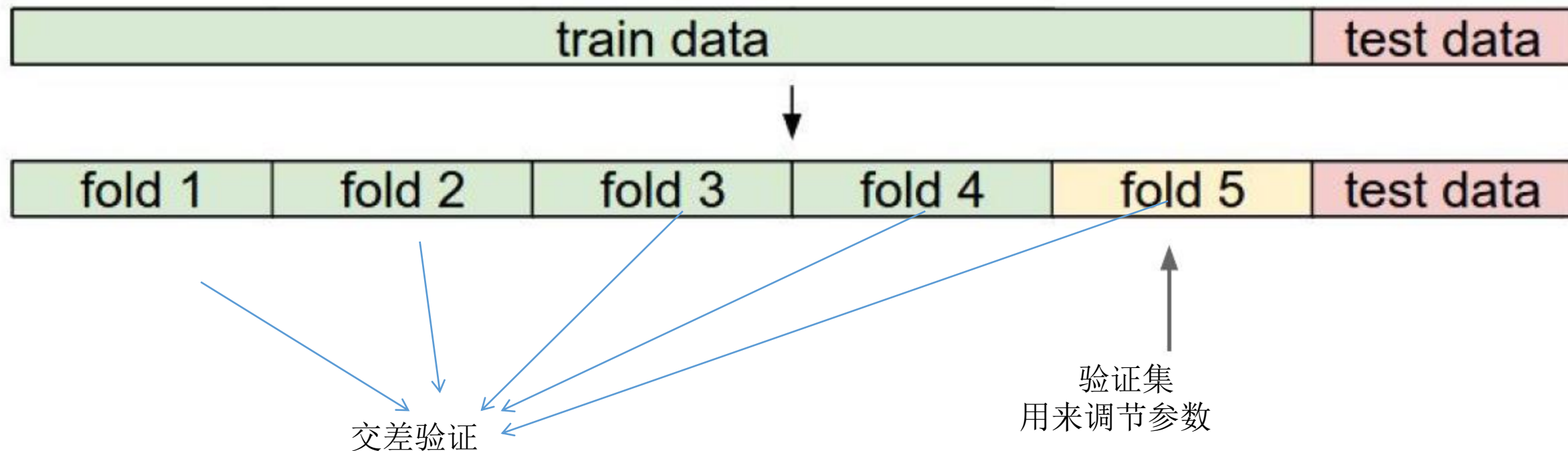
找到最好的参数：



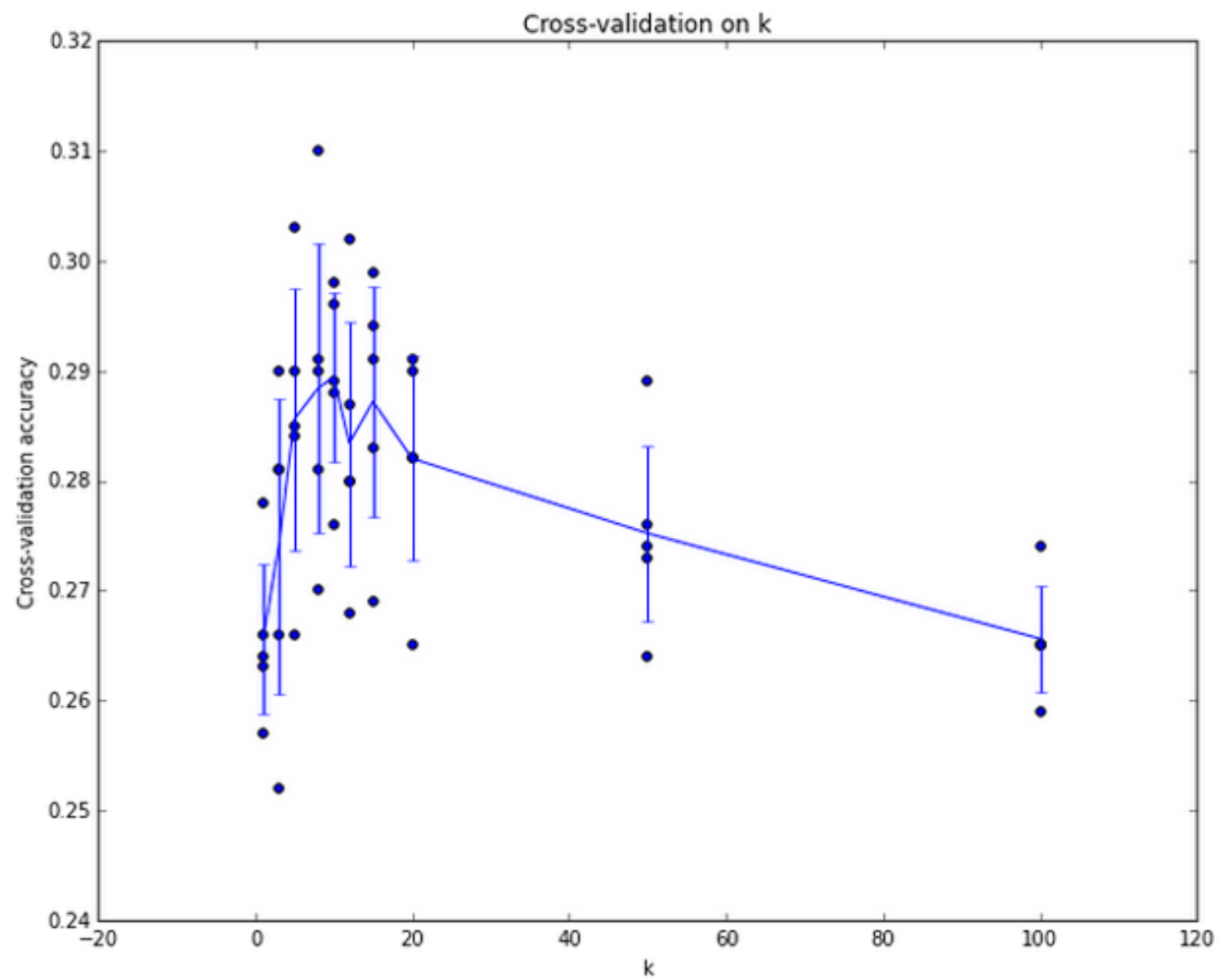
多次用测试数据试验，找到做好的一组参数组合？

错误的想法，测试数据只能最终用

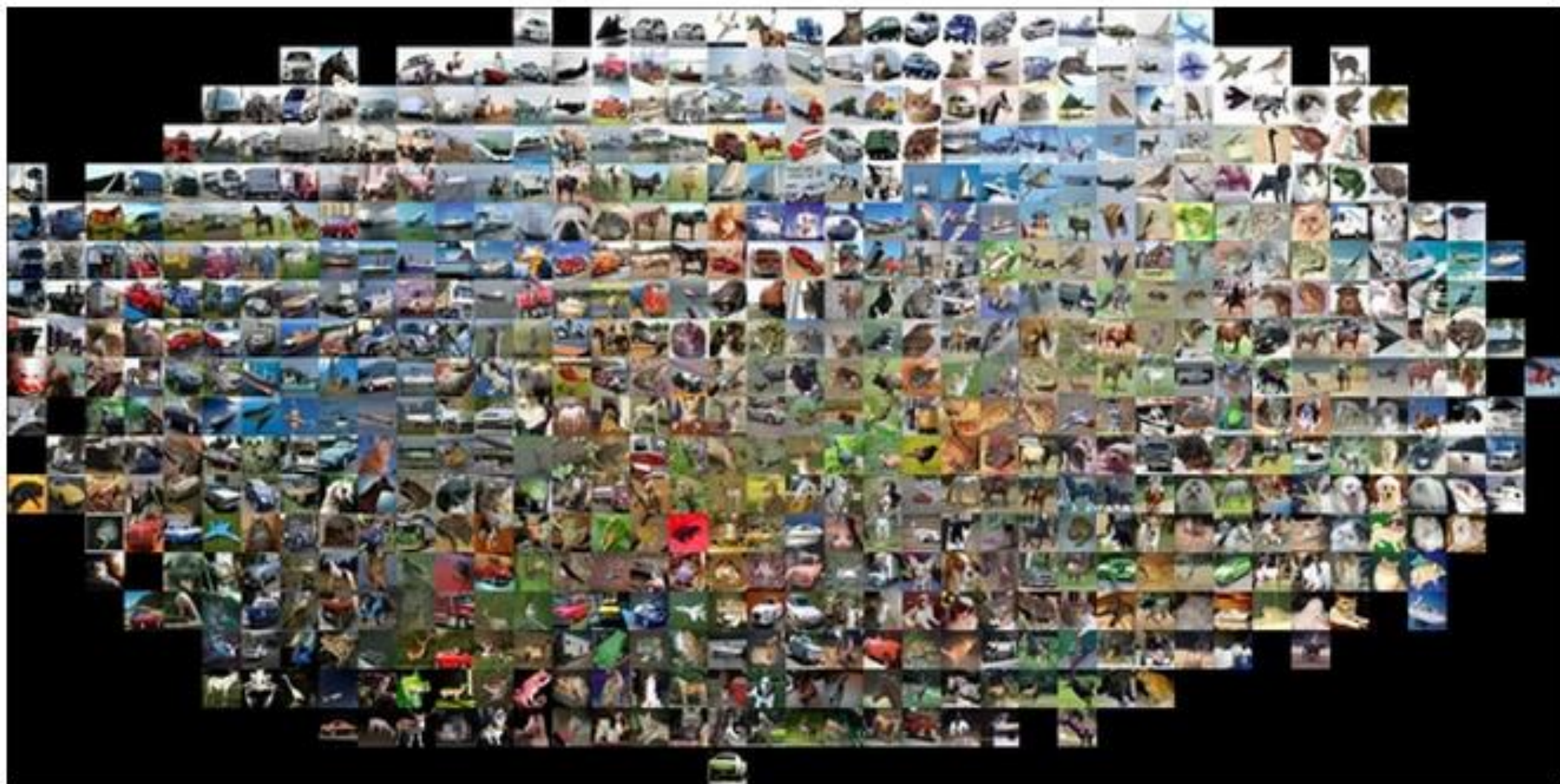
测试结果:



K-近邻



背景主导



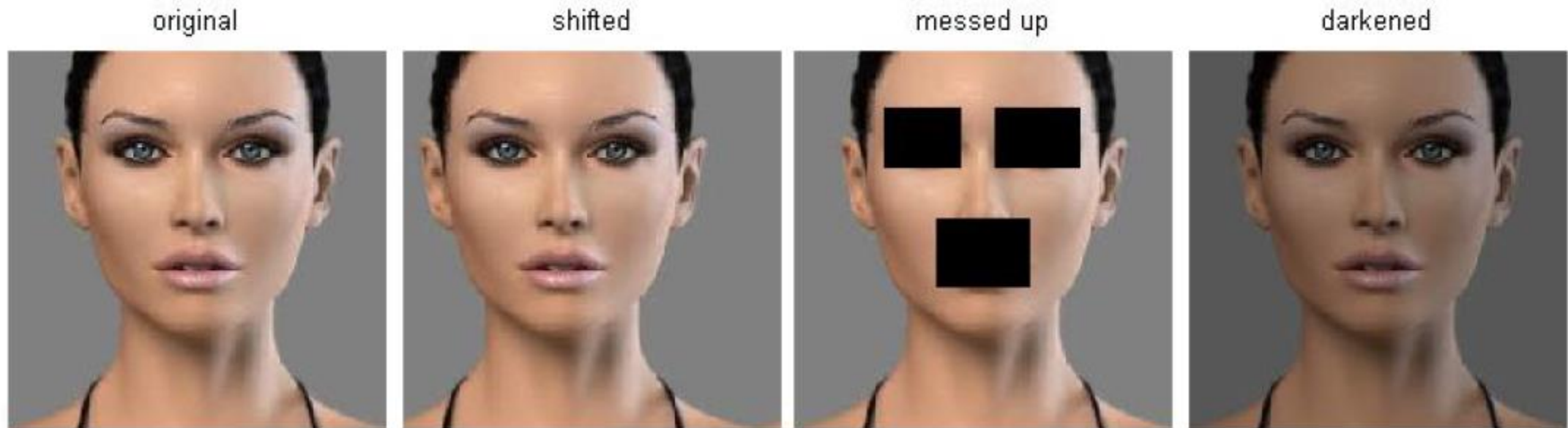
K-近邻

- 1.选取超参数的正确方法是：将原始训练集分为训练集和**验证集**，我们在验证集上尝试不同的超参数，最后保留表现最好那个
- 2.如果训练数据量不够，使用**交叉验证**方法，它能帮助我们在选取最优超参数的时候减少噪音。
- 3.一旦找到最优的超参数，就让算法以该参数在测试集跑且只跑一次，并根据测试结果评价算法。
- 4.最近邻分类器能够在**CIFAR-10**上得到将近**40%**的准确率。该算法简单易实现，但需要存储所有训练数据，并且在测试的时候过于耗费计算能力
- 5.最后，我们知道了仅仅使用**L1**和**L2**范数来进行像素比较是不够的，图像更多的是按照背景和颜色被分类，而不是语义主体分身。

K-近邻

1. 预处理你的数据：对你数据中的特征进行归一化（`normalize`），让其具有零平均值（`zero mean`）和单位方差（`unit variance`）。
2. 如果数据是高维数据，考虑使用降维方法，比如PCA
3. 将数据随机分入训练集和验证集。按照一般规律，70%-90% 数据作为训练集
4. 在验证集上调优，尝试足够多的k值，尝试L1和L2两种范数计算方式。

K近邻: never used



(不同的变换和原图具有相同的L2距离)

线性分类:



(32x32x3)

image parameters

$$f(\mathbf{x}, \mathbf{W})$$



每个类别的得分

得分函数:

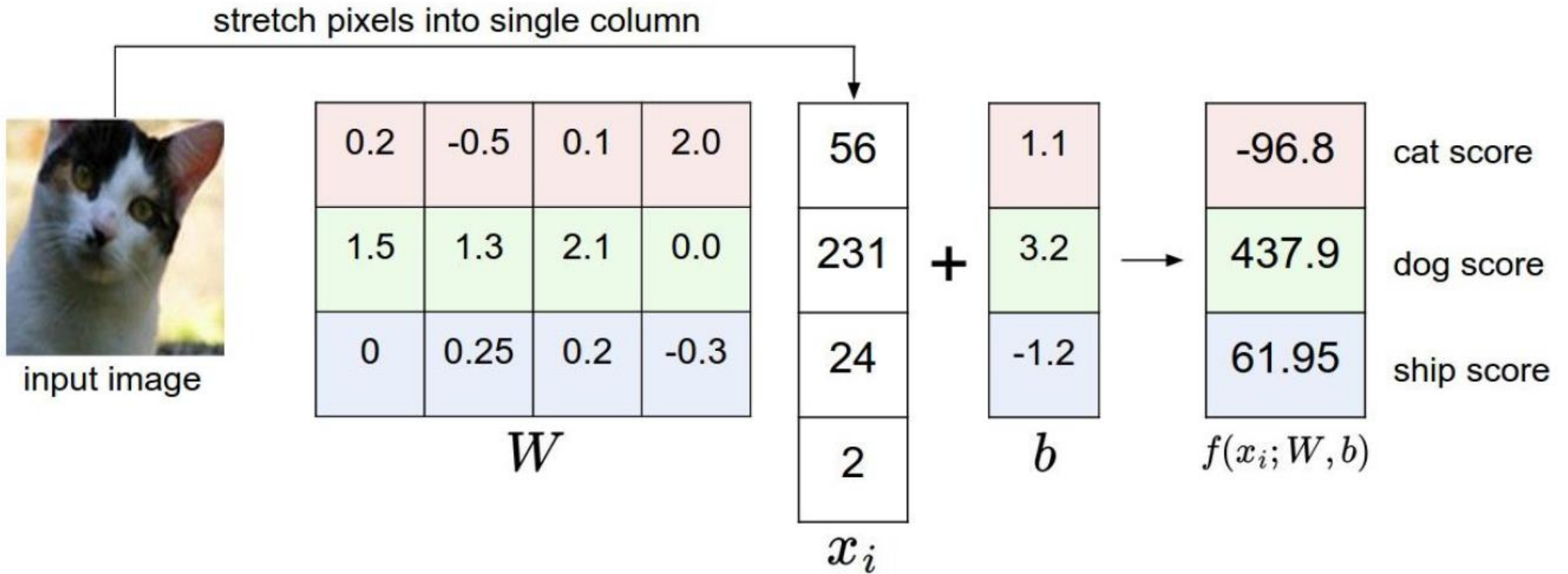


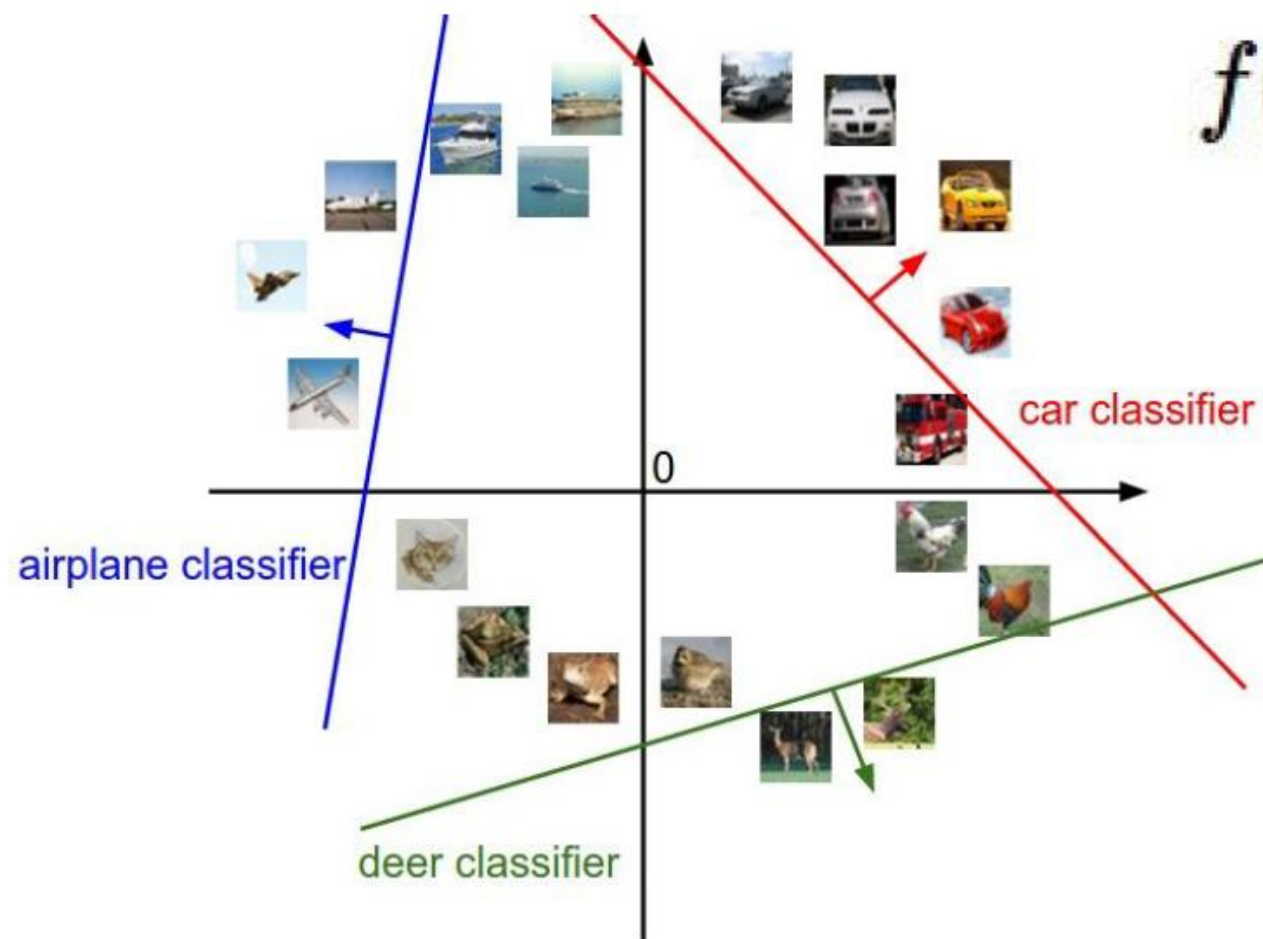
[32x32x3]

$$\boxed{f(x, W)}_{10 \times 1} = \boxed{W}_{10 \times 3072} \boxed{x}_{3072 \times 1} \quad \boxed{(+b)}_{10 \times 1}$$

权重

实例





$$f(x_i, W, b) = Wx_i + b$$

损失函数：



cat

3.2

1.3

2.2

car

5.1

4.9

2.5

frog

-1.7

2.0

-3.1

损失函数:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1

$$\begin{aligned} &= \max(0, 5.1 - 3.2 + 1) \\ &\quad + \max(0, -1.7 - 3.2 + 1) \\ &= \max(0, 2.9) + \max(0, -3.9) \\ &= 2.9 + 0 \\ &= 2.9 \\ &= \max(0, 1.3 - 4.9 + 1) \\ &\quad + \max(0, 2.0 - 4.9 + 1) \\ &= \max(0, -2.6) + \max(0, -1.9) \\ &= 0 + 0 \\ &= 0 \\ &= \max(0, 2.2 - (-3.1) + 1) \\ &\quad + \max(0, 2.5 - (-3.1) + 1) \\ &= \max(0, 5.3) + \max(0, 5.6) \\ &= 5.3 + 5.6 \\ &= 10.9 \end{aligned}$$

损失函数:



损失函数:

$$f(x, W) = Wx$$

$$L = \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, f(x_i; W)_j - f(x_i; W)_{y_i} + 1)$$

$$x = [1, 1, 1, 1]$$

$$w_1 = [1, 0, 0, 0]$$

$$w_2 = [0.25, 0.25, 0.25, 0.25]$$

正则化:

正则化惩罚项

$$L = \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, f(x_i; W)_j - f(x_i; W)_{y_i} + 1) + \lambda R(W)$$

L2正则化:

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

$$x = [1, 1, 1, 1]$$

$$w_1 = [1, 0, 0, 0]$$

$$w_2 = [0.25, 0.25, 0.25, 0.25]$$

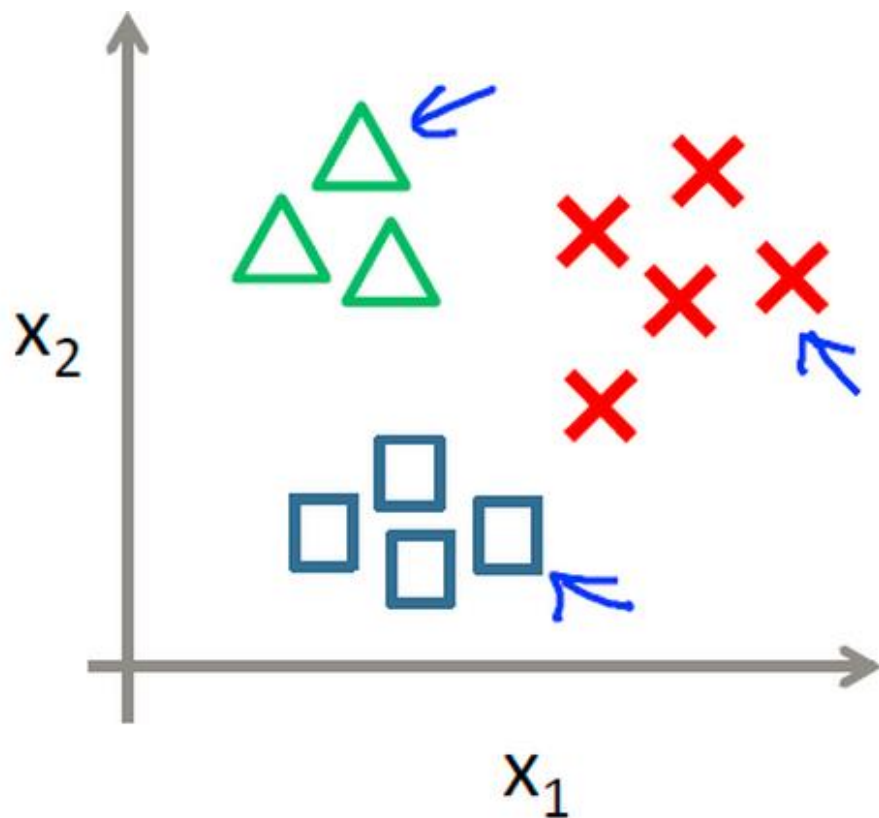
$$w_1^T x = w_2^T x = 1$$

损失函数终极版：

$$L = \frac{1}{N} \sum_i \sum_{j \neq y_i} [\max(0, f(x_i; W)_j - f(x_i; W)_{y_i} + \Delta)] + \lambda \sum_k \sum_l W_{k,l}^2$$

Softmax分类器:

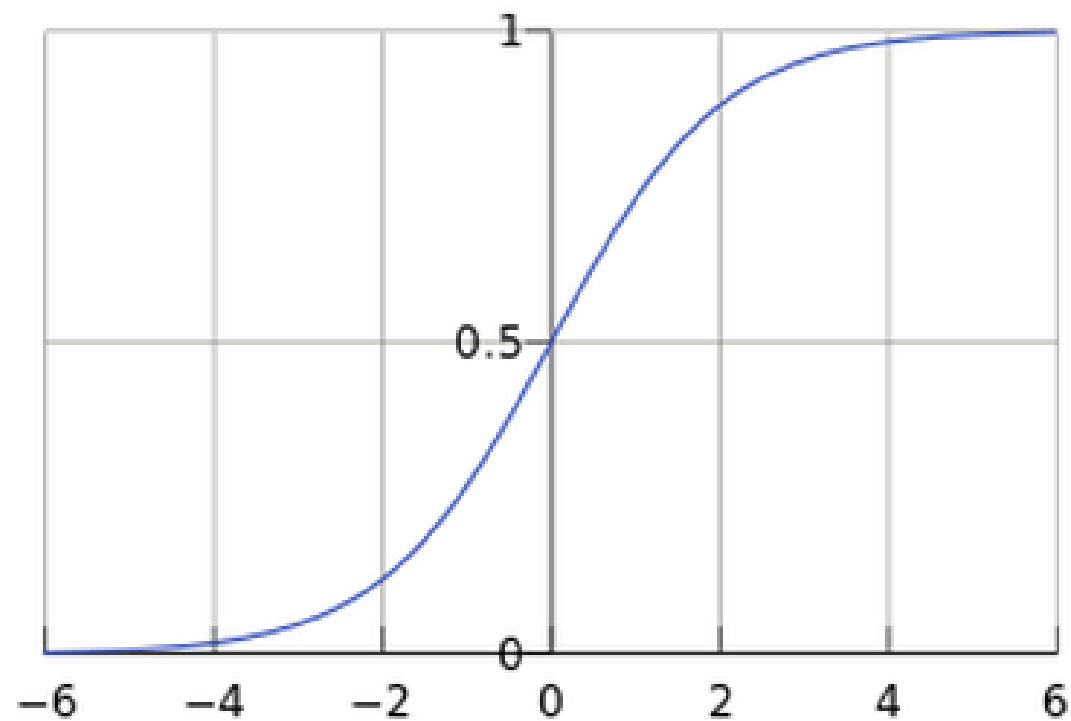
多类别分类



Softmax的输出是概率

Sigmoid函数:

$$g(z) = \frac{1}{1 + e^{-z}}$$



Softmax分类器:

Softmax的输出（归一化的分类概率）

损失函数：交叉熵损失（**cross-entropy loss**）

$$Li = -\log\left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}}\right)$$

$$f_j(z) = \frac{e^{z_j}}{\sum_k e^{z_k}}$$
 被称作**softmax** 函数

其输入值是一个向量，向量中元素为任意实数的评分值

输出一个向量，其中每个元素值在0到1之间，且所有元素之和为1

softmax:



$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

where

$$s = f(x_i; W)$$

$$L_i = -\log P(Y = y_i | X = x_i)$$

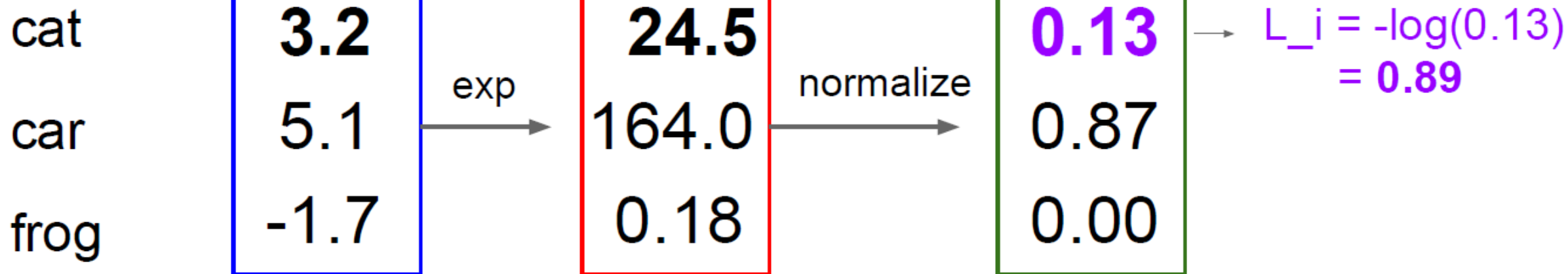
cat	3.2
car	5.1
frog	-1.7

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

Softmax实例:



$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$



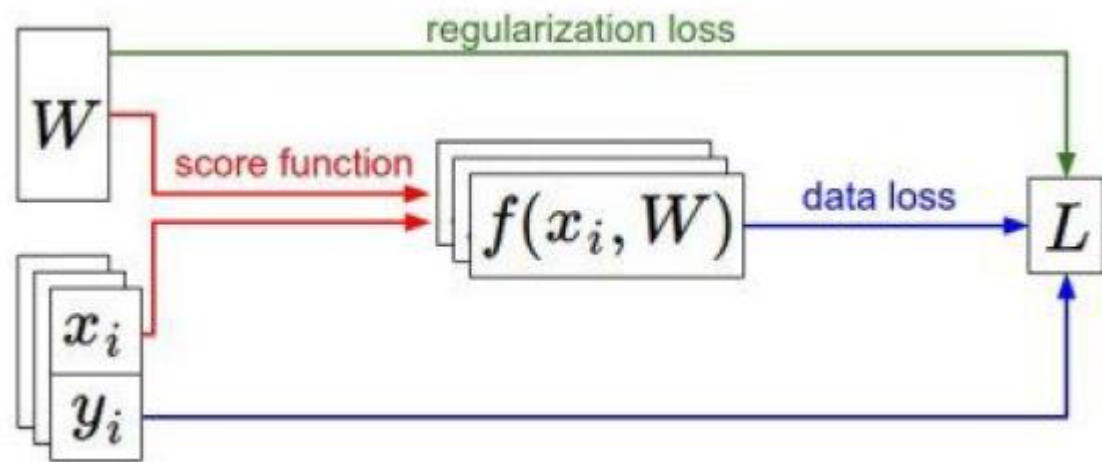
最优化:

$$s = f(x; W) \stackrel{\text{e.g.}}{=} Wx$$

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right) \quad \text{Softmax}$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \quad \text{SVM}$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + R(W) \quad \text{Full loss}$$



粗暴的想法：

```
# assume X_train is the data where each column is an example (e.g. 3073 x 50,000)
# assume Y_train are the labels (e.g. 1D array of 50,000)
# assume the function L evaluates the loss function

bestloss = float("inf") # Python assigns the highest possible float value
for num in xrange(1000):
    W = np.random.randn(10, 3073) * 0.0001 # generate random parameters
    loss = L(X_train, Y_train, W) # get the loss over the entire training set
    if loss < bestloss: # keep track of the best solution
        bestloss = loss
        bestW = W
    print 'in attempt %d the loss was %f, best %f' % (num, loss, bestloss)

# prints:
# in attempt 0 the loss was 9.401632, best 9.401632
# in attempt 1 the loss was 8.959668, best 8.959668
# in attempt 2 the loss was 9.044034, best 8.959668
# in attempt 3 the loss was 9.278948, best 8.959668
# in attempt 4 the loss was 8.857370, best 8.857370
# in attempt 5 the loss was 8.943151, best 8.857370
# in attempt 6 the loss was 8.605604, best 8.605604
# ... (truncated: continues for 1000 lines)
```

粗暴想法的结果:

```
# Assume X_test is [3073 x 10000], Y_test [10000 x 1]
scores = Wbest.dot(Xte_cols) # 10 x 10000, the class scores for all test examples
# find the index with max score in each column (the predicted class)
Yte_predict = np.argmax(scores, axis = 0)
# and calculate accuracy (fraction of predictions that are correct)
np.mean(Yte_predict == Yte)
# returns 0.1555
```

找到山坡最低点：



找到山坡最低点:

```
W = np.random.randn(10, 3073) * 0.001 # 生成随机初始W
bestloss = float("inf")
for i in xrange(1000):
    step_size = 0.0001
    Wtry = W + np.random.randn(10, 3073) * step_size
    loss = L(Xtr_cols, Ytr, Wtry)
    if loss < bestloss:
        W = Wtry
        bestloss = loss
    print 'iter %d loss is %f' % (i, bestloss)
```

跟隨梯度：

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

```
def eval_numerical_gradient(f, x):  
    """  
    a naive implementation of numerical gradient of f at x  
    - f should be a function that takes a single argument  
    - x is the point (numpy array) to evaluate the gradient at  
    """  
  
    fx = f(x) # evaluate function value at original point  
    grad = np.zeros(x.shape)  
    h = 0.00001  
  
    # iterate over all indexes in x  
    it = np.nditer(x, flags=['multi_index'], op_flags=['readwrite'])  
    while not it.finished:  
  
        # evaluate function at x+h  
        ix = it.multi_index  
        old_value = x[ix]  
        x[ix] = old_value + h # increment by h  
        fxh = f(x) # evaluate f(x + h)  
        x[ix] = old_value # restore to previous value (very important!)  
  
        # compute the partial derivative  
        grad[ix] = (fxh - fx) / h # the slope  
        it.iternext() # step to next dimension  
  
    return grad
```

梯度下降:

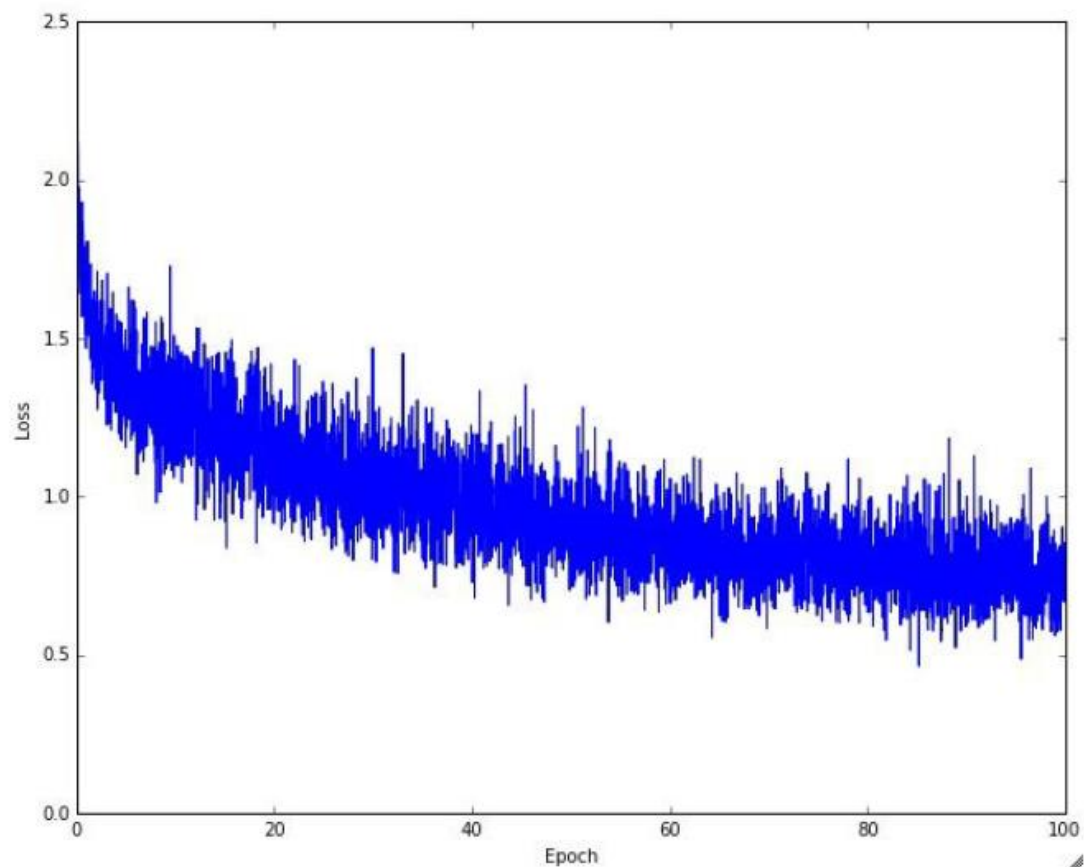
```
while True:  
    weights_grad = evaluate_gradient(loss_fun, data, weights)  
    weights += - step_size * weights_grad # perform parameter update
```

梯度下降：

```
while True:  
    data_batch = sample_training_data(data, 256) # sample 256 examples  
    weights_grad = evaluate_gradient(loss_fun, data_batch, weights)  
    weights += - step_size * weights_grad # perform parameter update
```

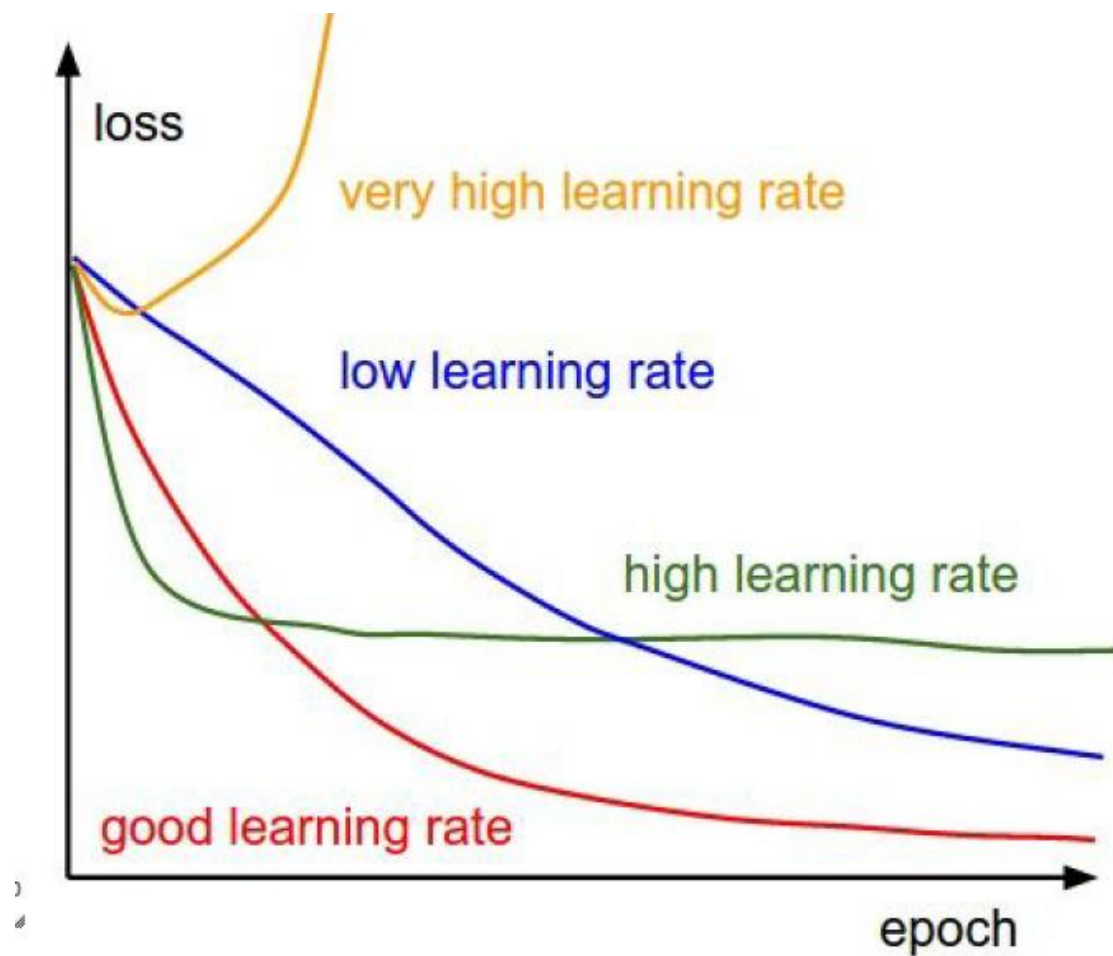
Bachsize通常是2的整数倍（32， 64， 128）

梯度下降:



训练网络时的LOSS值可视化结果

学习率:



训练网络时的LOSS值可视化结果

学习率:

```
while True:
```

```
    data_batch = sample_training_data(data, 256) # sample 256 examples
```

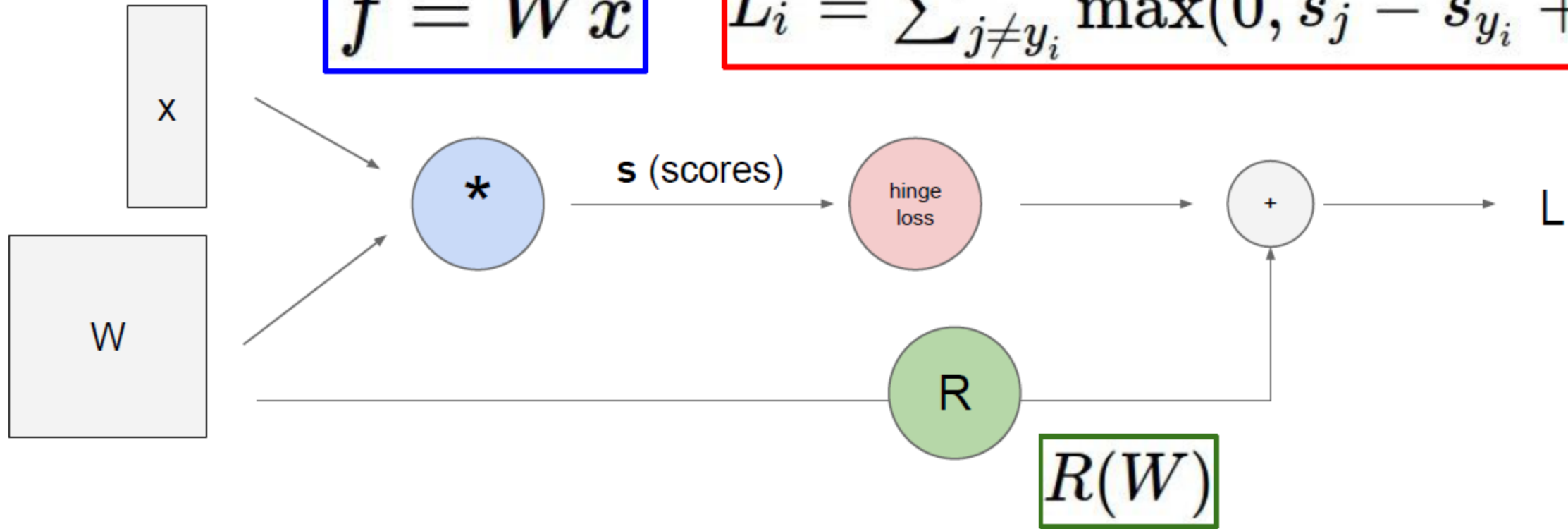
```
    weights_grad = evaluate_gradient(loss_fun, data_batch, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```

反向传播:

$$f = Wx$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$



反向传播:

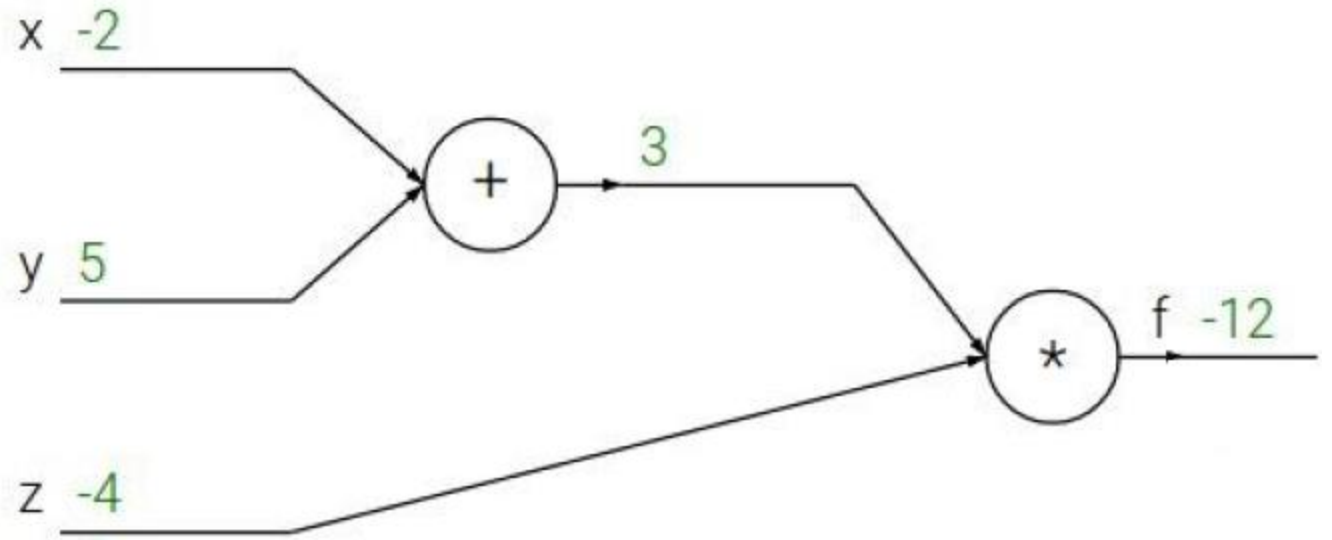
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

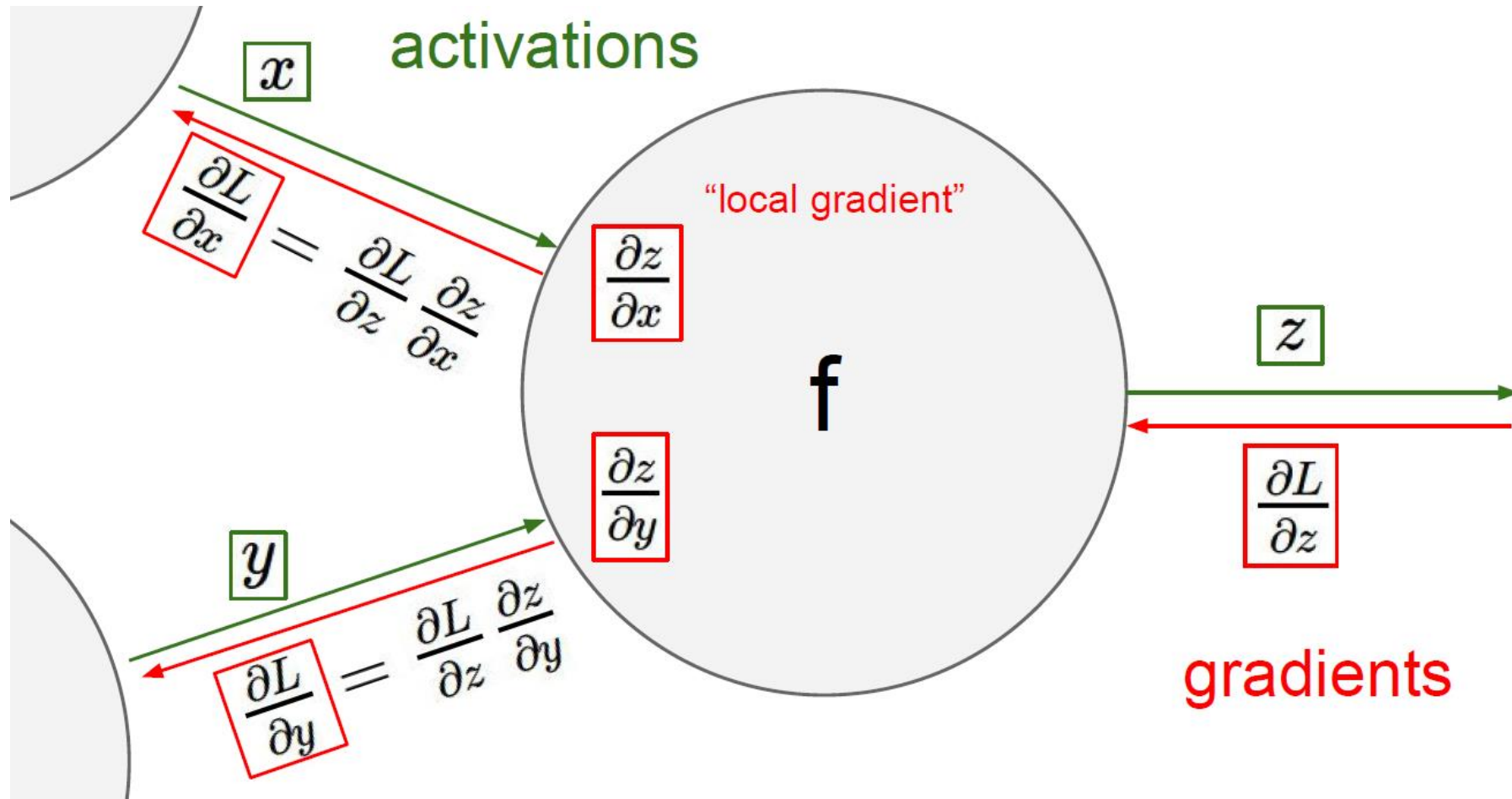
$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

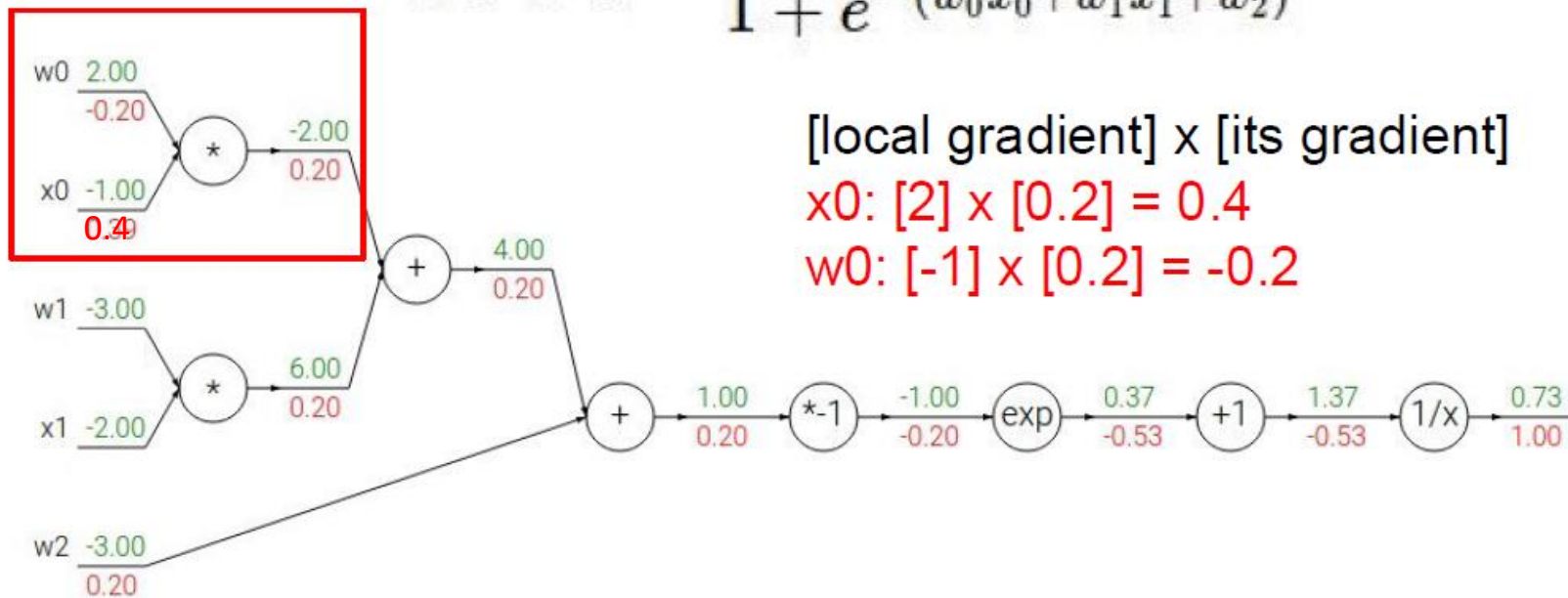


反向传播:



反向传播:

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



[local gradient] x [its gradient]

$$x_0: [2] \times [0.2] = 0.4$$

$$w_0: [-1] \times [0.2] = -0.2$$

$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

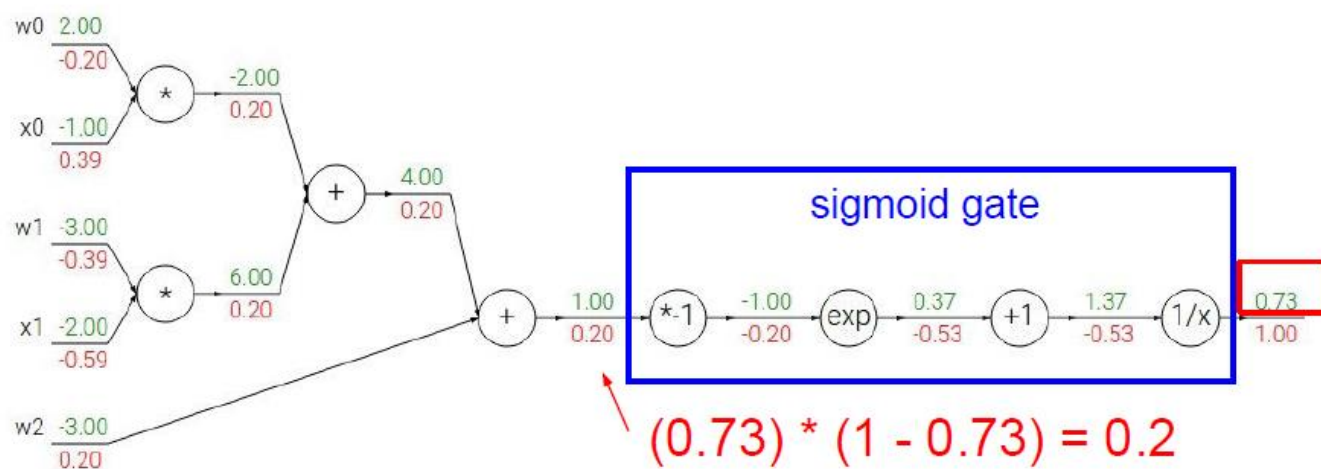
反向传播:

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

sigmoid function

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$$

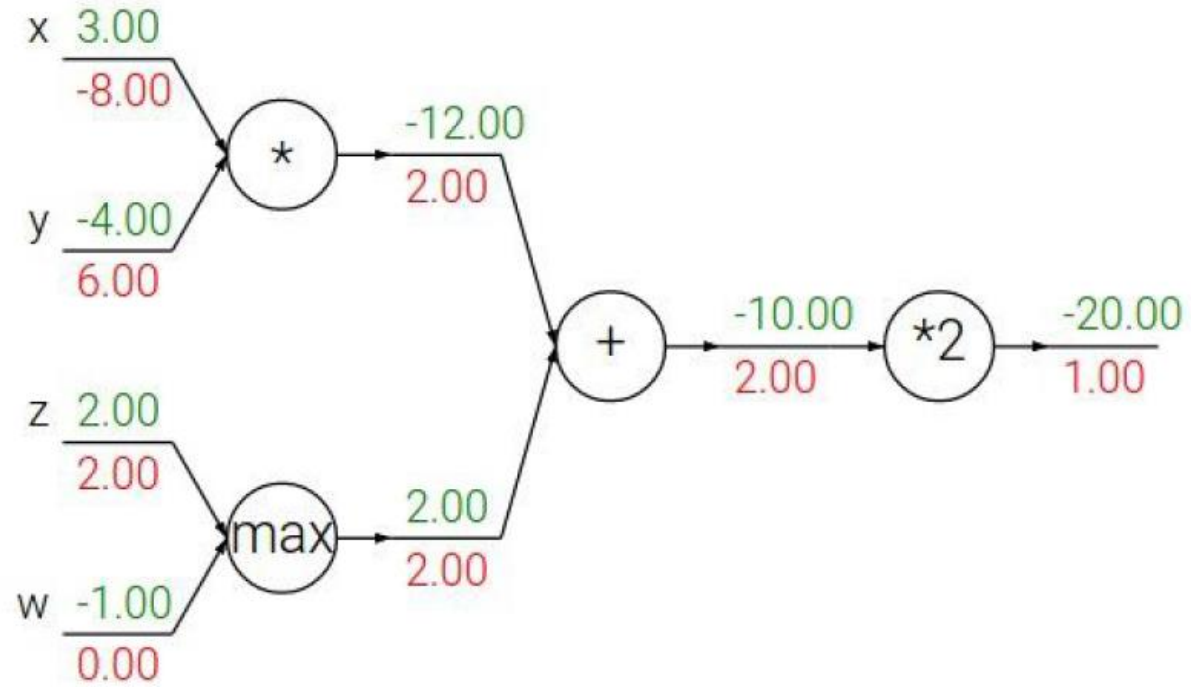


反向传播：

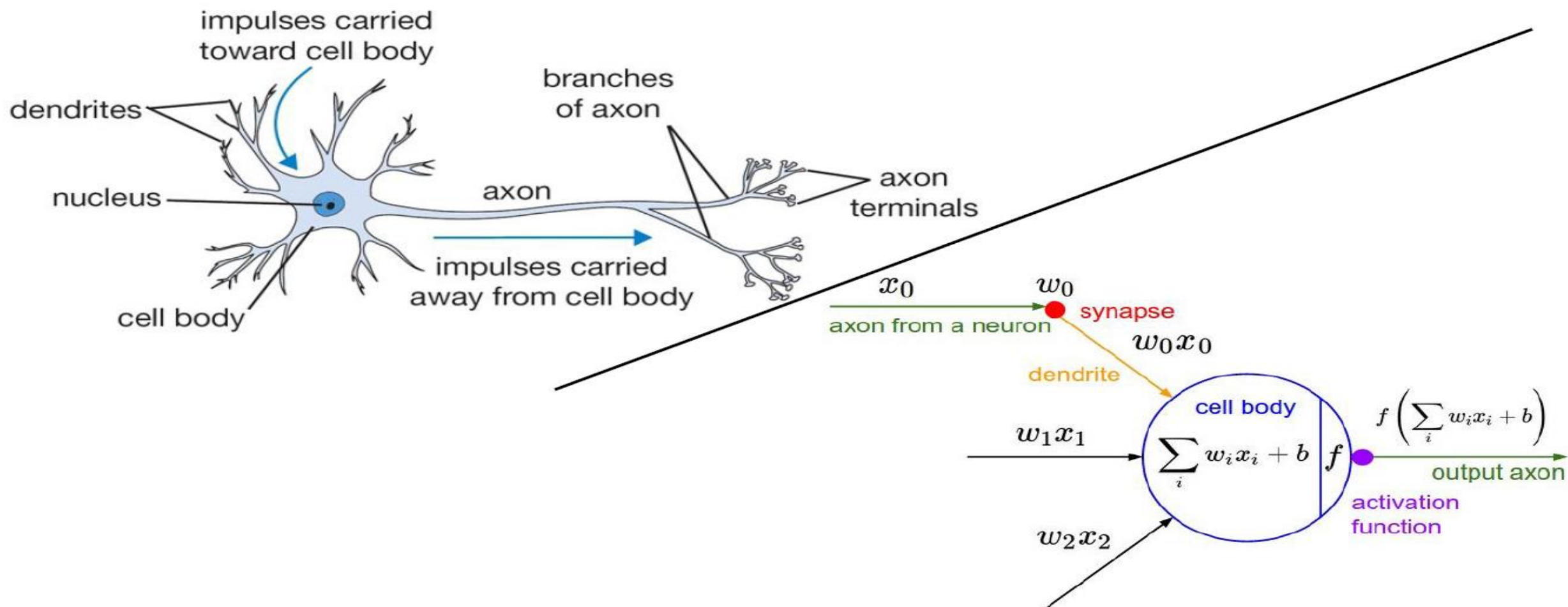
加法门单元：均等分配

MAX门单元：给最大的

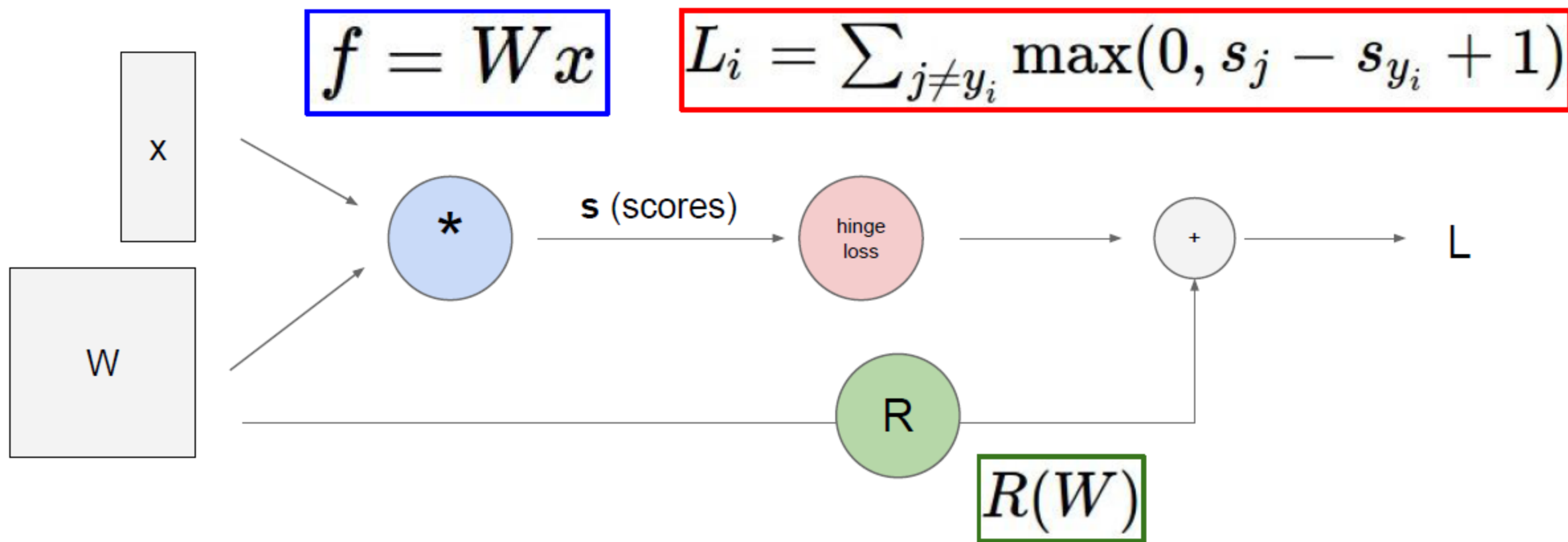
乘法门单元：互换的感觉



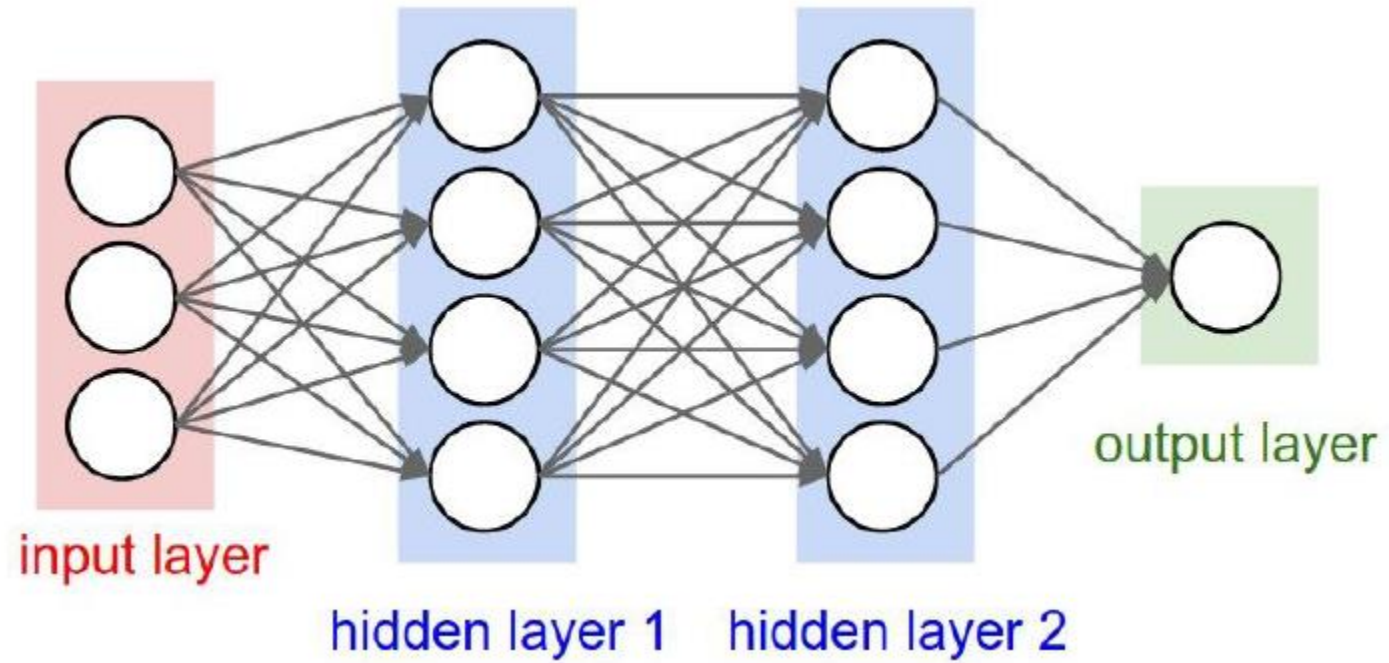
神经网络:



神经网络:



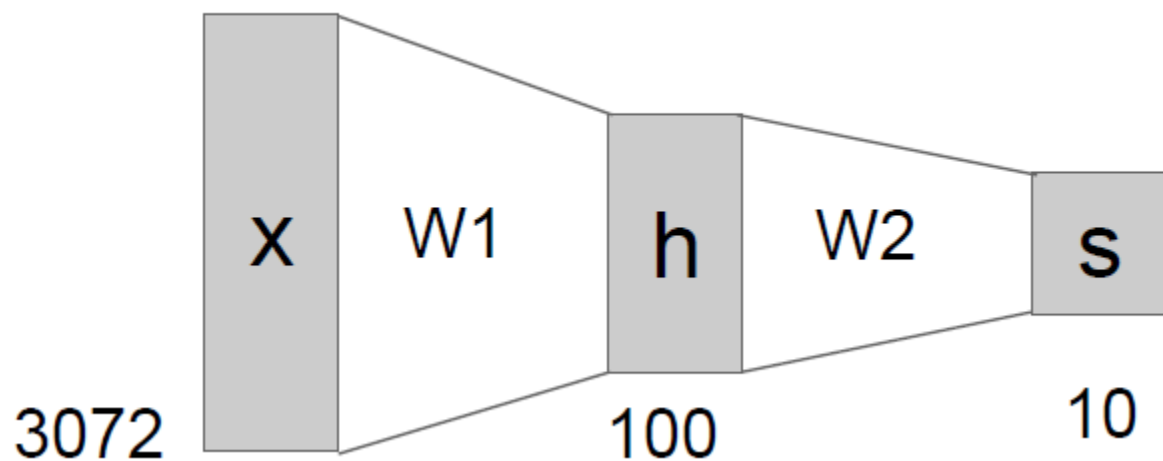
神经网络：



神经网络：

线性方程： $f = Wx$

非线性方程： $f = W_2 \max(0, W_1 x)$



神经网络：

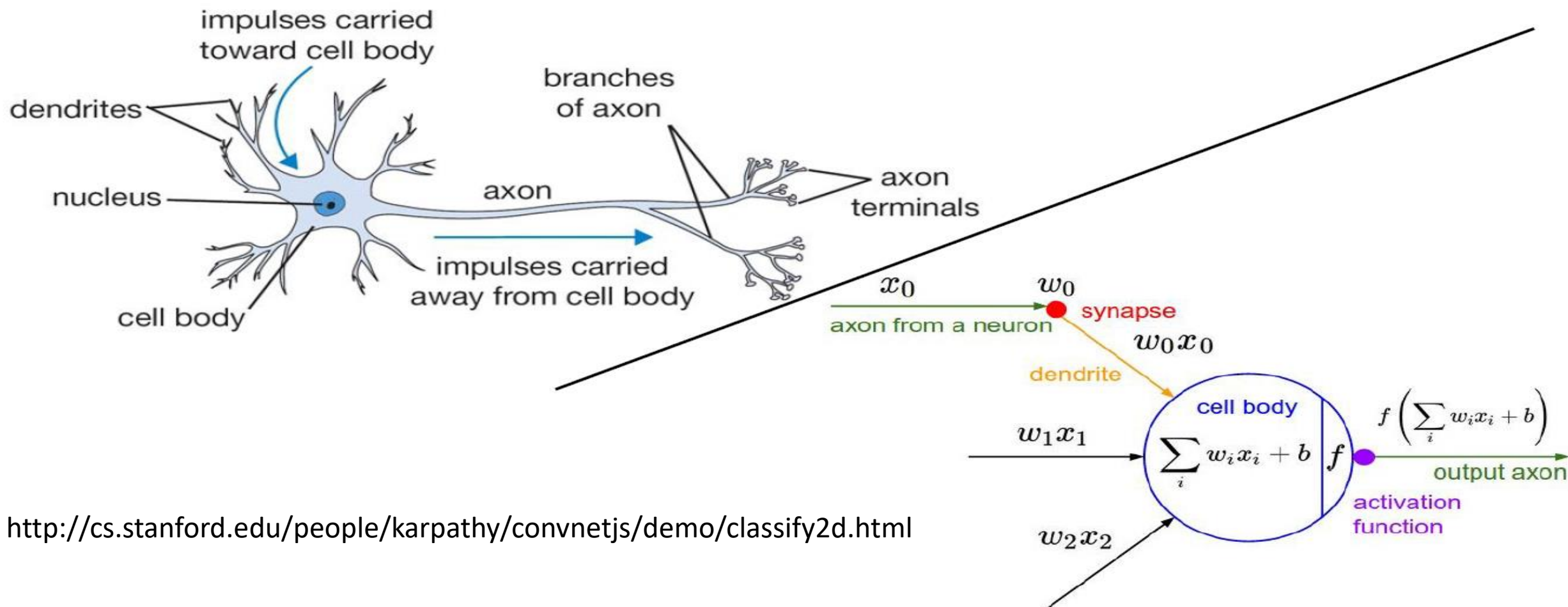
单层的神经网络：

$$f = W_2 \max(0, W_1 x)$$

双层的神经网络：

$$f = W_3 \max(0, W_2 \max(0, W_1 x))$$

神经网络:



<http://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html>

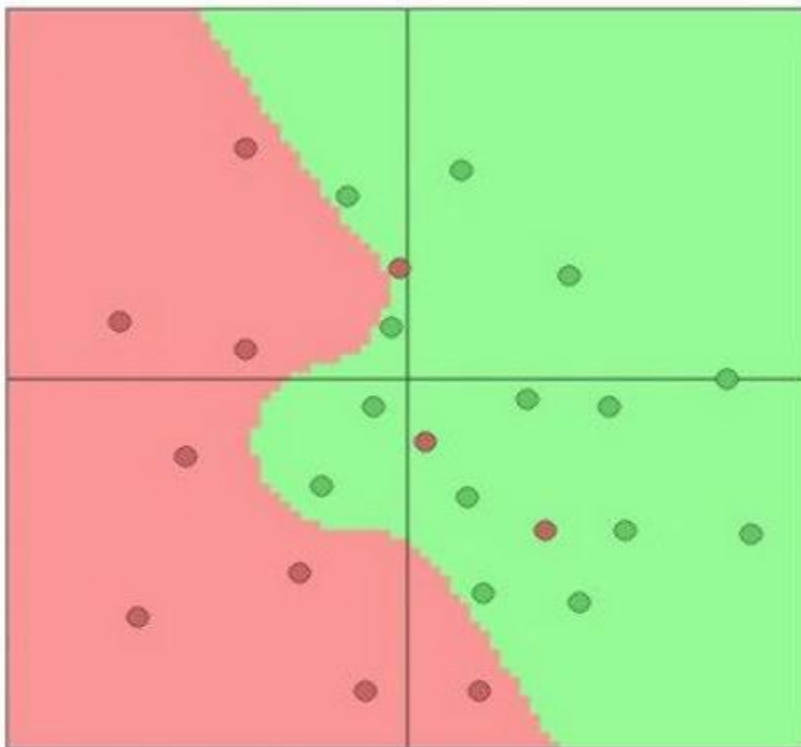
神经网络：

正则化项在神经网络中的重要作用

$\lambda = 0.001$



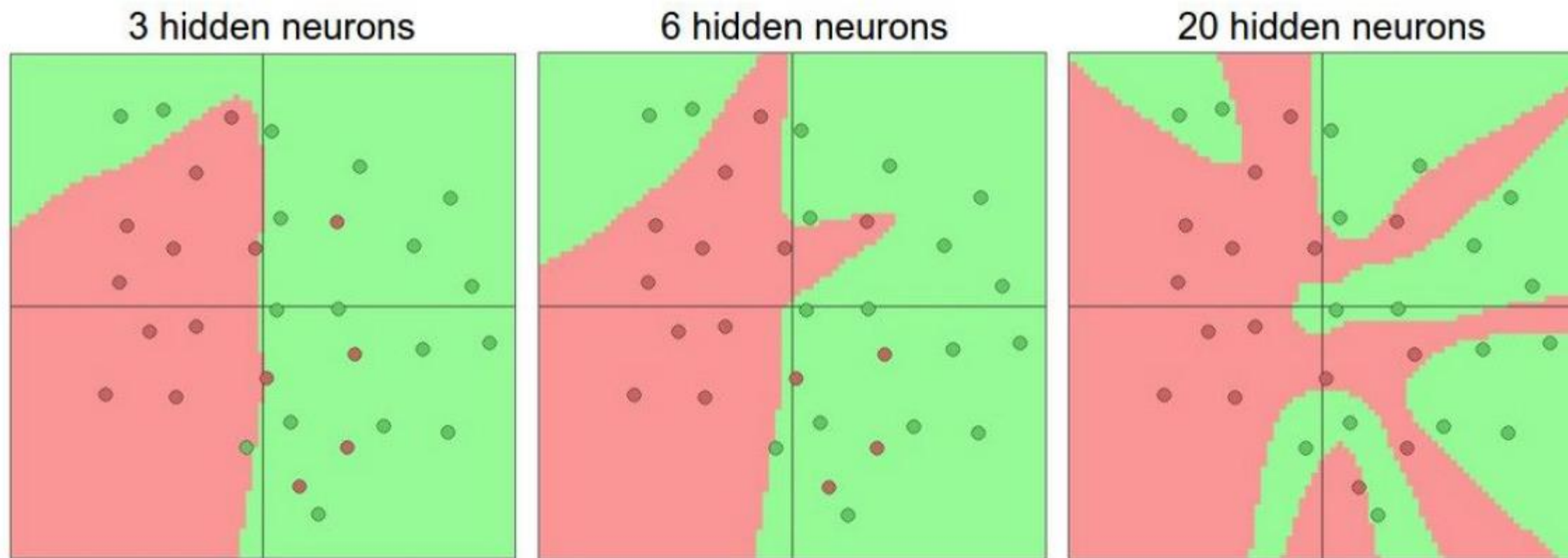
$\lambda = 0.01$



$\lambda = 0.1$

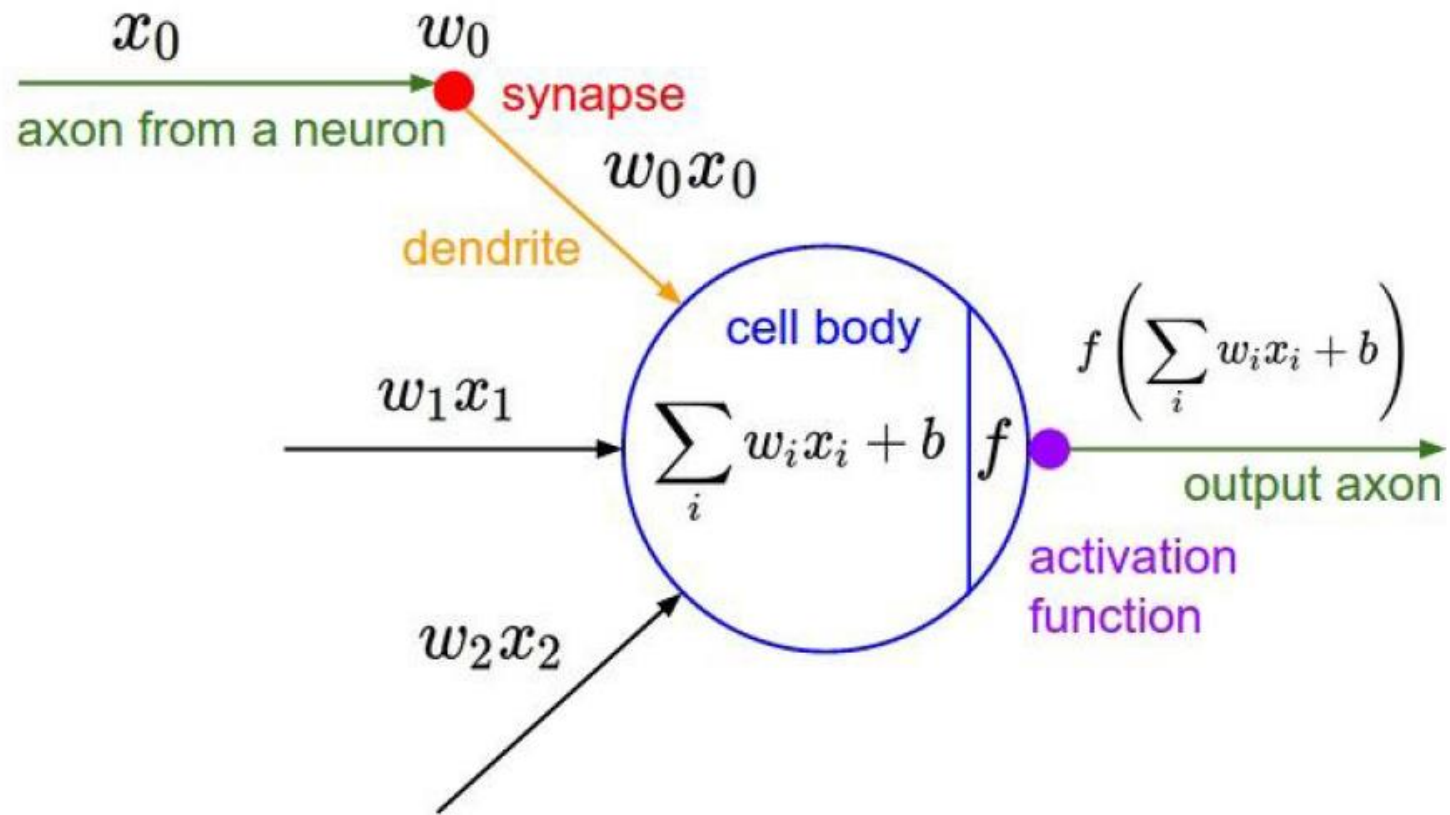


神经网络：



越多的神经元，就越能够表达复杂的模型

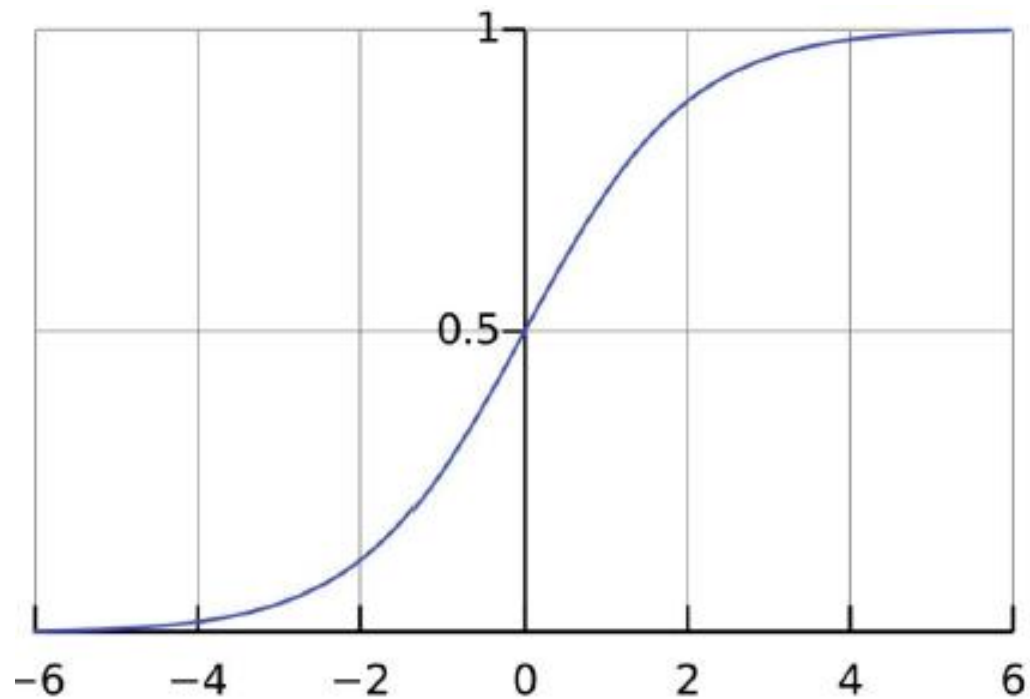
激活函数:



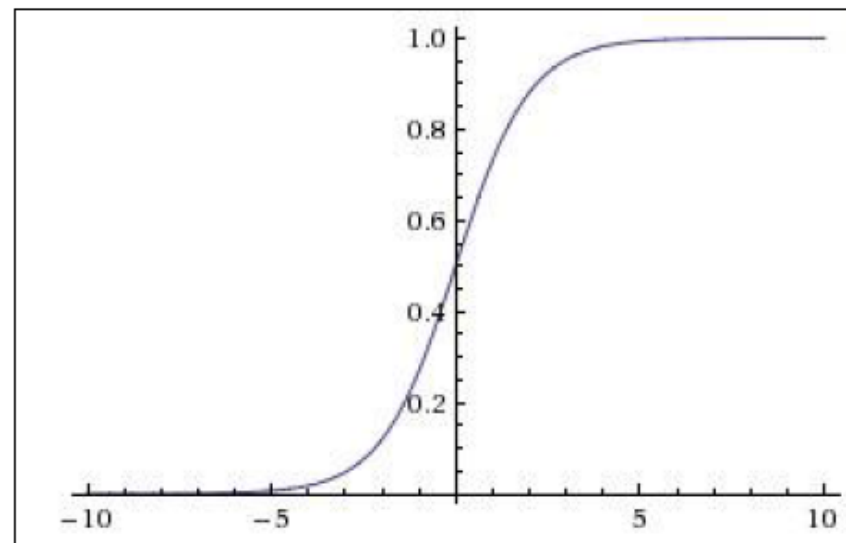
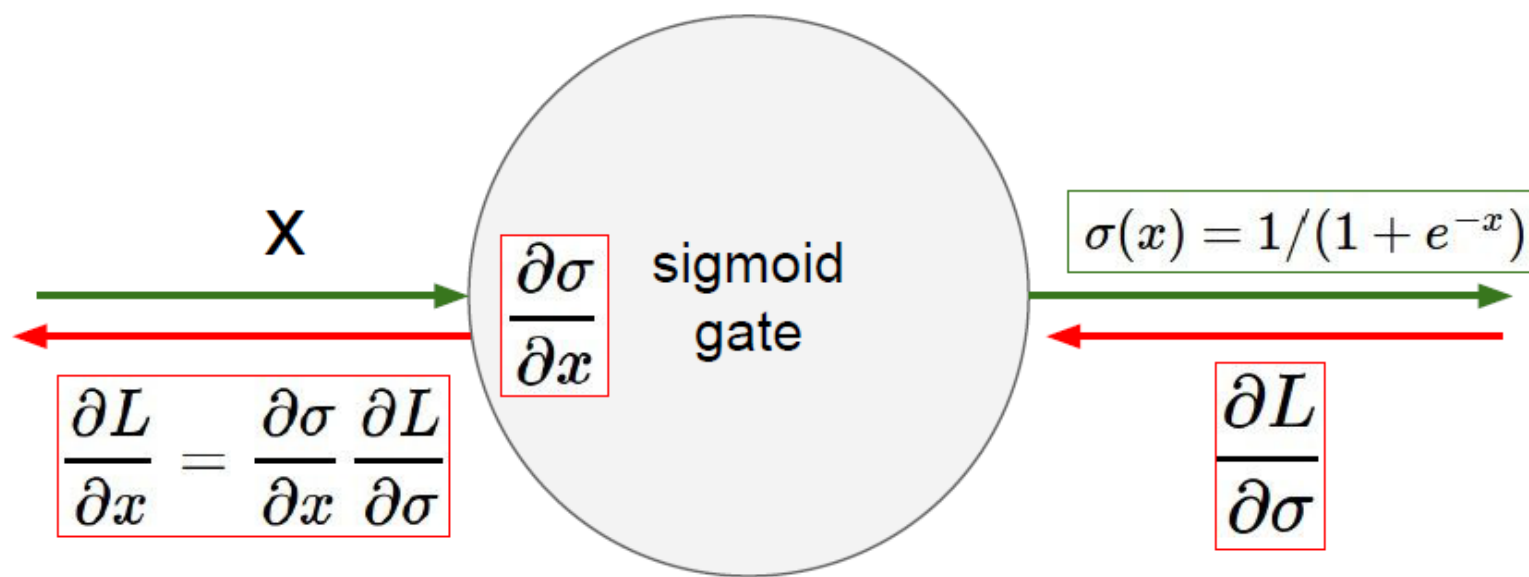
激活函数:

Sigmoid

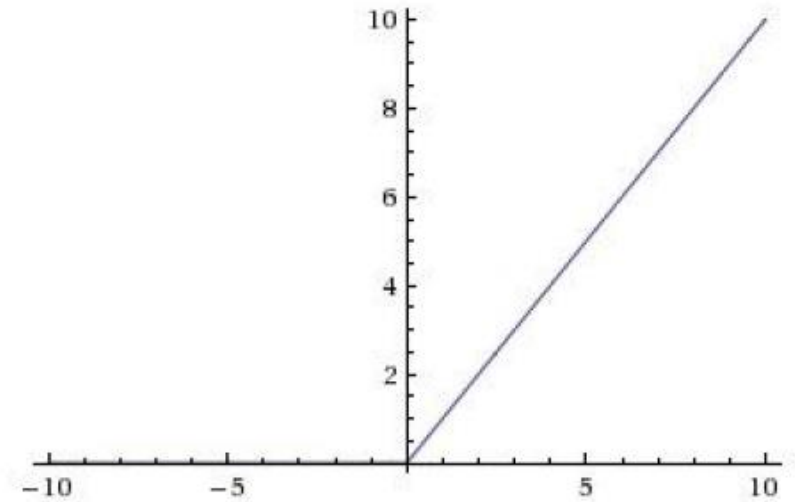
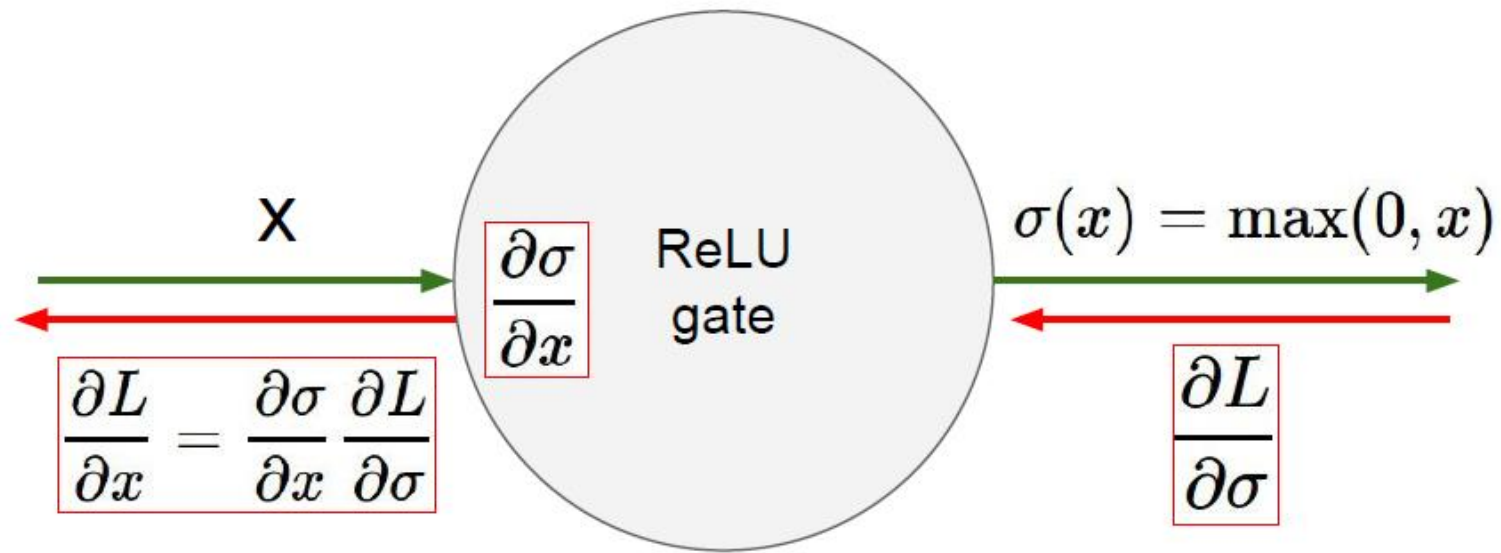
$$\sigma(x) = 1 / (1 + e^{-x})$$



激活函数:

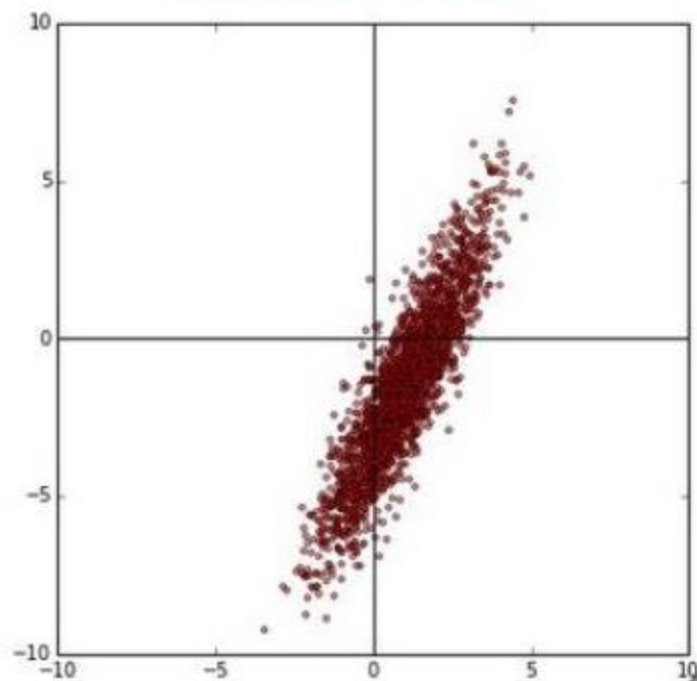


激活函数:

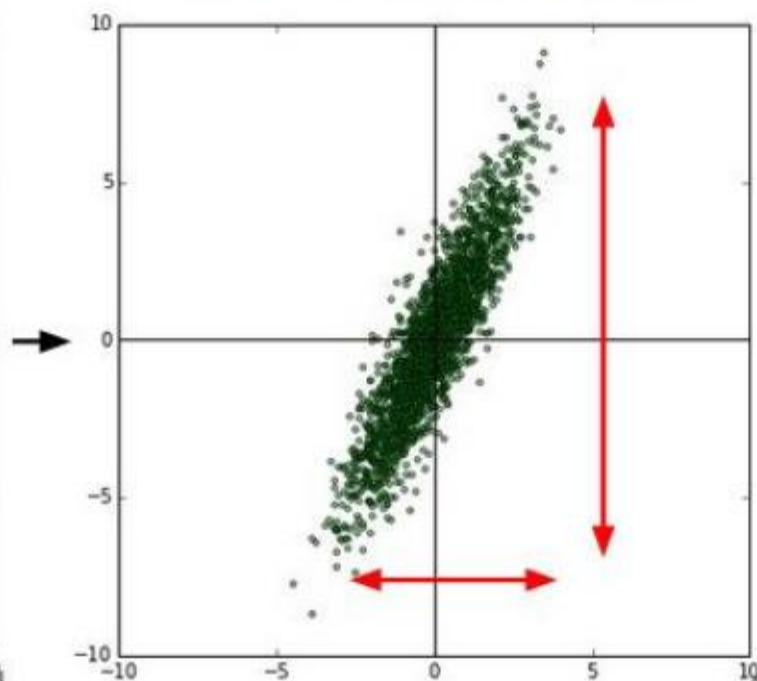


数据预处理:

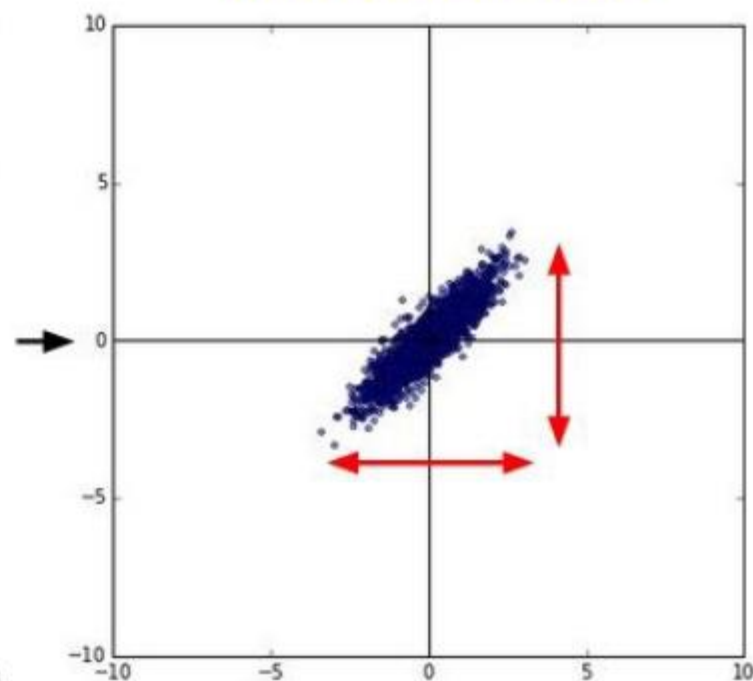
original data



zero-centered data



normalized data

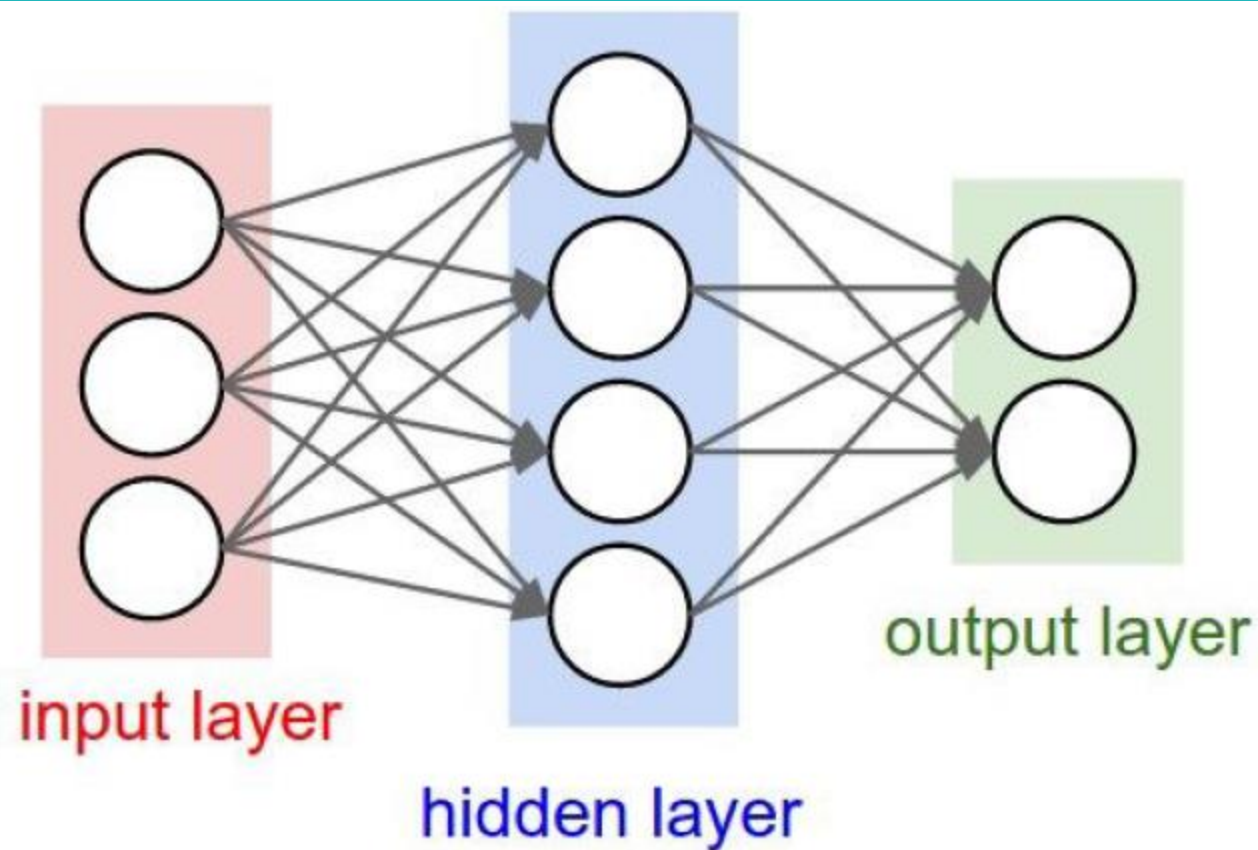


```
X -= np.mean(X, axis = 0)
```

```
X /= np.std(X, axis = 0)
```

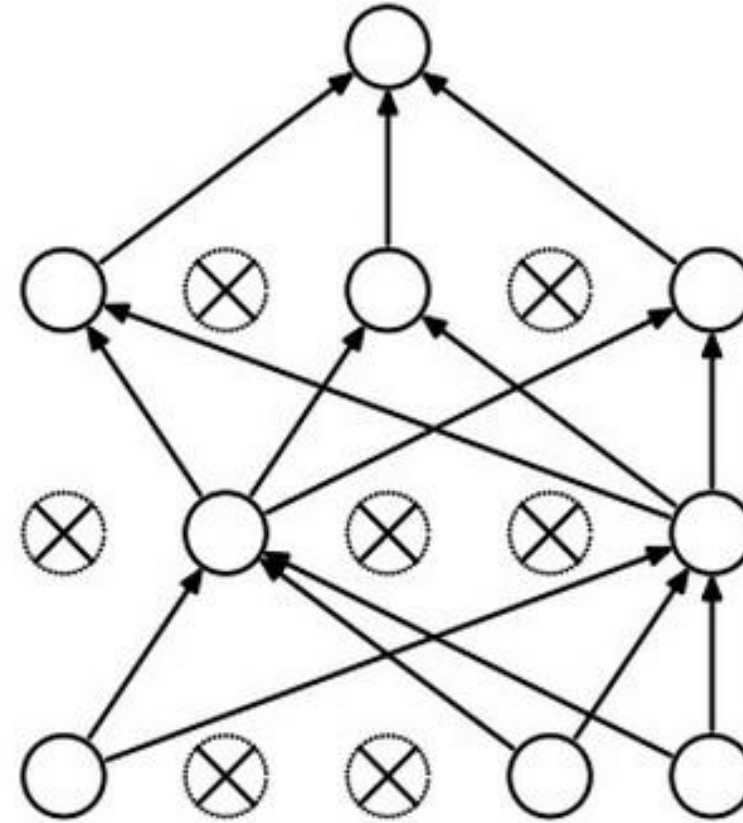
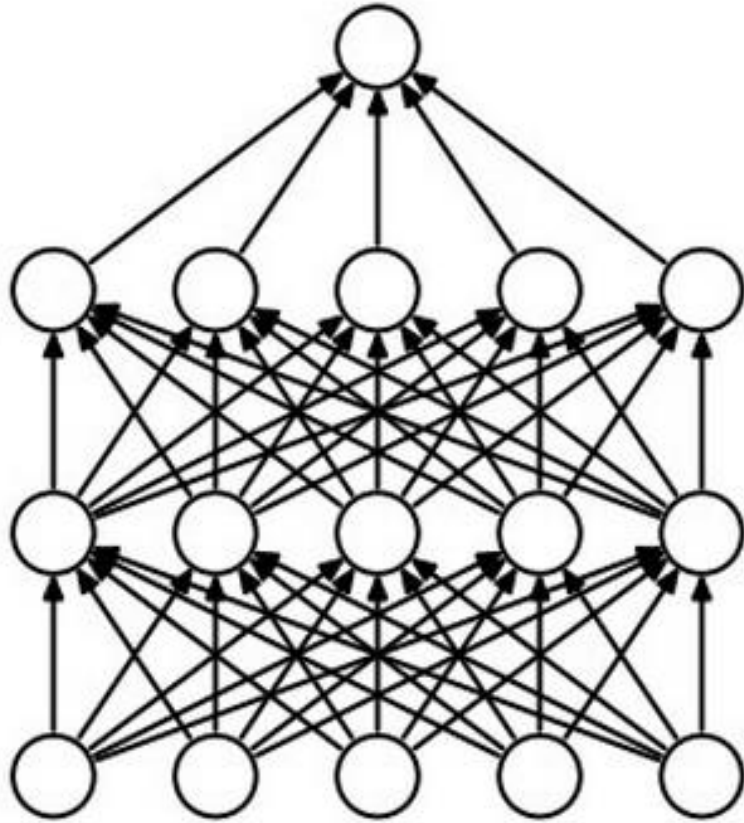
权重初始化:

全零值初始化?



```
W = 0.01* np.random.randn(D,H)
```

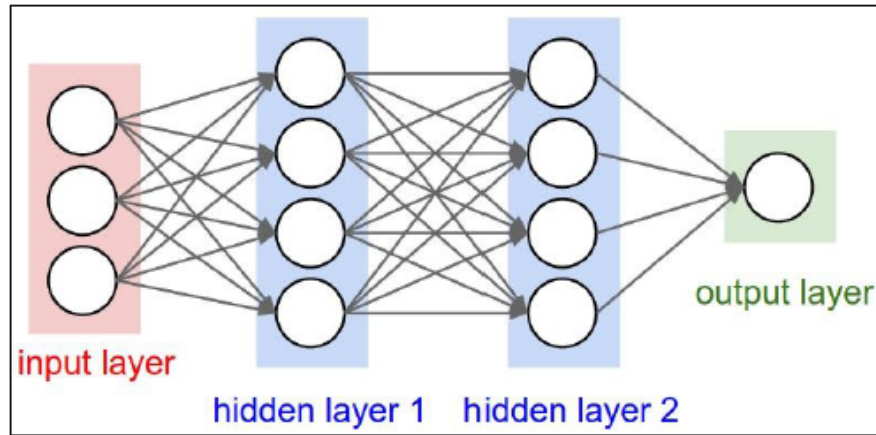
DROP-OUT:



卷积神经网络:

Loop:

1. **Sample** a batch of data
2. **Forward** prop it through the graph, get loss
3. **Backprop** to calculate the gradients
4. **Update** the parameters using the gradient



卷积神经网络:

Classification

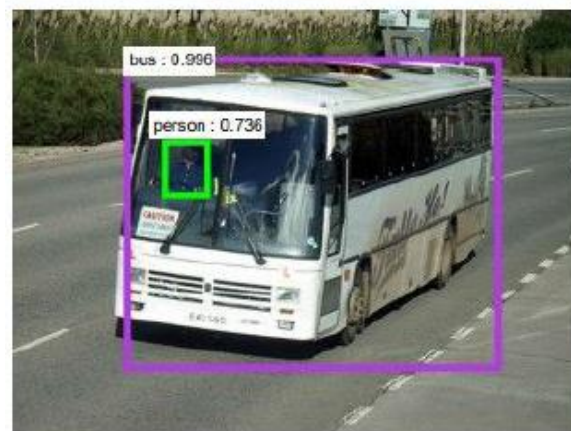
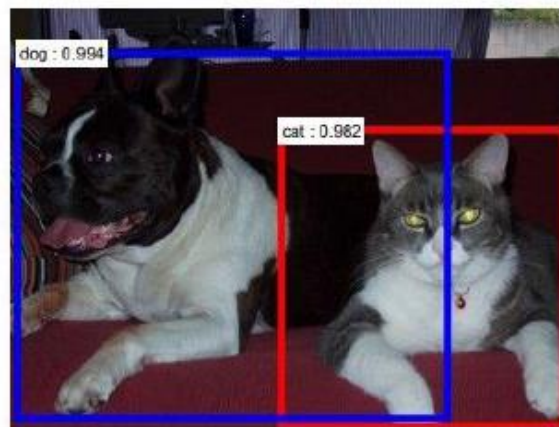
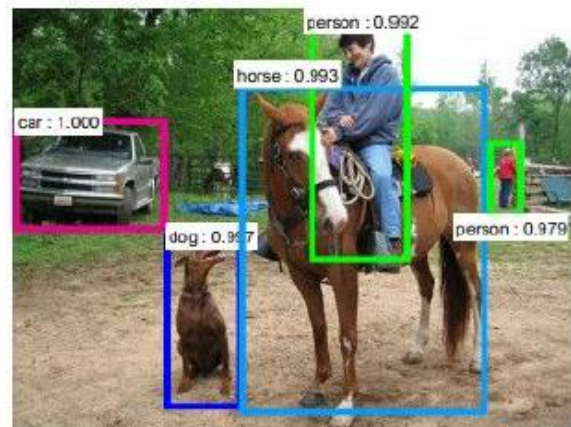


Retrieval



卷积神经网络:

Detection



Segmentation



卷积神经网络:

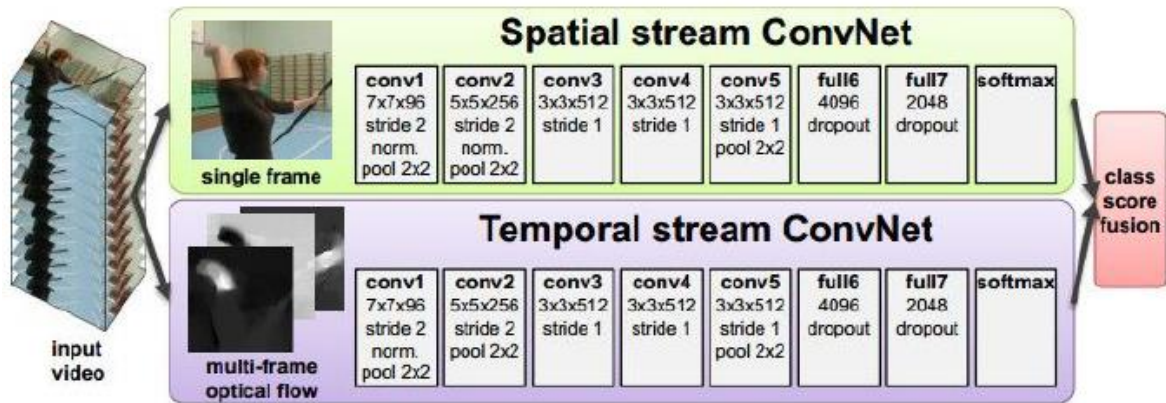
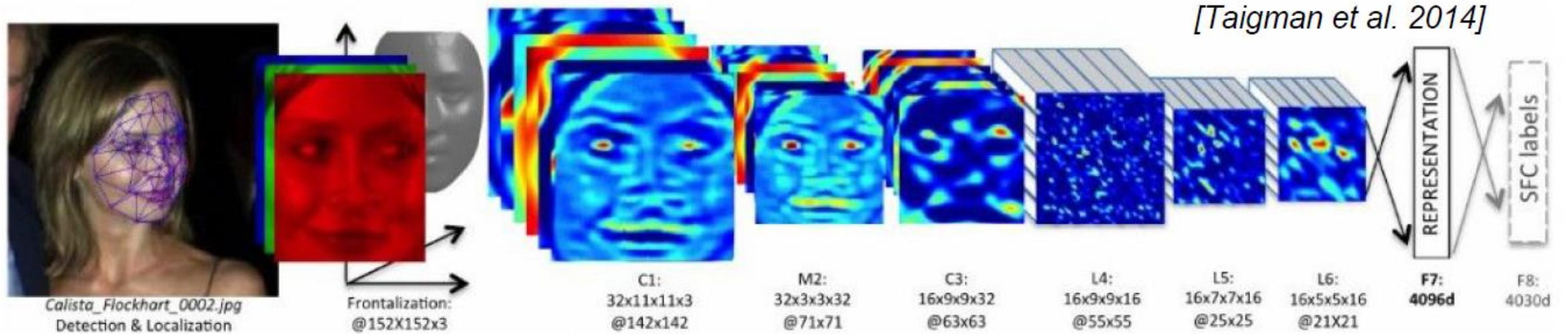


self-driving cars



NVIDIA Tegra X1

卷积神经网络:



[Goodfellow 2014]

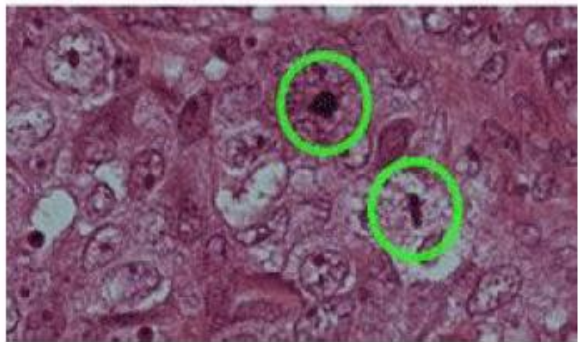
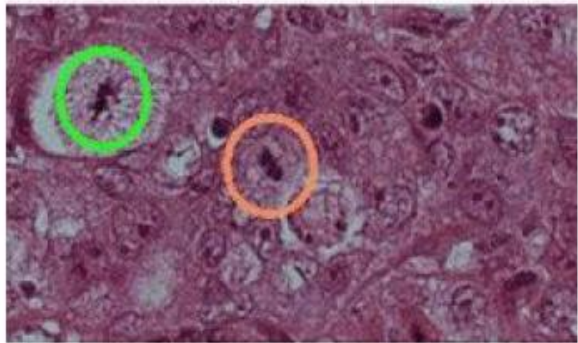
[Simonyan et al. 2014]

卷积神经网络:



[Toshev, Szegedy 2014]

卷积神经网络:



咱	攢	暫	贊	賍	脏	葬	遭
择	则	泽	贼	怎	增	憎	曾
派	摘	寄	宅	窄	债	寨	瞻
湛	綻	樟	章	彰	漳	张	掌
照	罩	兆	肇	召	遮	折	哲
针	侦	枕	疼	诊	震	振	镇
郑	证	芝	枝	支	吱	蜘	知
止	趾	只	兹	纸	志	挚	擢



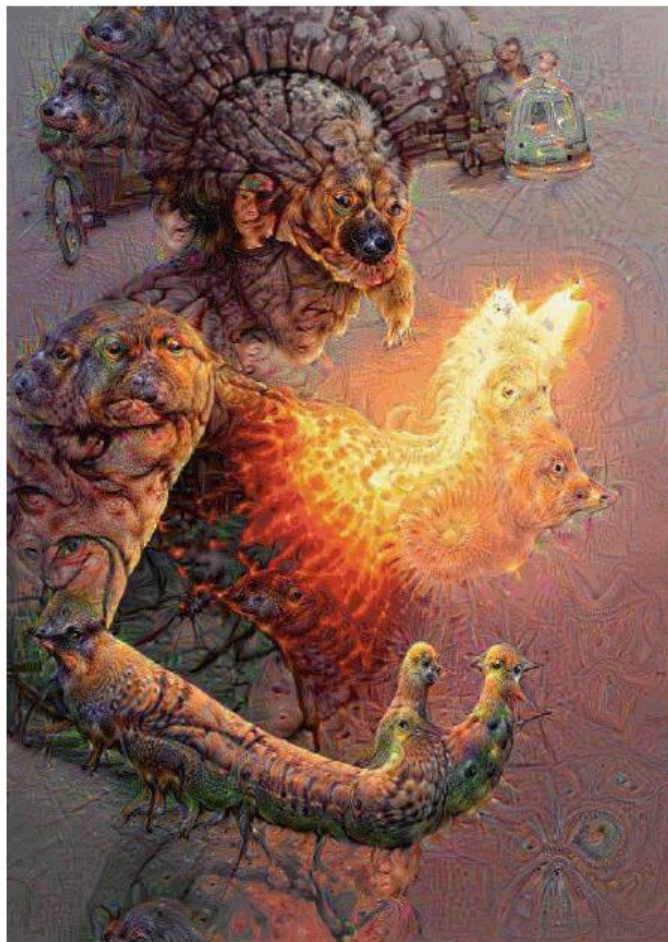
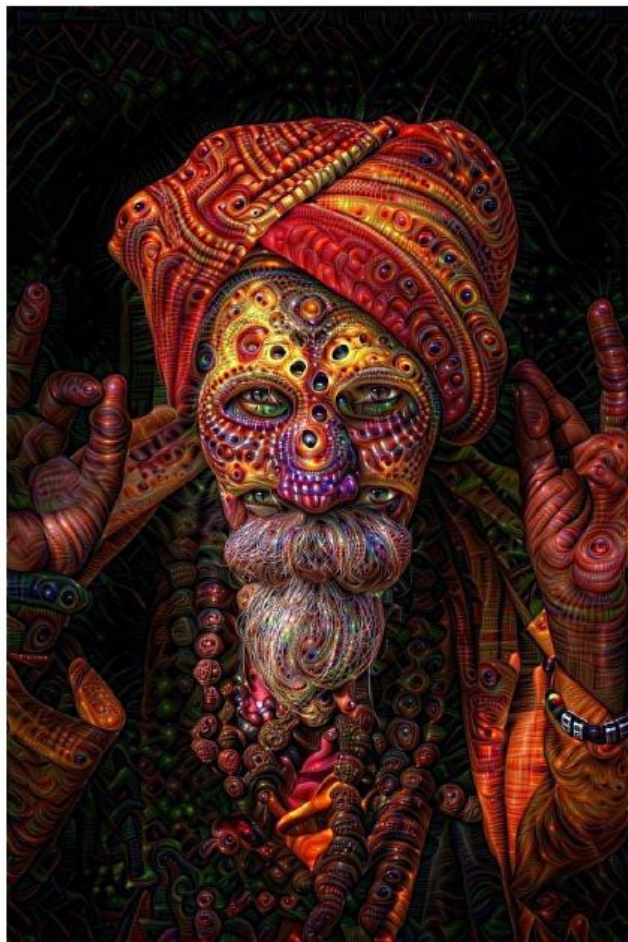
卷积神经网络:

Describes without errors	Describes with minor errors	Somewhat related to the image	Unrelated to the image
 A person riding a motorcycle on a dirt road.	 Two dogs play in the grass.	 A skateboarder does a trick on a ramp.	 A dog is jumping to catch a frisbee.
 A group of young people playing a game of frisbee.	 Two hockey players are fighting over the puck.	 A little girl in a pink hat is blowing bubbles.	 A refrigerator filled with lots of food and drinks.
 A herd of elephants walking across a dry grass field.	 A close up of a cat laying on a couch.	 A red motorcycle parked on the side of the road.	 A yellow school bus parked in a parking lot.

Image Captioning

[Vinyals et al., 2015]

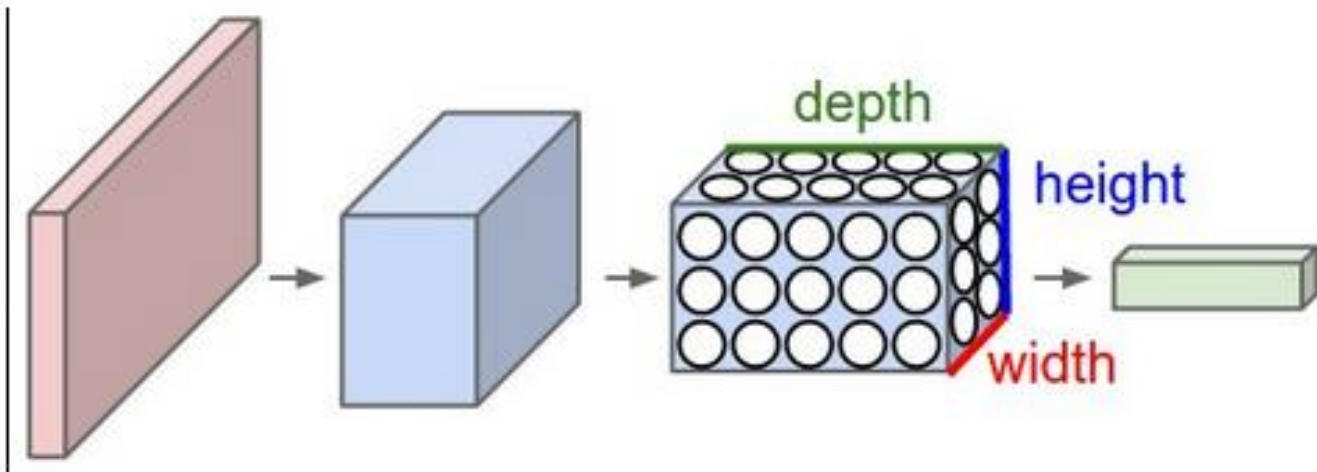
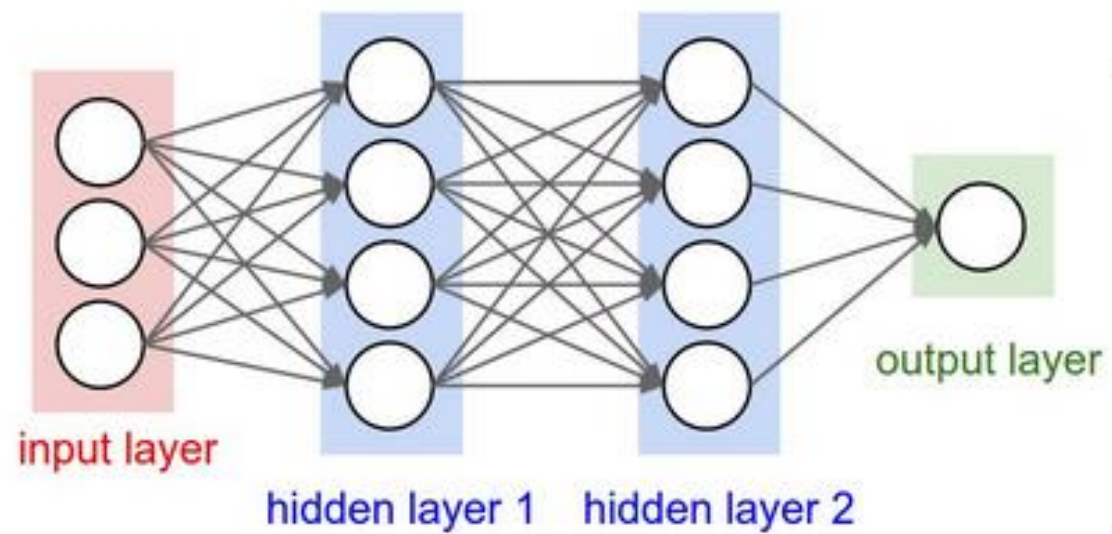
卷积神经网络：



卷积神经网络:



卷积神经网络:

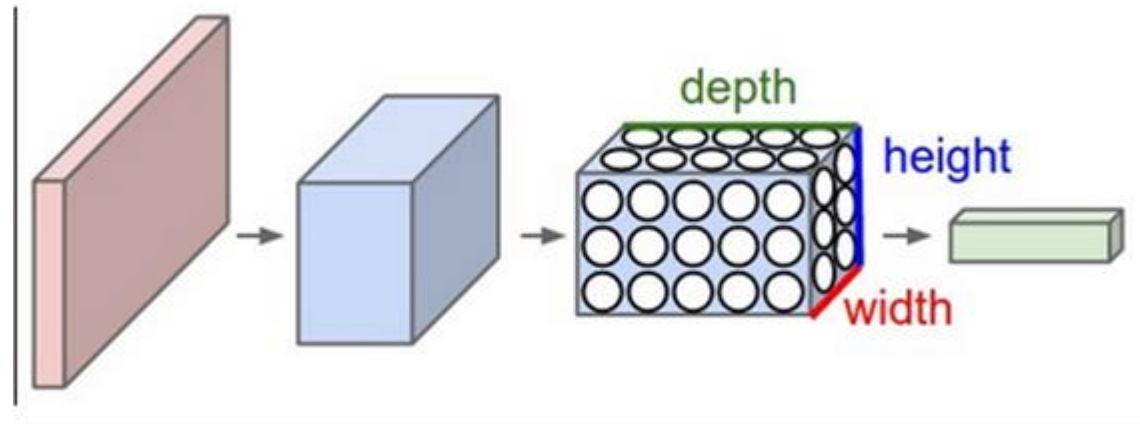


卷积神经网络：

卷积神经网络组成：

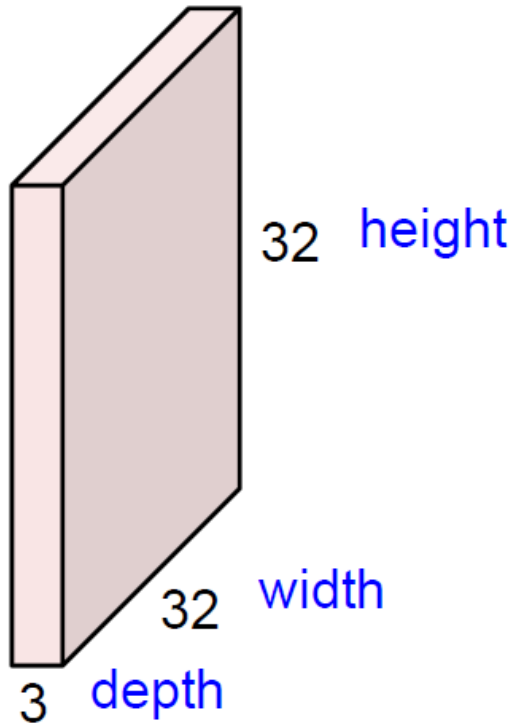
[INPUT - CONV - RELU - POOL - FC]

- 输入层
- 卷积层
- 激活函数
- 池化层
- 全连接层



卷积神经网络:

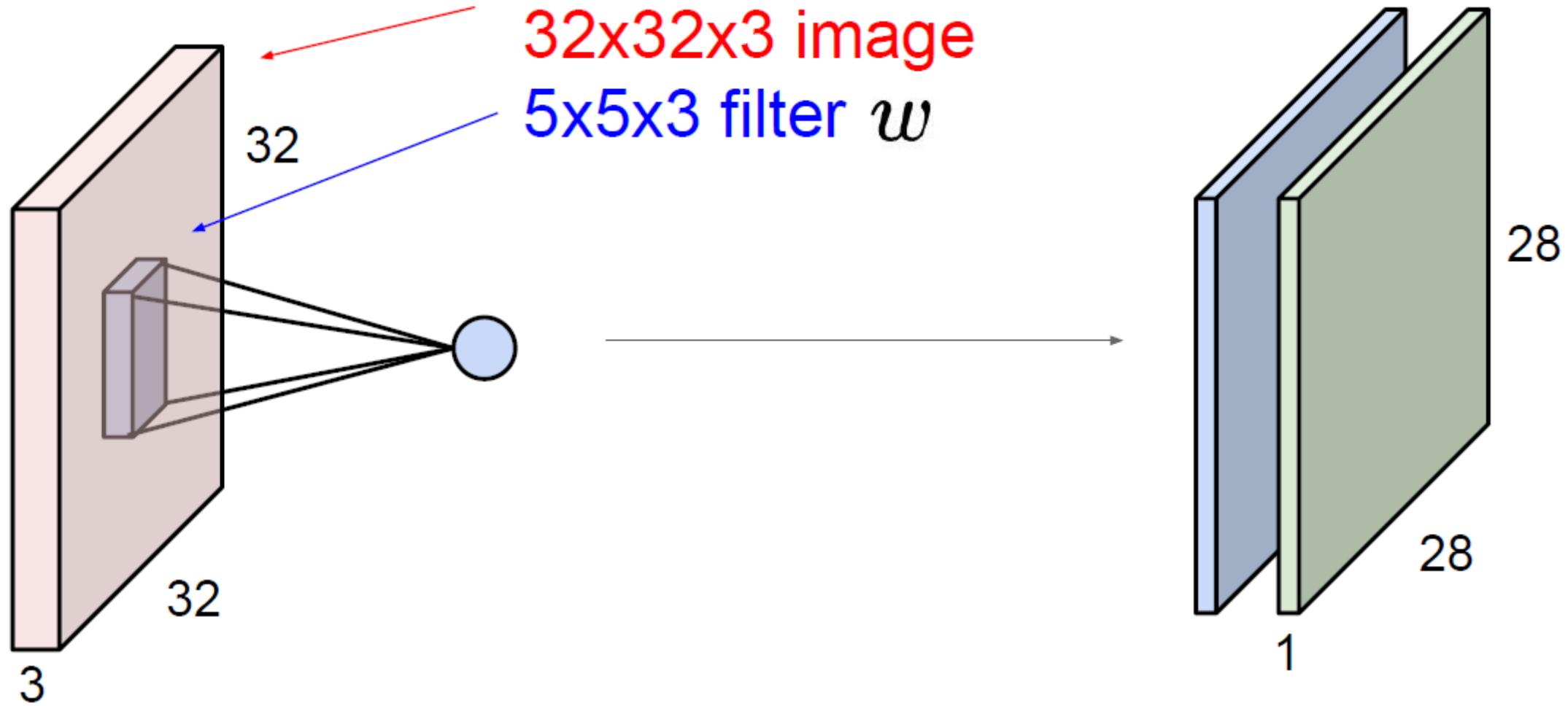
32x32x3 image



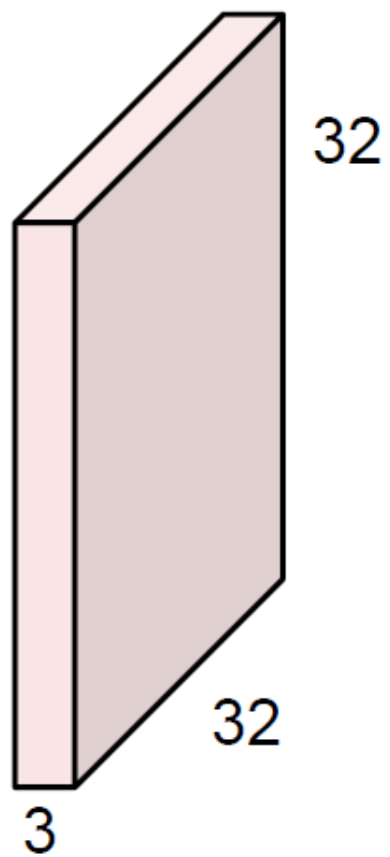
5x5x3



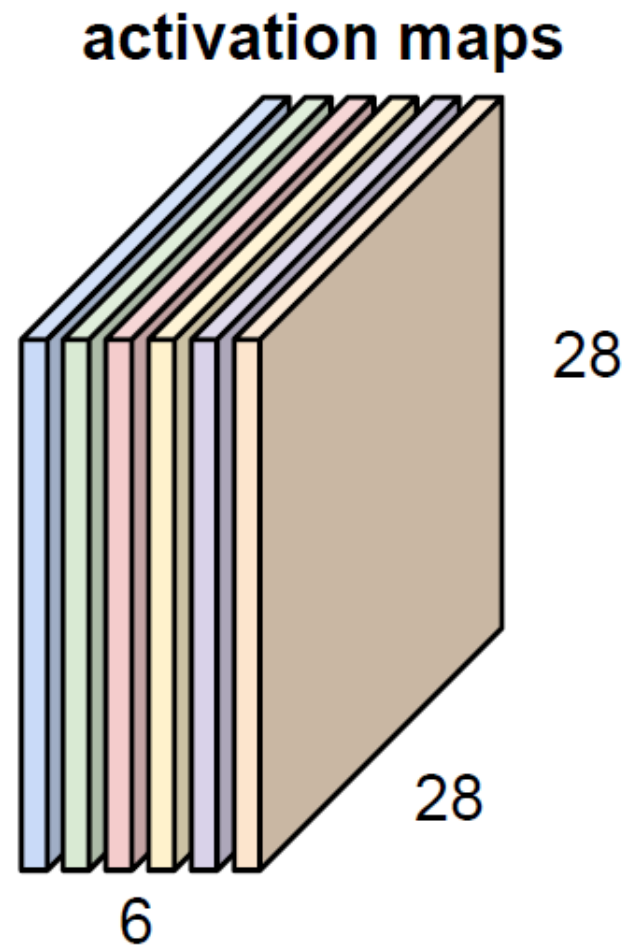
卷积神经网络:



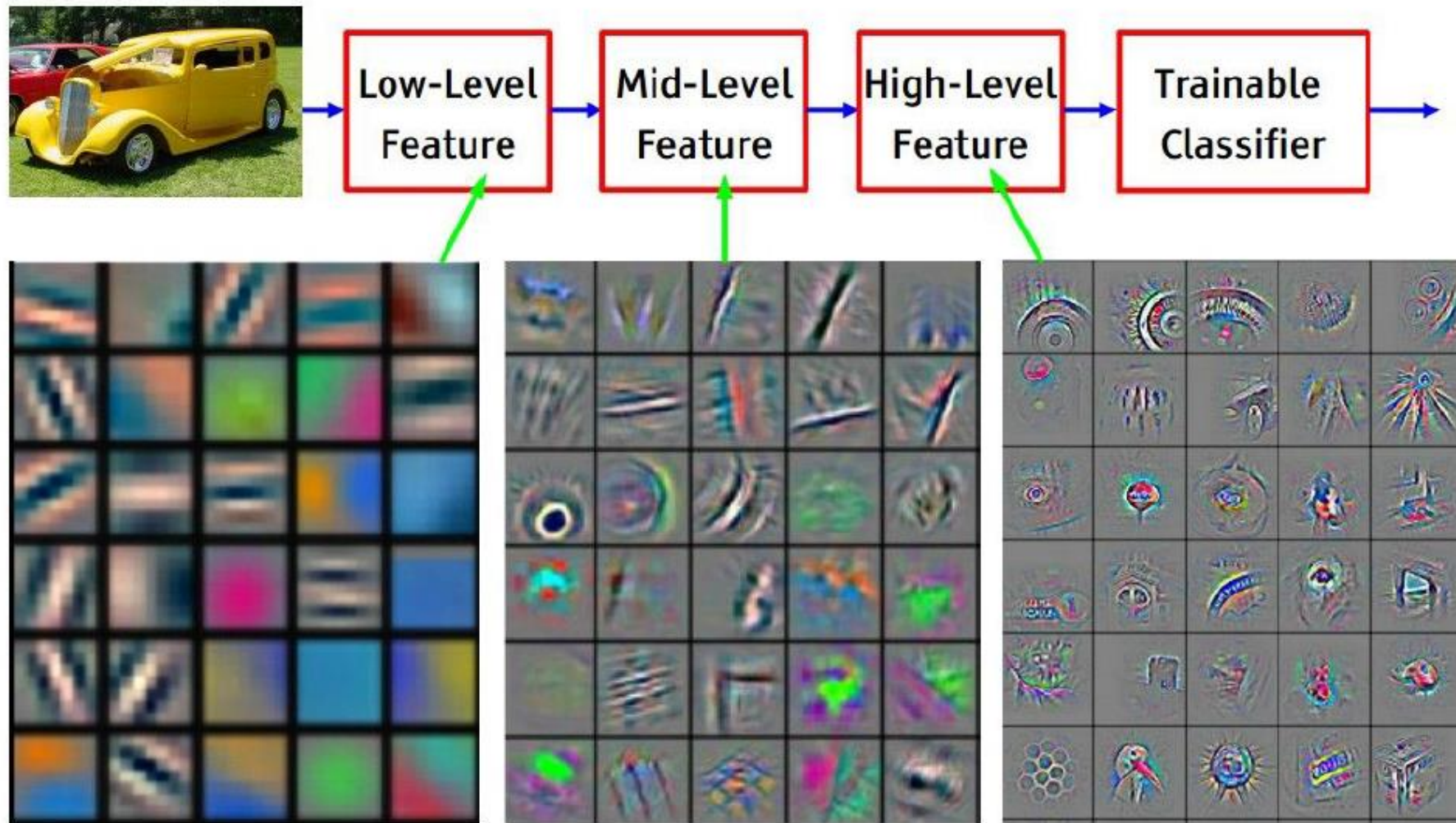
卷积神经网络:



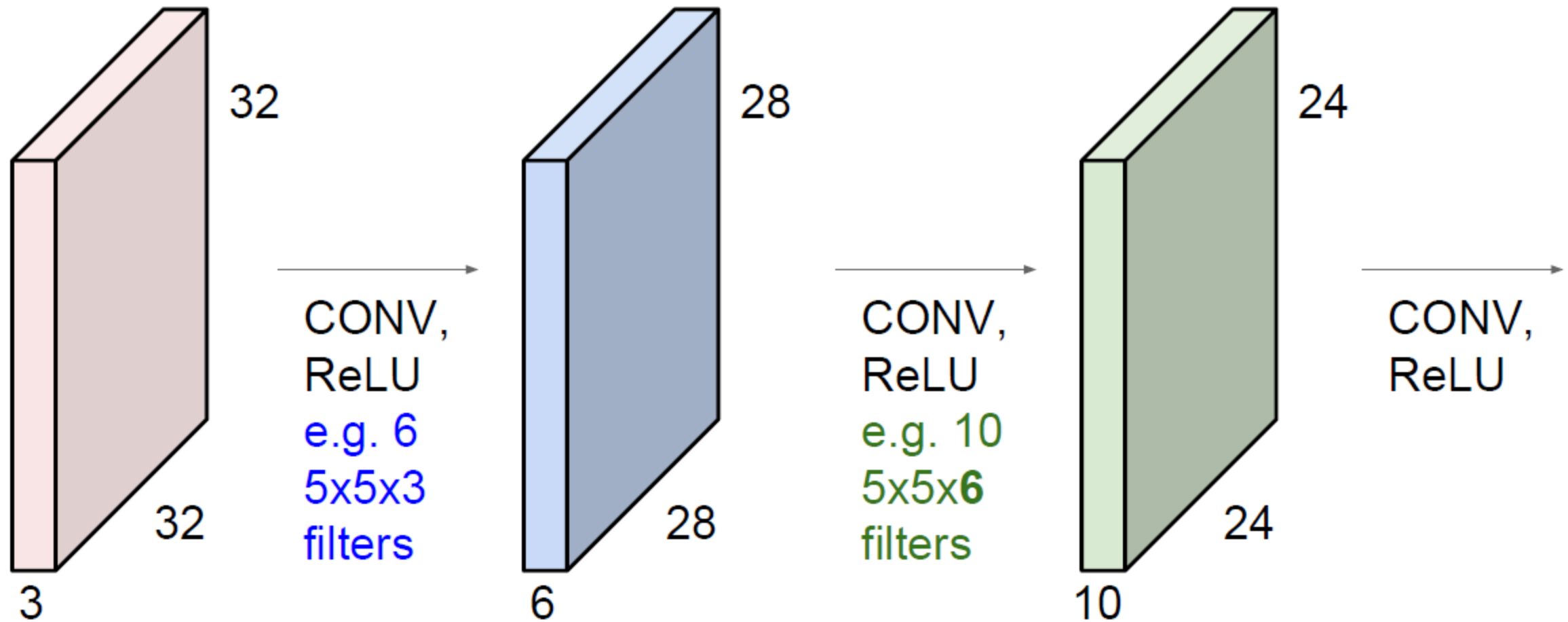
Convolution Layer



卷积神经网络:



卷积神经网络:



Input Volume (+pad 1) (7x7x3)

 $x[:, :, 0]$

0	0	0	0	0	0	0
0	1	1	2	2	1	0
0	1	1	1	2	1	0

0	2	1	1	0	2	0
0	2	1	0	1	2	0
0	2	1	2	2	2	0
0	0	0	0	0	0	0

 $x[:, :, 1]$

0	0	0	0	0	0	0
0	0	1	2	0	1	0
0	2	2	1	1	0	0

0	2	1	0	0	2	0
0	1	0	0	0	2	0
0	0	1	0	1	2	0
0	0	0	0	0	0	0

 $x[:, :, 2]$

0	0	0	0	0	0	0
0	2	2	0	1	2	0
0	0	0	2	1	2	0

0	2	1	0	2	1	0
0	1	1	0	0	0	0
0	0	0	1	1	1	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

 $w0[:, :, 0]$

1	1	1
-1	-1	0
-1	1	0

 $w0[:, :, 1]$

-1	-1	1
-1	1	0
-1	1	0

 $w0[:, :, 2]$

1	0	-1
0	0	0
1	-1	-1

Bias b0 (1x1x1)

 $b0[:, :, 0]$

1

Filter W1 (3x3x3)

 $w1[:, :, 0]$

0	0	-1
-1	1	1
0	0	0

 $w1[:, :, 1]$

0	0	1
1	0	1
0	-1	-1

 $w1[:, :, 2]$

-1	1	1
0	1	1
1	-1	1

Bias b1 (1x1x1)

 $b1[:, :, 0]$

0

Output Volume (3x3x2)

 $o[:, :, 0]$

3	-5	-4
4	-1	7
0	0	-1

 $o[:, :, 1]$

3	2	0
7	6	2
5	5	2

toggle movement

Input Volume (+pad 1) (7x7x3)

 $x[:, :, 0]$

0	0	0	0	0	0	0
0	1	1	2	2	1	0
0	1	1	1	2	1	0
0	2	1	1	0	2	0
0	2	1	0	1	2	0
0	2	1	2	2	2	0
0	0	0	0	0	0	0

 $x[:, :, 1]$

0	0	0	0	0	0	0
0	0	1	2	0	1	0
0	2	2	1	1	0	0
0	2	1	0	0	2	0
0	1	0	0	0	2	0
0	0	1	0	1	2	0
0	0	0	0	0	0	0

 $x[:, :, 2]$

0	0	0	0	0	0	0
0	2	2	0	1	2	0
0	0	0	2	1	2	0
0	2	1	0	2	1	0
0	1	1	0	0	0	0
0	0	0	1	1	1	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

 $w0[:, :, 0]$

1	1	1
-1	-1	0
-1	1	0

 $w0[:, :, 1]$

-1	-1	1
-1	1	0
-1	1	0

 $w0[:, :, 2]$

1	0	-1
0	0	0
1	-1	-1

Bias b0 (1x1x1)

 $b0[:, :, 0]$

1

Filter W1 (3x3x3)

 $w1[:, :, 0]$

0	0	-1
-1	1	1
0	0	0

 $w1[:, :, 1]$

0	0	1
1	0	1
0	-1	-1

 $w1[:, :, 2]$

-1	1	1
0	1	1
1	-1	1

Bias b1 (1x1x1)

 $b1[:, :, 0]$

0

Output Volume (3x3x2)

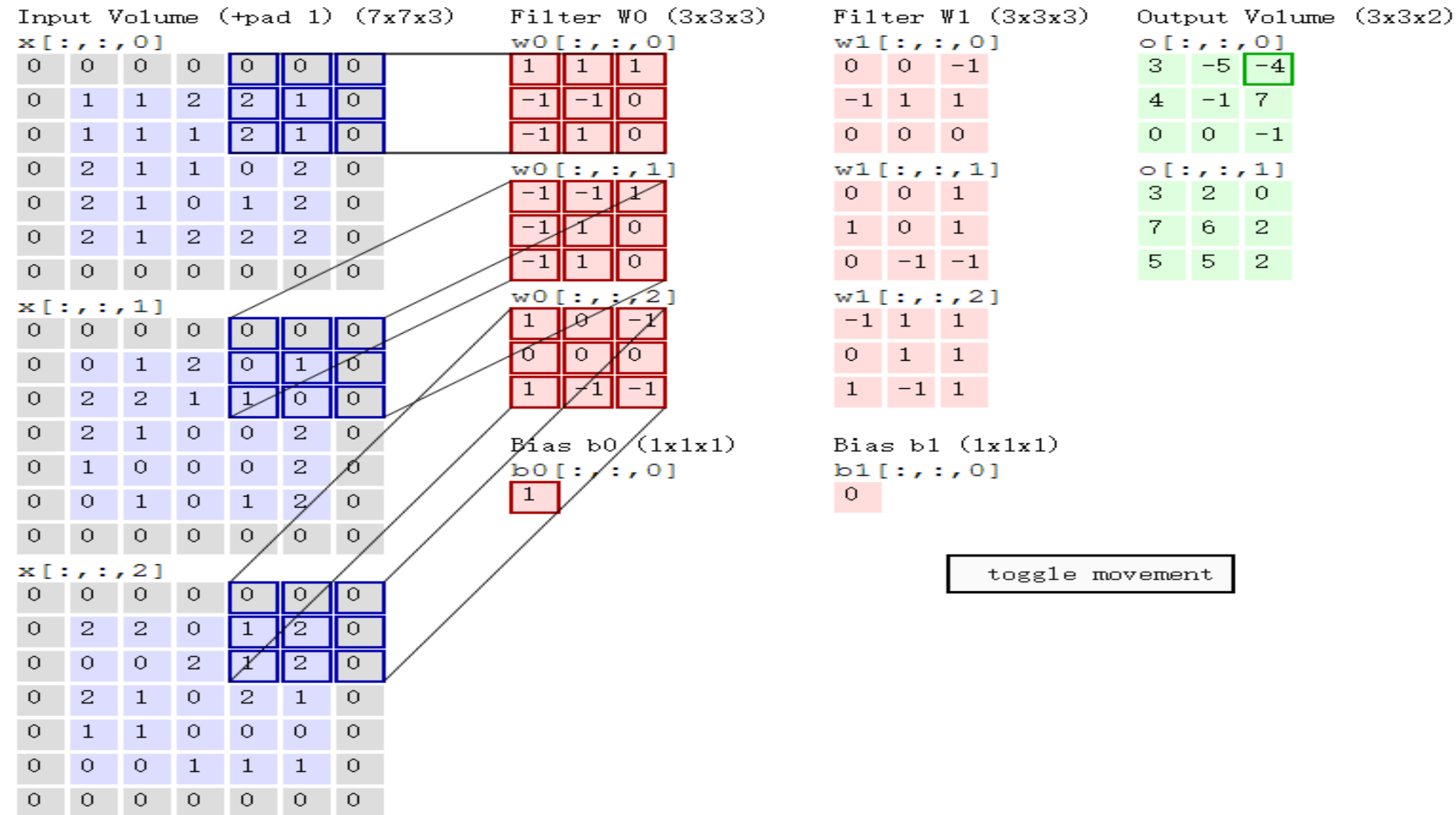
 $o[:, :, 0]$

3	-5	-4
4	-1	7
0	0	-1

 $o[:, :, 1]$

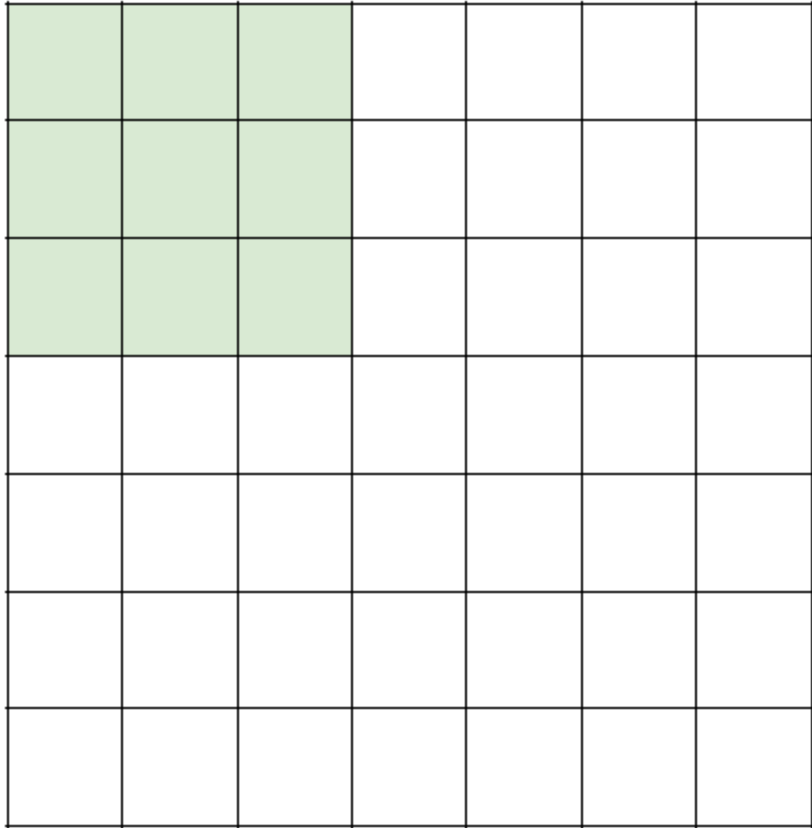
3	2	0
7	6	2
5	5	2

toggle movement



卷积神经网络:

7



7x7 input (spatially)
assume 3x3 filter

=> **5x5 output**

applied **with stride 2**
=> **3x3 output!**

applied **with stride 3?**

7

卷积神经网络:

0	0	0	0	0	0			
0								
0								
0								
0								

输入 = 7×7

Filter = 3×3

Pad = 1

Output = ?

$F = 3 \Rightarrow$ zero pad with 1

$F = 5 \Rightarrow$ zero pad with 2

$F = 7 \Rightarrow$ zero pad with 3

卷积神经网络:

Examples time:

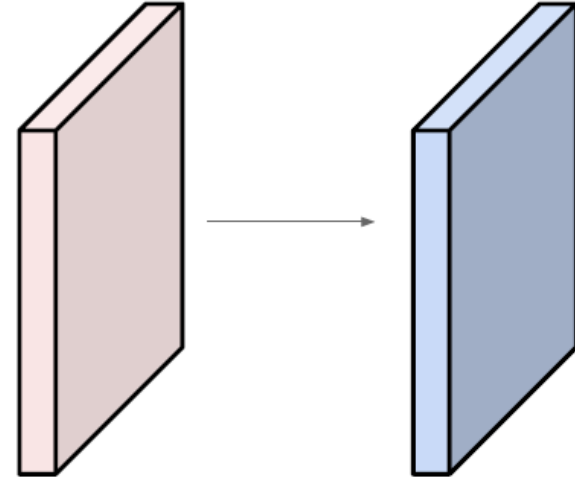
Input volume: **32x32x3**

10 **5x5** filters with stride **1**, pad **2**

Output volume size:

$(32 + 2 * 2 - 5) / 1 + 1 = 32$ spatially, so

32x32x10

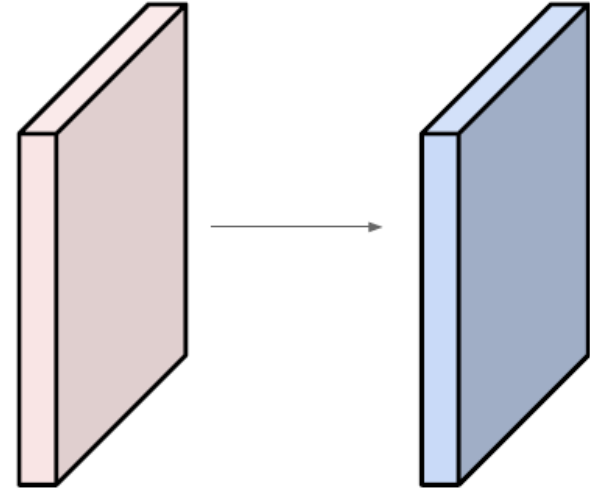


卷积神经网络:

联机将文件转换为 PDF...

Examples time:

Input volume: **32x32x3**
10 **5x5** filters with stride 1, pad 2



Number of parameters in this layer?
each filter has $5*5*3 + 1 = 76$ params (+1 for bias)
 $\Rightarrow 76*10 = 760$

卷积神经网络：

输入大小为： $W_1 \times H_1 \times D_1$

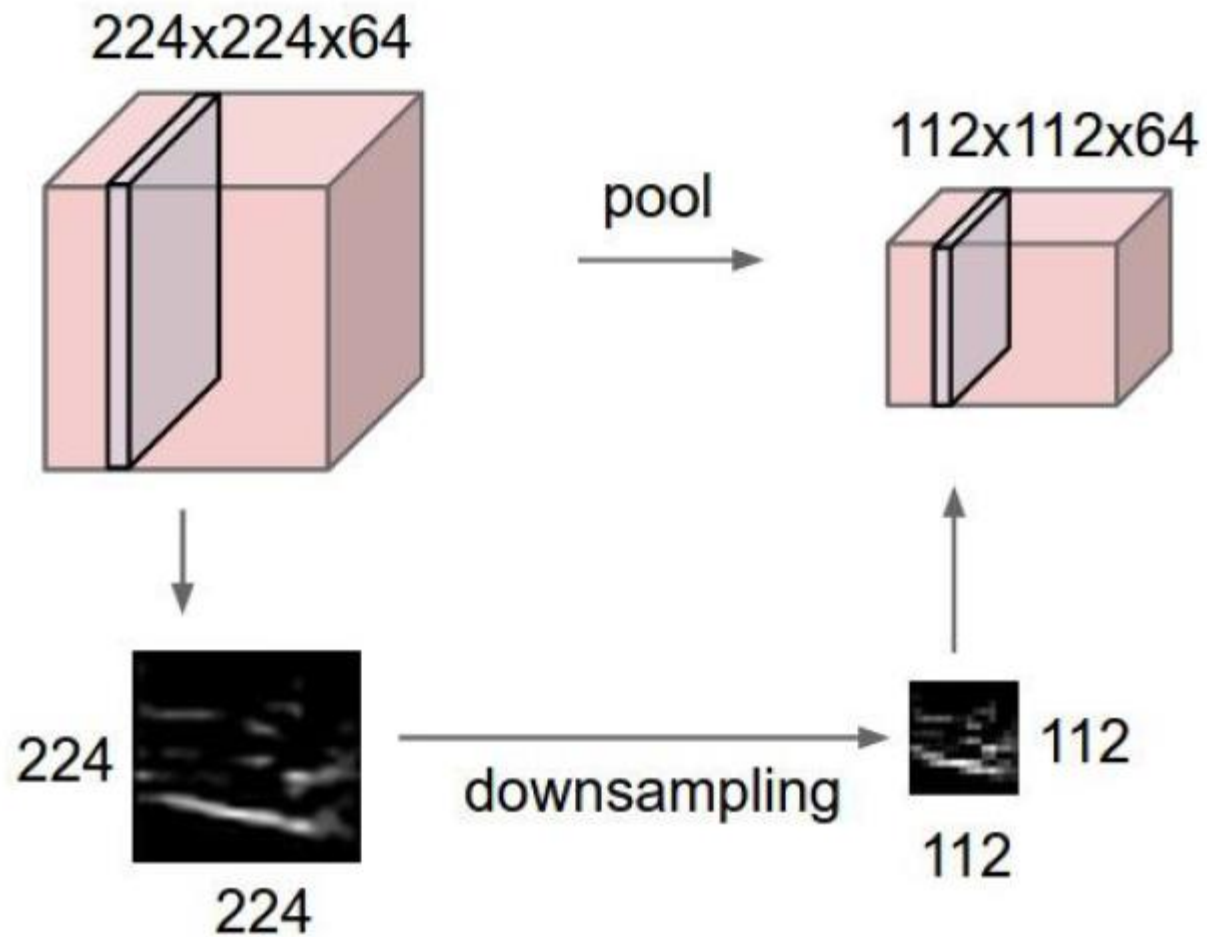
需要指定的超参数： filter个数（K）， filter大小（F）， 步长（S）， 边界填充（P）

输出：

$$W_2 = (W_1 - F + 2P) / S + 1$$
$$H_2 = (H_1 - F + 2P) / S + 1$$
$$D_2 = K$$

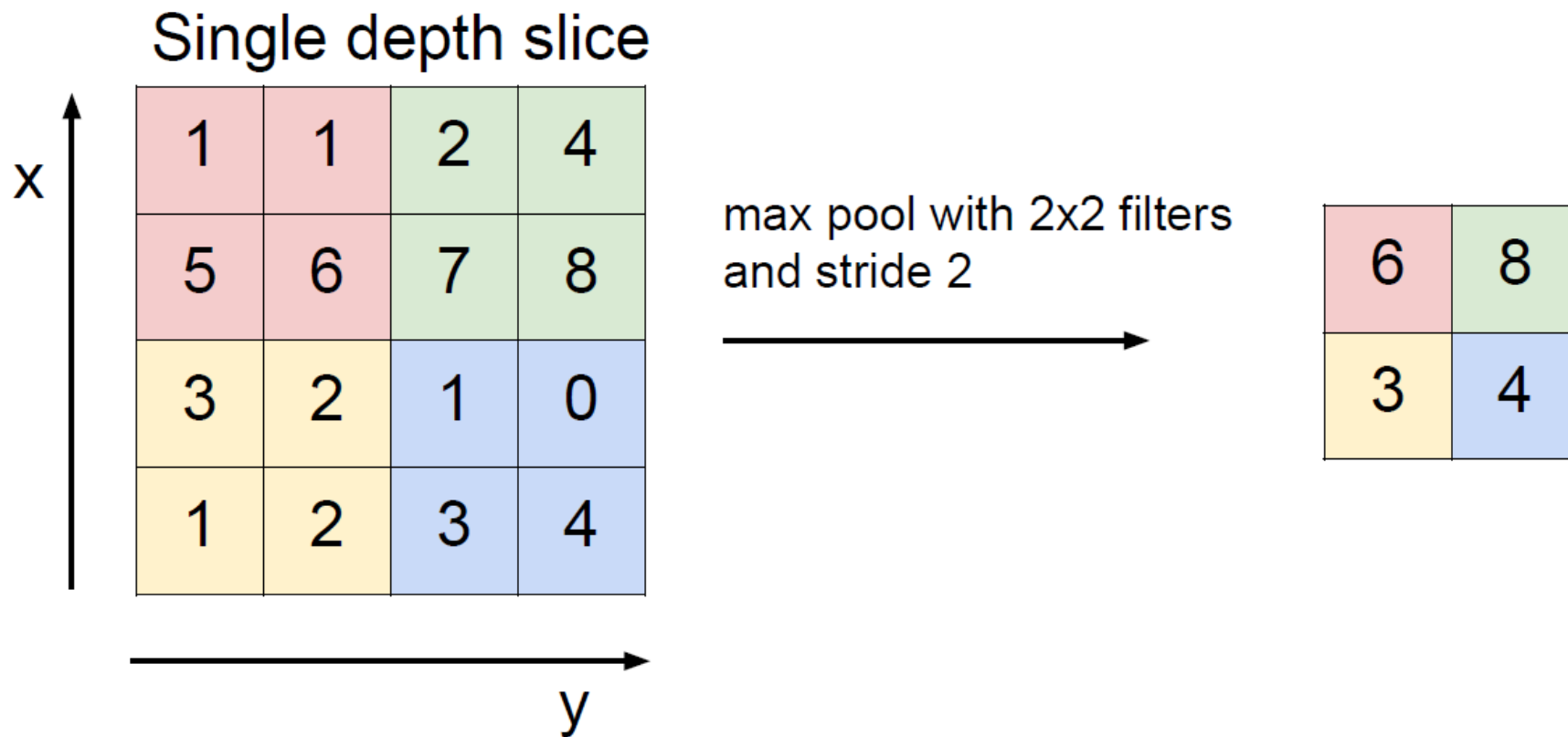
卷积神经网络:

Pooling layer

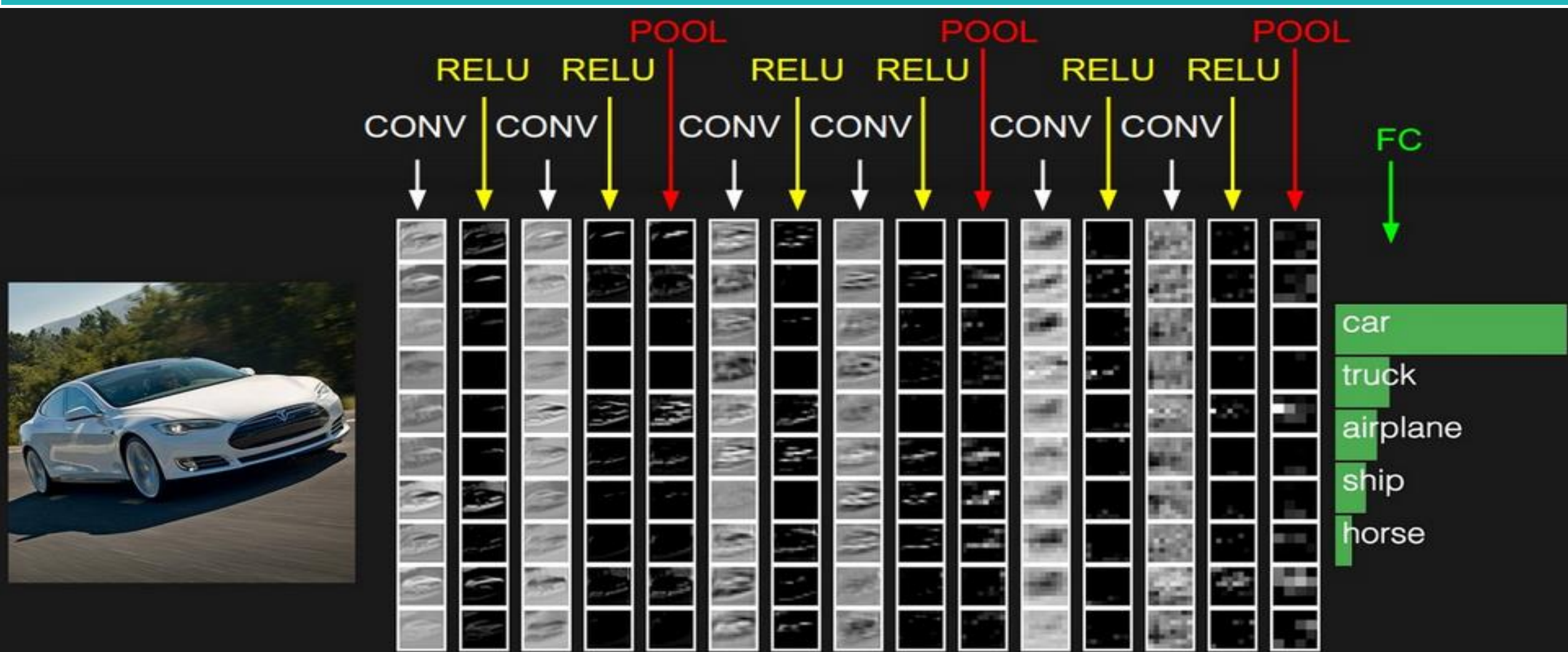


卷积神经网络:

MAX POOLING

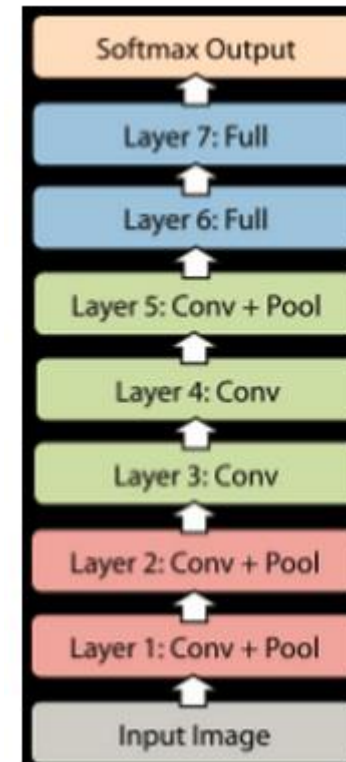
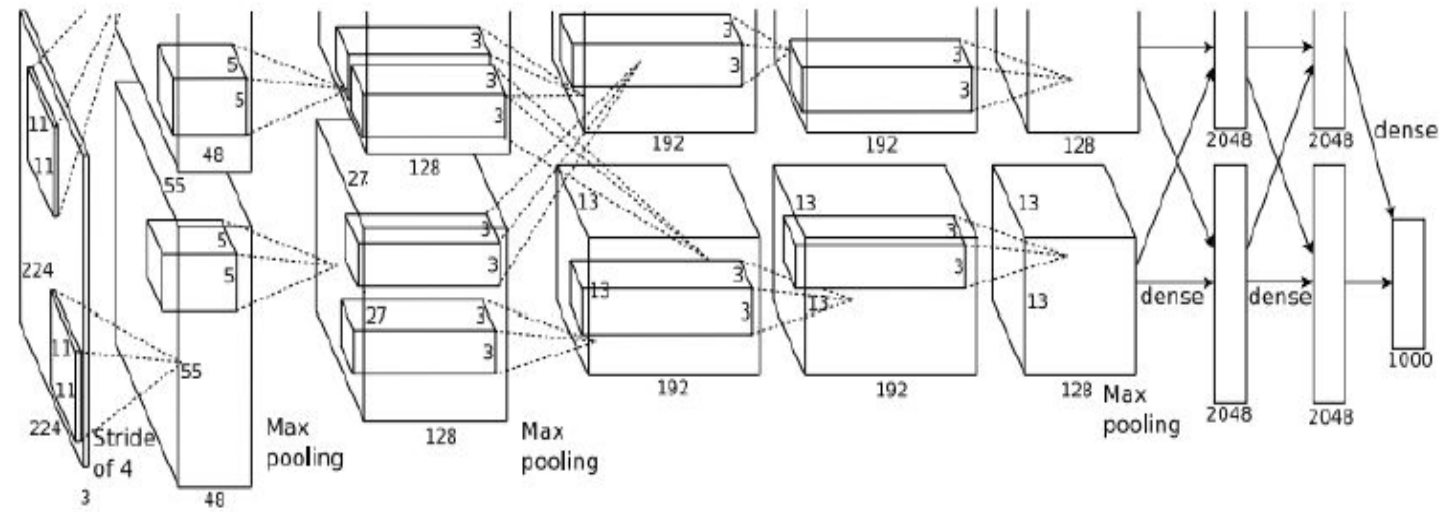


卷积神经网络:



Case Study: AlexNet

[Krizhevsky et al. 2012]



Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] **CONV1**: 96 11x11 filters at stride 4, pad 0

[27x27x96] **MAX POOL1**: 3x3 filters at stride 2

[27x27x96] **NORM1**: Normalization layer

[27x27x256] **CONV2**: 256 5x5 filters at stride 1, pad 2

[13x13x256] **MAX POOL2**: 3x3 filters at stride 2

[13x13x256] **NORM2**: Normalization layer

[13x13x384] **CONV3**: 384 3x3 filters at stride 1, pad 1

[13x13x384] **CONV4**: 384 3x3 filters at stride 1, pad 1

[13x13x256] **CONV5**: 256 3x3 filters at stride 1, pad 1

[6x6x256] **MAX POOL3**: 3x3 filters at stride 2

[4096] **FC6**: 4096 neurons

[4096] **FC7**: 4096 neurons

[1000] **FC8**: 1000 neurons (class scores)

Case Study: VGGNet

[Simonyan and Zisserman, 2014]

Only 3x3 CONV stride 1, pad 1
and 2x2 MAX POOL stride 2

best model

11.2% top 5 error in ILSVRC 2013

->

7.3% top 5 error

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: **Number of parameters** (in millions).

Network	A, A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

INPUT: [224x224x3] memory: $224*224*3=150K$ params: 0 (not counting biases)

CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*3)*64 = 1,728$

CONV3-64: [224x224x64] memory: $224*224*64=3.2M$ params: $(3*3*64)*64 = 36,864$

POOL2: [112x112x64] memory: $112*112*64=800K$ params: 0

CONV3-128: [112x112x128] memory: $112*112*128=1.6M$ params: $(3*3*64)*128 = 73,728$

CONV3-128: [112x112x128] memory: $112*112*128=1.6M$ params: $(3*3*128)*128 = 147,456$

POOL2: [56x56x128] memory: $56*56*128=400K$ params: 0

CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*128)*256 = 294,912$

CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*256)*256 = 589,824$

CONV3-256: [56x56x256] memory: $56*56*256=800K$ params: $(3*3*256)*256 = 589,824$

POOL2: [28x28x256] memory: $28*28*256=200K$ params: 0

CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*256)*512 = 1,179,648$

CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [28x28x512] memory: $28*28*512=400K$ params: $(3*3*512)*512 = 2,359,296$

POOL2: [14x14x512] memory: $14*14*512=100K$ params: 0

CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$

CONV3-512: [14x14x512] memory: $14*14*512=100K$ params: $(3*3*512)*512 = 2,359,296$

POOL2: [7x7x512] memory: $7*7*512=25K$ params: 0

FC: [1x1x4096] memory: 4096 params: $7*7*512*4096 = 102,760,448$

FC: [1x1x4096] memory: 4096 params: $4096*4096 = 16,777,216$

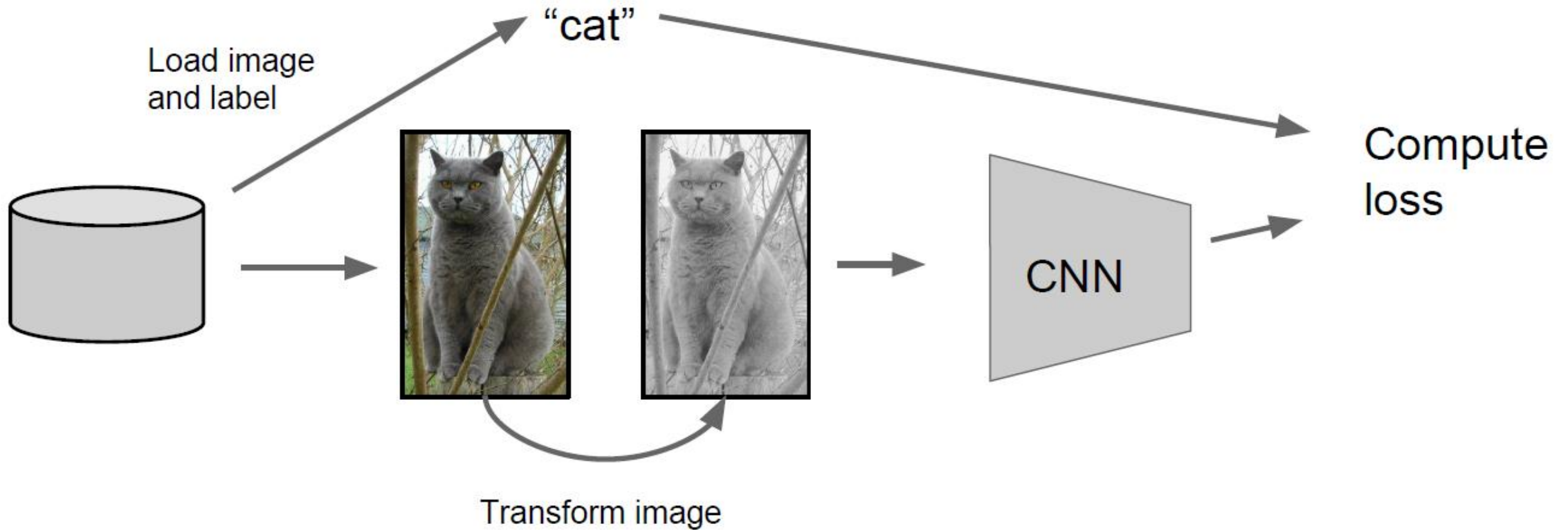
FC: [1x1x1000] memory: 1000 params: $4096*1000 = 4,096,000$

TOTAL memory: $24M * 4 \text{ bytes} \sim 93MB$ / image (only forward! ~ 2 for bwd)

TOTAL params: 138M parameters

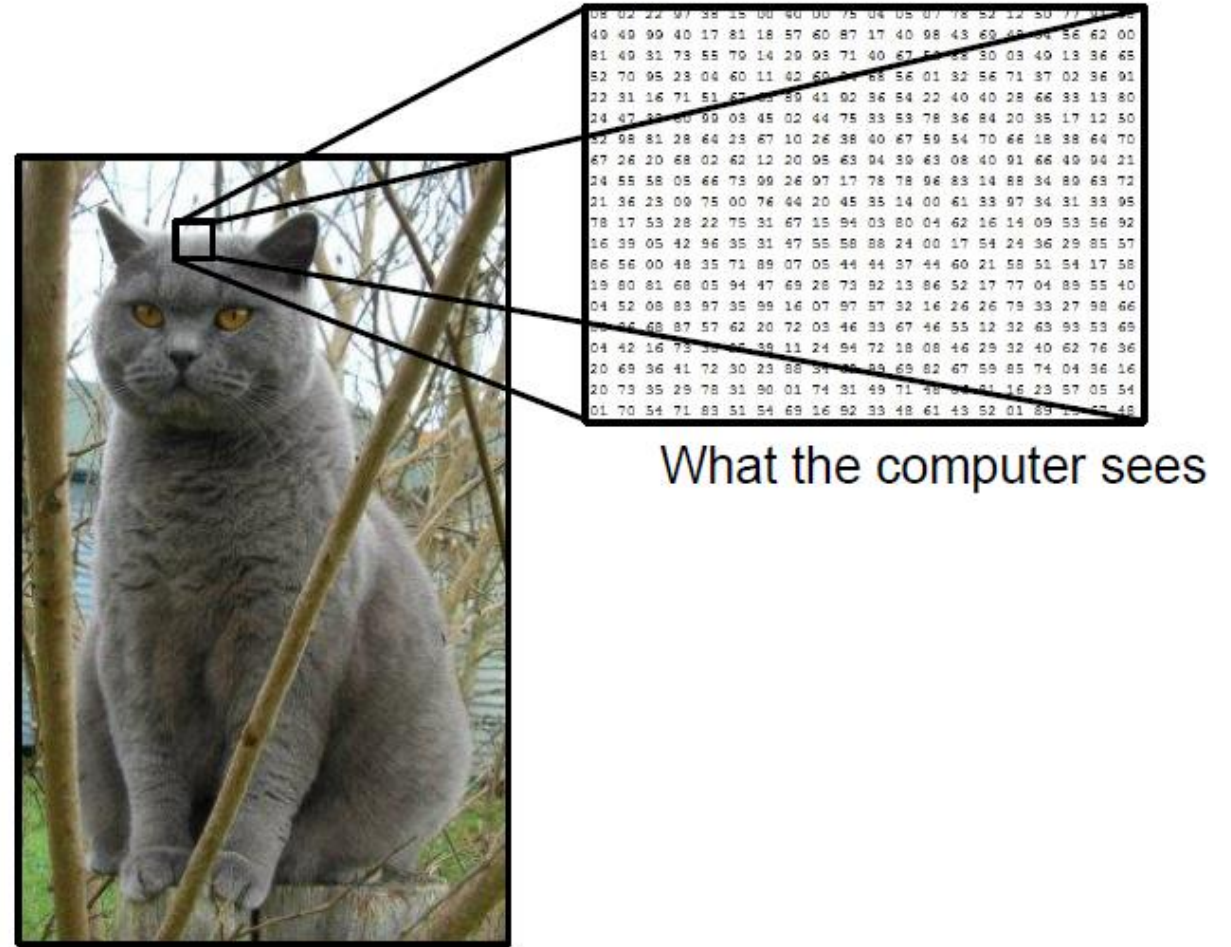
ConvNet Configuration			
B	C	D	
13 weight layers	16 weight layers	16 weight layers	19
put (224 × 224 RGB image)			
conv3-64	conv3-64	conv3-64	cc
conv3-64	conv3-64	conv3-64	cc
maxpool			
conv3-128	conv3-128	conv3-128	co
conv3-128	conv3-128	conv3-128	co
maxpool			
conv3-256	conv3-256	conv3-256	co
conv3-256	conv3-256	conv3-256	co
	conv1-256	conv3-256	co
			co
maxpool			
conv3-512	conv3-512	conv3-512	co
conv3-512	conv3-512	conv3-512	co
	conv1-512	conv3-512	co
			co
maxpool			
conv3-512	conv3-512	conv3-512	co
conv3-512	conv3-512	conv3-512	co
	conv1-512	conv3-512	co
			co
maxpool			
FC-4096			
FC-4096			
FC-1000			
soft-max			

Data Augmentation:



Data Augmentation:

- Change the pixels without changing the label
- Train on transformed data
- VERY widely used



Data Augmentation:

1. Horizontal flips



Data Augmentation:

2. Random crops/scales

Training: sample random crops / scales

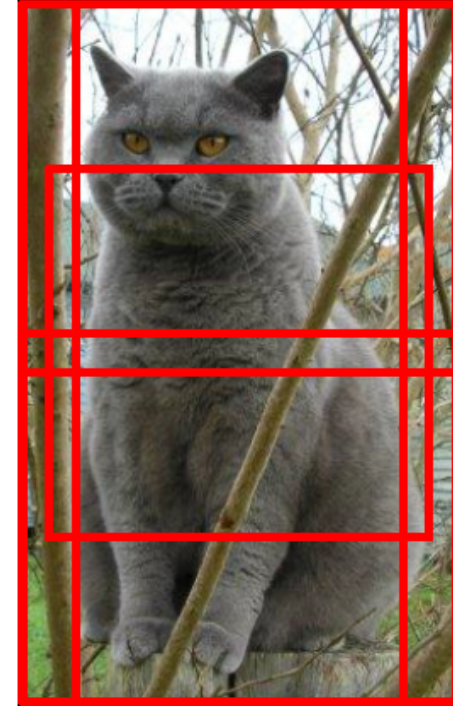
ResNet:

1. Pick random L in range $[256, 480]$
2. Resize training image, short side = L
3. Sample random 224×224 patch

Testing: average a fixed set of crops

ResNet:

1. Resize image at 5 scales: $\{224, 256, 384, 480, 640\}$
2. For each size, use 10 224×224 crops: 4 corners + center, + flips



Data Augmentation:

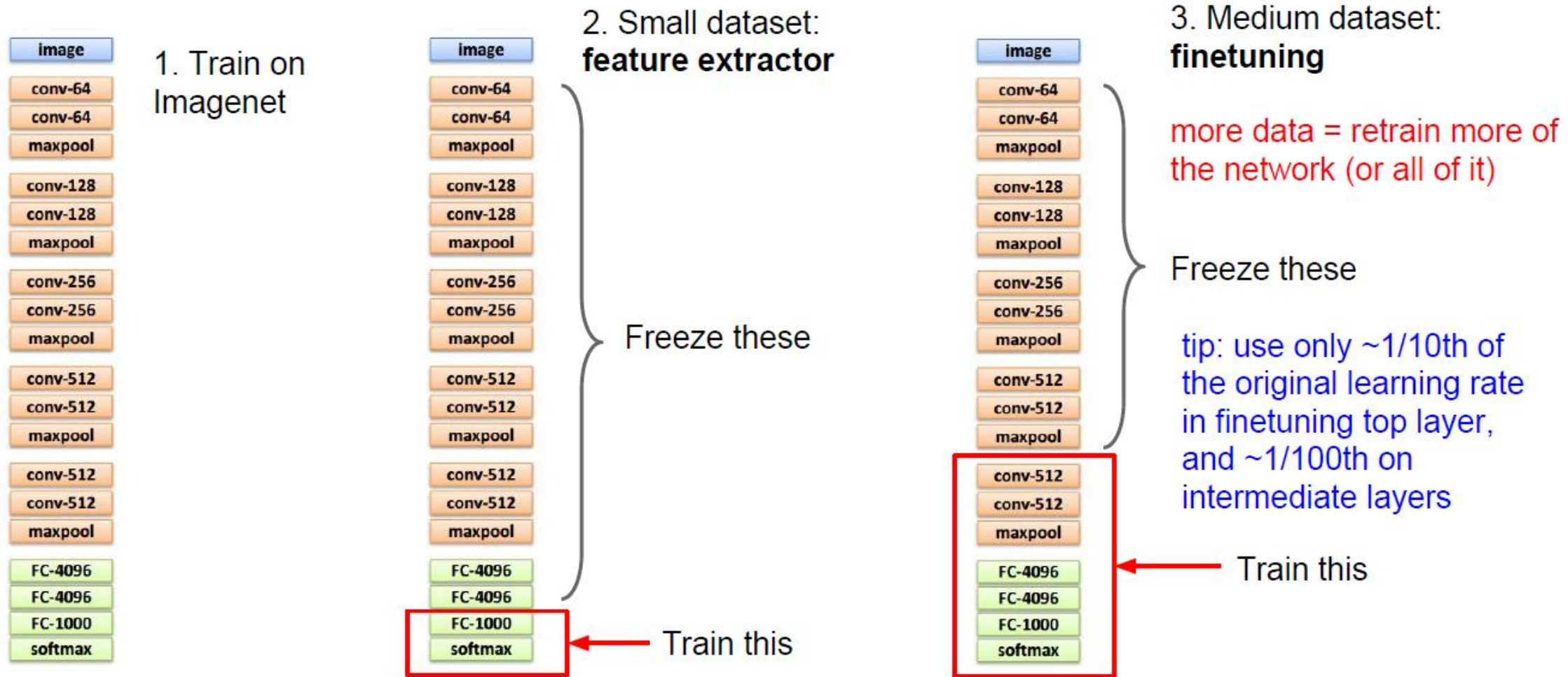
Random mix/combinations of :

- translation
- rotation
- stretching
- shearing,
- lens distortions, ... (go crazy)

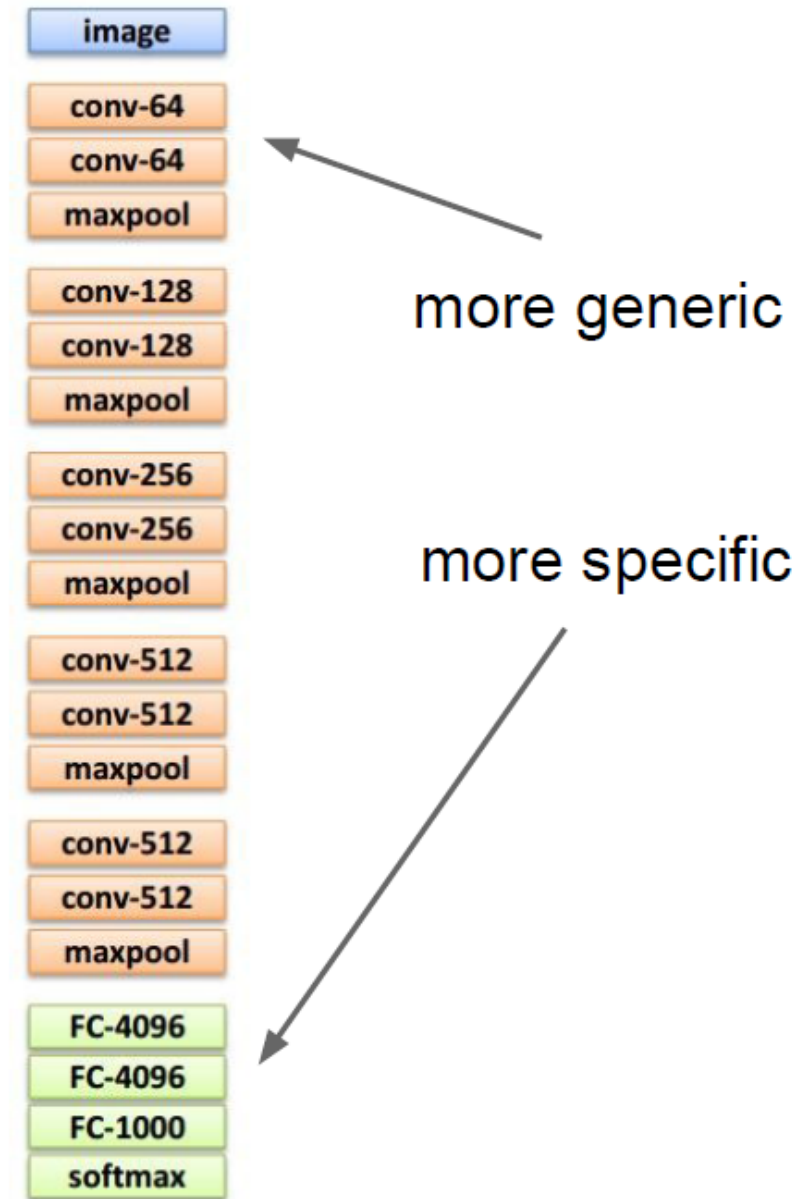
Transfer Learning:

“You need a lot of a data if you want to train/use CNNs”

Transfer Learning:



Transfer Learning:



	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer	You're in trouble... Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers

Transfer Learning:

Takeaway for your projects/beyond:

Have some dataset of interest but it has $< \sim 1\text{M}$ images?

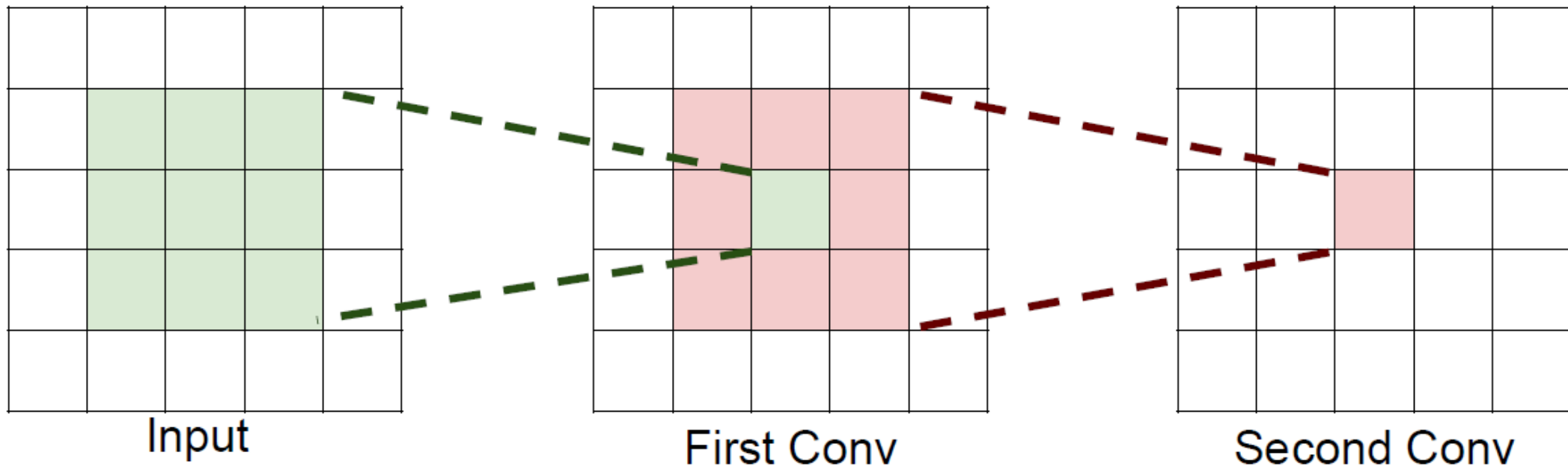
1. Find a very large dataset that has similar data, train a big ConvNet there.
2. Transfer learn to your dataset

Caffe ConvNet library has a “**Model Zoo**” of pretrained models:

<https://github.com/BVLC/caffe/wiki/Model-Zoo>

Design Conv Network:

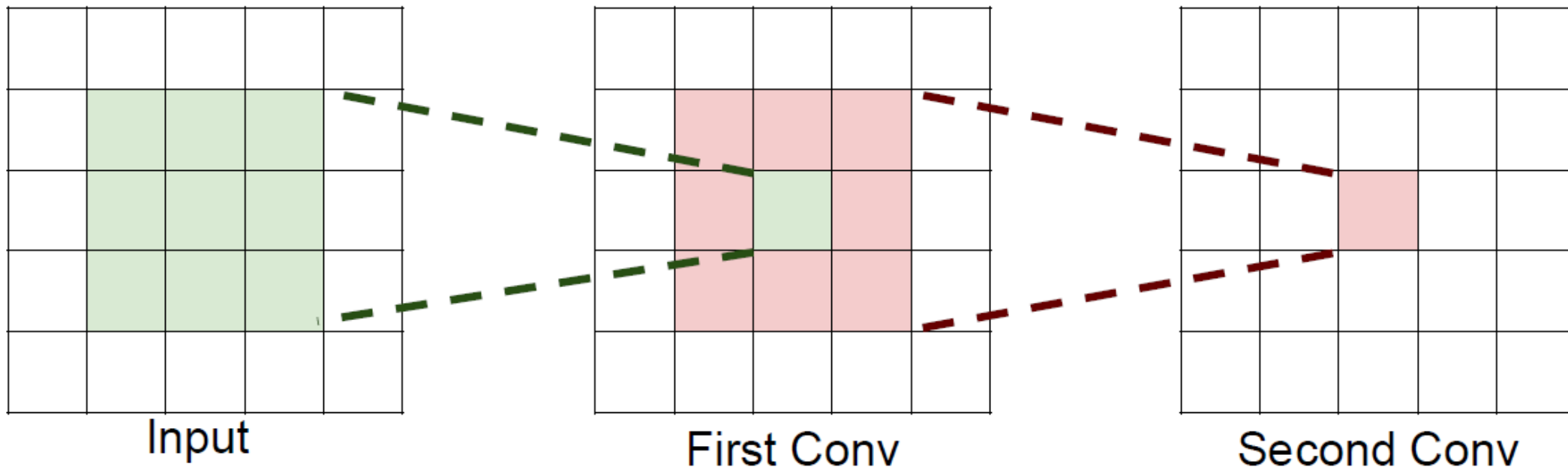
Suppose we stack two 3x3 conv layers (stride 1)
Each neuron sees 3x3 region of previous activation map



Design Conv Network:

Question: How big of a region in the input does a neuron on the second conv layer see?

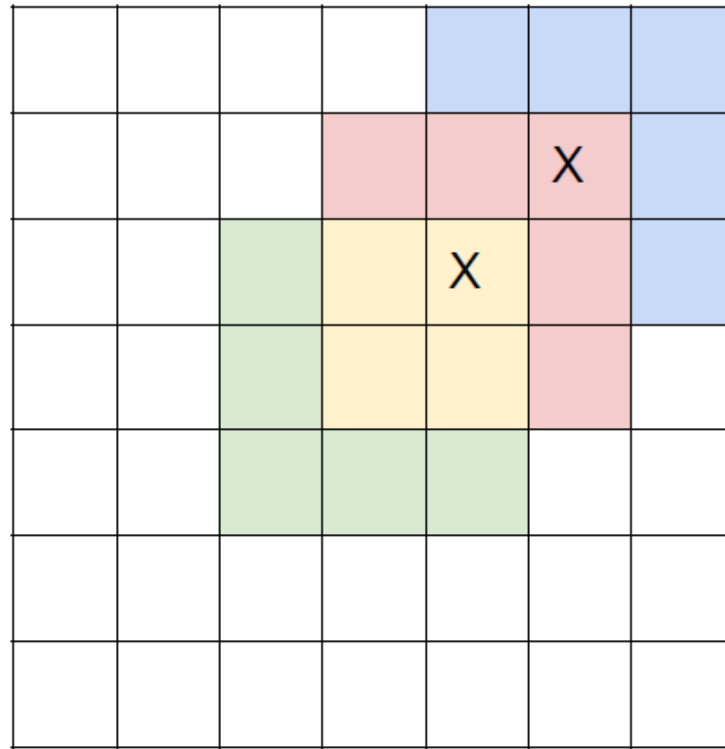
Answer: 5×5



Design Conv Network:

Question: If we stack **three** 3x3 conv layers, how big of an input region does a neuron in the third layer see?

Answer: 7 x 7



Three 3 x 3 conv
gives similar
representational
power as a single
7 x 7 convolution

Design Conv Network:

Suppose input is $H \times W \times C$ and we use convolutions with C filters to preserve depth (stride 1, padding to preserve H, W)

one CONV with 7×7 filters

Number of weights:
 $= C \times (7 \times 7 \times C) = 49 C^2$

three CONV with 3×3 filters

Number of weights:
 $= 3 \times C \times (3 \times 3 \times C) = 27 C^2$

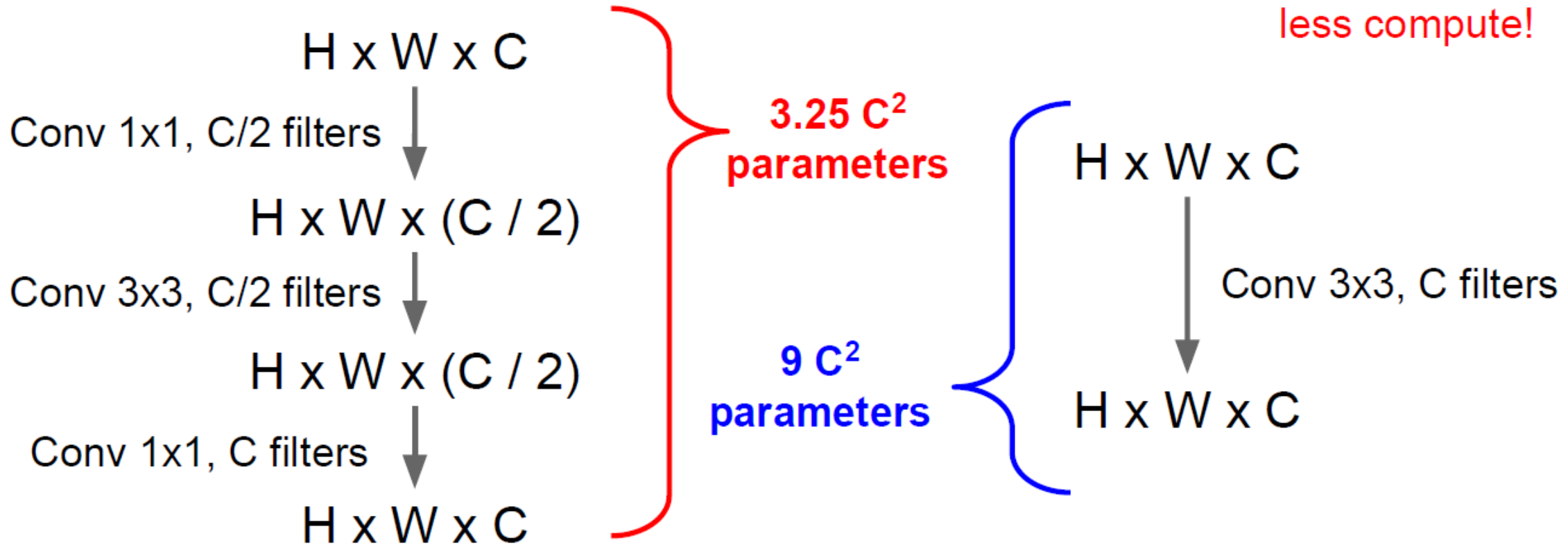
Fewer parameters, more nonlinearity = GOOD



Design Conv Network:

Why stop at 3 x 3 filters? Why not try 1 x 1?

More nonlinearity,
fewer params,
less compute!



Design Conv Network:

- Replace large convolutions (5×5 , 7×7) with stacks of 3×3 convolutions
- 1×1 “bottleneck” convolutions are very efficient
- Can factor $N \times N$ convolutions into $1 \times N$ and $N \times 1$
- All of the above give fewer parameters, less compute, more nonlinearity

GPU VS CPU:

Spot the CPU!
“central processing unit”



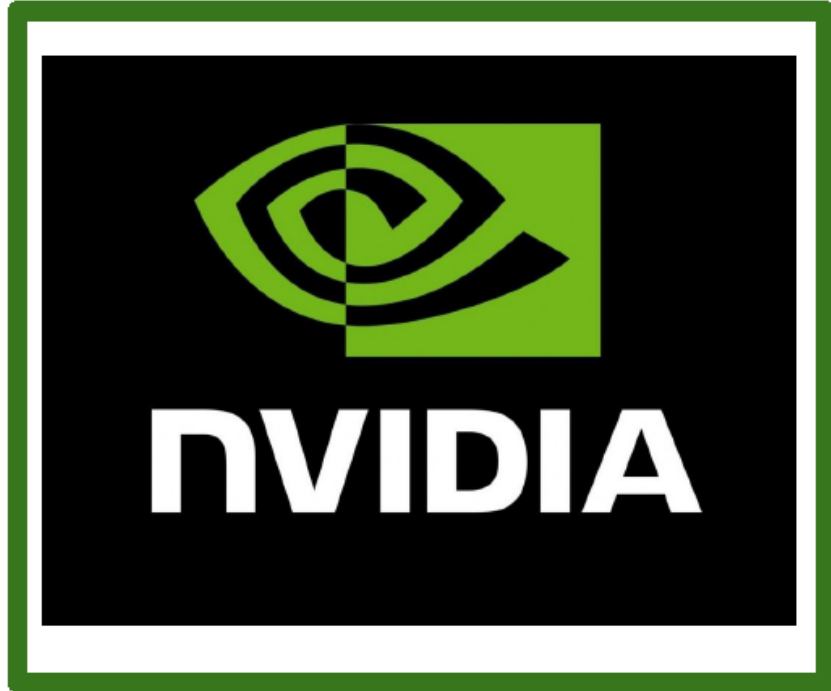
GPU VS CPU:

Spot the GPU!

“graphics processing unit”



GPU VS CPU:



VS



NVIDIA is much more
common for deep learning

GPU VS CPU:

CPU

Few, fast cores (1 - 16)

Good at sequential processing



GPU

Many, slower cores (thousands)

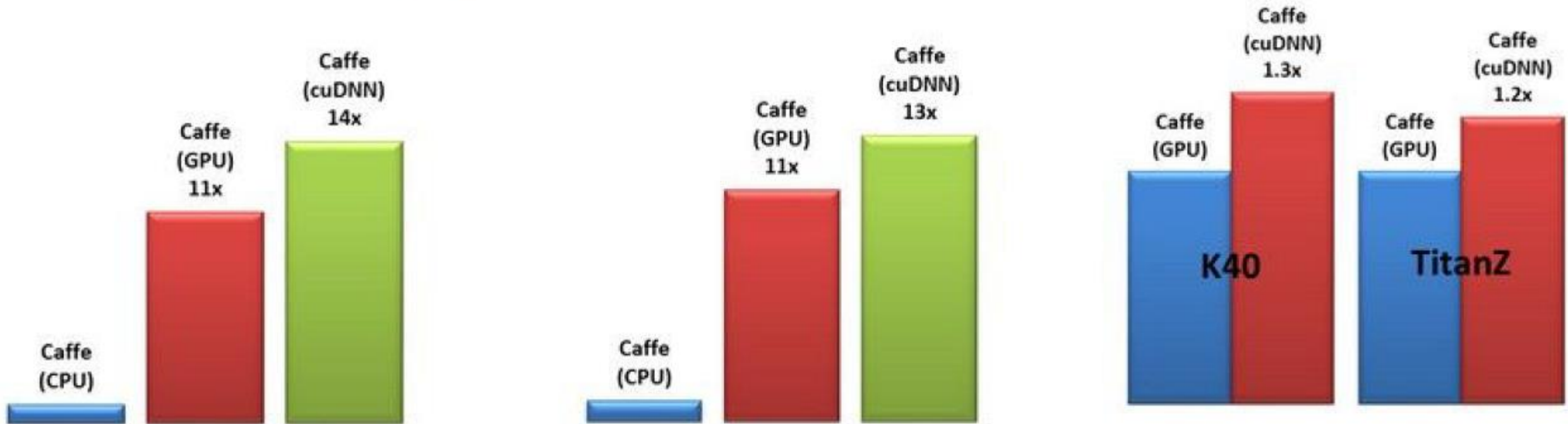
Originally for graphics

Good at parallel computation

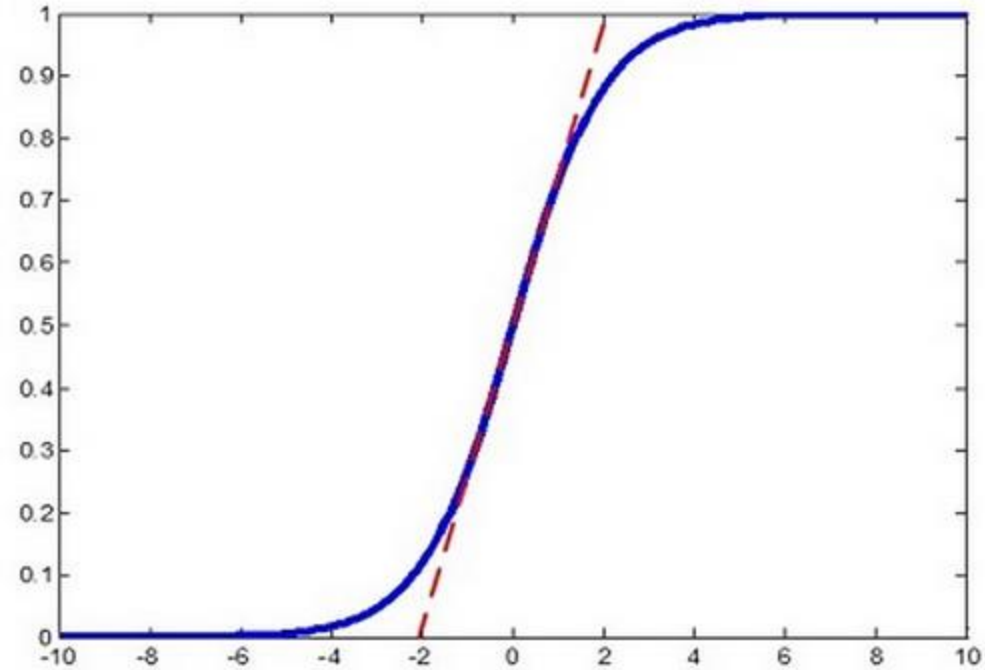


GPU VS CPU:

GPUs are really good at convolution (cuDNN):



Batch Normalization:



Caffe:

Caffe

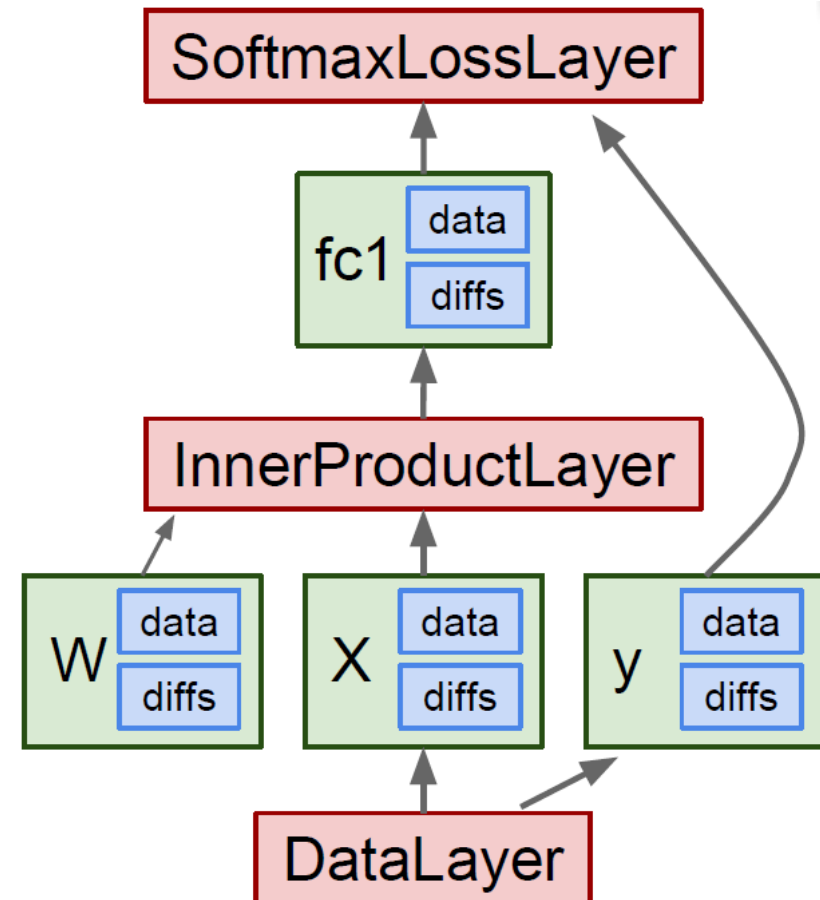
<http://caffe.berkeleyvision.org>

Caffe Overview:

- From U.C. Berkeley
- Written in C++
- Has Python and MATLAB bindings
- Good for training or finetuning feedforward models

Caffe Main Classes:

- **Blob**: Stores data and derivatives ([header source](#))
- **Layer**: Transforms bottom blobs to top blobs ([header + source](#))
- **Net**: Many layers; computes gradients via forward / backward ([header source](#))
- **Solver**: Uses gradients to update weights ([header source](#))



Caffe Training:

No need to write code!

1. Convert data (run a script)
2. Define net (edit prototxt)
3. Define solver (edit prototxt)
4. Train (with pretrained weights) (run a script)

Caffe Step 1: Convert Data

- DataLayer reading from LMDB is the easiest
- Create LMDB using [convert_imageset](#)
- Need text file where each line is
 - “[path/to/image.jpeg] [label]”
- Create HDF5 file yourself using h5py

Caffe Step 2: Define Net

```
name: "LogisticRegressionNet"
layers {
  top: "data"
  top: "label"
  name: "data"
  type: HDF5_DATA
  hdf5_data_param {
    source: "examples/hdf5_classification/data/train.txt"
    batch_size: 10
  }
  include {
    phase: TRAIN
  }
}
layers {
  bottom: "data"
  top: "fc1"
  name: "fc1"
  type: INNER_PRODUCT
  blobs_lr: 1
  blobs_lr: 2
  weight_decay: 1
  weight_decay: 0
```

```
  inner_product_param {
    num_output: 2
    weight_filler {
      type: "gaussian"
      std: 0.01
    }
    bias_filler {
      type: "constant"
      value: 0
    }
  }
}
layers {
  bottom: "fc1"
  bottom: "label"
  top: "loss"
  name: "loss"
  type: SOFTMAX_LOSS
}
```

Caffe Step 3: Define Solver

- Write a prototxt file defining a [SolverParameter](#)
- If finetuning, copy existing solver. prototxt file
 - Change net to be your net
 - Change snapshot_prefix to your output
 - Reduce base learning rate (divide by 100)
 - Maybe change max_iter and snapshot

```
1 net: "models/bvlc_alexnet/train_val.prototxt"
2 test_iter: 1000
3 test_interval: 1000
4 base_lr: 0.01
5 lr_policy: "step"
6 gamma: 0.1
7 stepsize: 100000
8 display: 20
9 max_iter: 450000
10 momentum: 0.9
11 weight_decay: 0.0005
12 snapshot: 10000
13 snapshot_prefix: "models/bvlc_alexnet/caffe_alexnet_train"
14 solver_mode: GPU
```

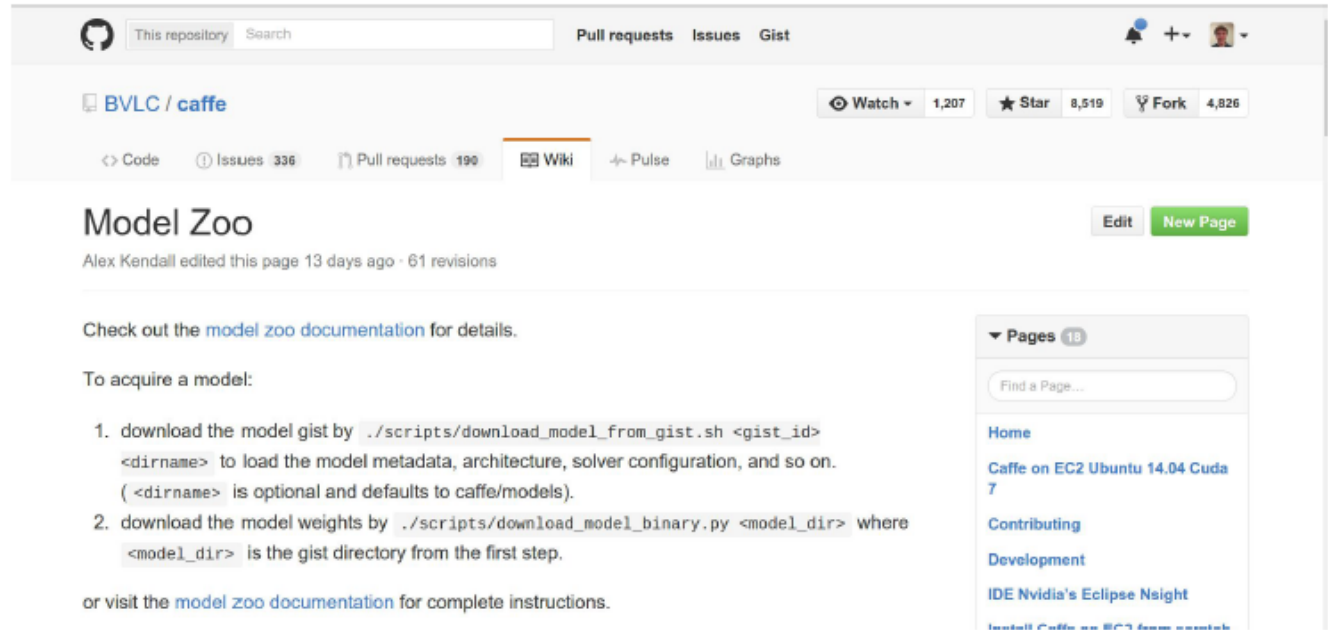
Caffe Step 4: Train!

```
./build/tools/caffe train \  
-gpu 0 \  
-model path/to/trainval.prototxt \  
-solver path/to/solver.prototxt \  
-weights path/to/pretrained_weights.caffemodel
```

Caffe:

Caffe: Model Zoo

AlexNet, VGG,
GoogLeNet, ResNet,
plus others



The screenshot shows the GitHub repository for BVLC / caffe. The repository has 1,207 watches, 8,519 stars, and 4,826 forks. The 'Wiki' tab is selected, showing the 'Model Zoo' page. The page was last edited by Alex Kendall 13 days ago with 61 revisions. The content includes a link to the model zoo documentation, instructions on how to acquire a model, and a list of two steps: 1. download the model gist by `./scripts/download_model_from_gist.sh <gist_id> <dirname>` to load the model metadata, architecture, solver configuration, and so on. (<dirname> is optional and defaults to caffe/models). 2. download the model weights by `./scripts/download_model_binary.py <model_dir>` where <model_dir> is the gist directory from the first step. The page also includes a sidebar with a 'Pages' section listing 13 pages: Home, Caffe on EC2 Ubuntu 14.04 Cuda 7, Contributing, Development, IDE Nvidia's Eclipse Nsight, and Install Caffe on EC2 from scratch.

Model Zoo

Alex Kendall edited this page 13 days ago · 61 revisions

Check out the [model zoo documentation](#) for details.

To acquire a model:

1. download the model gist by `./scripts/download_model_from_gist.sh <gist_id> <dirname>` to load the model metadata, architecture, solver configuration, and so on. (<dirname> is optional and defaults to caffe/models).
2. download the model weights by `./scripts/download_model_binary.py <model_dir>` where <model_dir> is the gist directory from the first step.

or visit the [model zoo documentation](#) for complete instructions.

Pages 13

Find a Page...

[Home](#)

[Caffe on EC2 Ubuntu 14.04 Cuda 7](#)

[Contributing](#)

[Development](#)

[IDE Nvidia's Eclipse Nsight](#)

[Install Caffe on EC2 from scratch](#)